Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Licenciatura en Ciencias de la Computación



Framework para el análisis de noticias de portales digitales mediante inteligencia artificial generativa

Tesina de grado

Trabajo presentado para optar al título de
Licenciado en Ciencias de la Computación

Autor: Lucio Bassani
Director: Alejandro Sartorio

Rosario, 2026

Índice general

1. Introducción **4**
- 1.1. Motivación 4
- 1.2. Objetivos 6
- 1.3. Responsabilidades transversales del framework 8

2. Marco teórico y revisión sistemática de la literatura **11**
- 2.1. Marco Teórico 11
- 2.2. Revisión Sistemática de la Literatura 14
  - 2.2.1. Preguntas de Investigación 15
  - 2.2.2. Metodología de Búsqueda y Selección 15
  - 2.2.3. Estudios Incluidos 18
- 2.3. Estado del Arte 20
  - 2.3.1. Limitaciones del Procesamiento de Lenguaje Natural (NLP) Tradicional 20
  - 2.3.2. Aplicaciones Computacionales del Agenda-Setting de Segundo Nivel 20
  - 2.3.3. Arquitecturas de Software con IAG para el Procesamiento de Noticias 21
- 2.4. Síntesis Crítica y Brechas en la Investigación 22
- 2.5. Conclusión del Capítulo 25

3. Arquitectura **27**
- 3.1. El estilo Tubos y Filtros 29
  - 3.1.1. Componentes, conectores y patrón estructural 29
  - 3.1.2. Modelo computacional e invariantes 29
  - 3.1.3. Por qué Tubos y Filtros para el procesamiento generativo de noticias 31
- 3.2. El framework como sistema de Tubos y Filtros 34

3.2.1. Vista de conjunto y patrón estructural 35
3.2.2. Componentes: los filtros del framework 38
3.2.3. Conectores: tubos tipados 48
3.2.4. Especializaciones aplicadas: pipelines y tubos tipados . 49
3.2.5. Deformación aplicada: repositorio común sobre PostgreSQL 50
3.2.6. Sintonizadores 51
3.2.7. Verificación de los invariantes esenciales 53
3.2.8. Ventajas y limitaciones de la elección 54
3.2.9. Contratos de datos del pipeline 55
3.3. Síntesis del capítulo 57

4. Implementación del framework en Odoo 61
4.1. Infraestructura preexistente del entorno CAETI/CIM 66
4.1.1. La plataforma y el *addon* institucional 67
4.1.2. Realización del filtro de Extracción 68
4.1.3. El contrato `Noticia` sobre el modelo preexistente . . . 70
4.1.4. Delimitación de lo reutilizado 71
4.2. Stack tecnológico 72
4.3. Materialización del pipeline en el módulo 74
4.3.1. El patrón común de los filtros 76
4.3.2. El estudio como configuración del pipeline 78
4.3.3. Los filtros del pipeline 81
4.4. Síntesis del capítulo 100

5. Mapa de procesos 101
5.1. Proceso general 101
5.2. Recopilación de requerimientos 103
5.3. Scraping 104
5.4. Definición de Contexto 108
5.5. Procesamiento IAG 114
5.6. Visualización previa 116
5.7. Refinamiento iterativo 117
5.8. Presentación de resultados 118
5.9. Síntesis del capítulo 119

6. Ejemplo de aplicación **120**
6.1. Selección del caso 120
6.2. Configuración inicial 120
6.3. Ejecución del scraping 120
6.4. Definición de contexto y template utilizado 120
6.5. Resultados del análisis generativo 120
6.6. Discusión 120

7. Conclusiones y Trabajo a Futuro **121**

A. Tipos de datos del pipeline **122**
A.1. Notación y criterios de diseño 122
A.2. `Noticia` 123
A.3. `Embedding` 124
A.4. `Prompt` 125
A.5. `RespuestaIAG` 126
A.6. `Análisis` 126
A.7. `Veredicto` 127
A.8. `Resultado` 128
A.9. `Template` 128
A.10. Los filtros como transformaciones de estado 129
A.11. El grafo de procedencia 130

Capítulo 1

Introducción

1.1. Motivación

El panorama informativo actual se caracteriza por la proliferación de portales digitales, que ofrecen una amplia variedad de noticias en línea. Hoy día, vivimos en un contexto de sociedades hipermediatizadas (Carlón, 2020), en donde el acceso y análisis de los discursos mediatizados a través de Internet presenta dos limitaciones fundamentales que deben ser sorteadas tanto por investigadores sociales, profesionales de la información como por gestores públicos para determinar y comprender su impacto en la sociedad.

1. La presencia de atributos sustanciales de los discursos (como metadatos descriptivos o técnicos) que residen en la "infraestructura subyacente" (Couldry and Hepp, 2017), siendo inaccesibles al "ojo desnudo", es decir, mediante la observación directa.

2. La magnitud de los paquetes textuales y el volumen de los datos y metadatos asociados demandan análisis pormenorizados para generar conocimiento relevante, una tarea abrumadora sin las herramientas adecuadas.

Esta necesidad se inscribe en un contexto más amplio dentro de las ciencias sociales y humanas, marcado por lo que se ha denominado el "giro computacional" (Berry, 2011). Este giro exige la incorporación de herramientas, técnicas y métodos que permitan enriquecer y sofisticar los procedimientos de análisis a través de su computarización (Meunier, 2021). La relevancia

de este enfoque se extiende también a sectores de gestión gubernamental, donde la información actualizada proveniente de las conversaciones públicas es crucial para la toma de decisiones y el diseño de políticas públicas.

Para superar este desafío, se necesitan instrumentos de análisis de texto más sofisticados que permitan identificar correlaciones entre noticias, acoplamiento y cohesión de los términos, conceptos y significantes. Estos instrumentos proporcionarán a los investigadores un sólido respaldo tecnológico para automatizar y optimizar sus protocolos de estudio, incluyendo la formación de la agenda mediática y la difusión de información de manera más eficiente.

Una de las áreas de aplicación más relevantes de estos estudios se centra en la influencia de los medios de comunicación tradicionales en la configuración de la agenda pública, así como en el creciente impacto de las redes sociales como caja de resonancia de noticias y su llegada a públicos segmentados. Desde la publicación del influyente artículo "La función de establecer la agenda de los medios de comunicación de masas" (McCombs and Shaw, 1972), se ha reconocido el papel fundamental de los medios de comunicación en la configuración de la percepción pública. No obstante, en la última década, la irrupción de las redes sociales, primero con la blogosfera y posteriormente con los sitios de aquellas, ha alterado significativamente este panorama.

El desarrollo de las mediciones y la obtención de indicadores referidos a noticias publicadas en medios digitales se fue consolidando en los últimos tiempos. Para este propósito se utilizan técnicas de análisis y **procesamiento de lenguaje natural** (NLP, por sus siglas en inglés de "Natural Language Processing"); **la minería de texto** (Tan, 1999; Hotho et al., 2005); **ciencia de datos** para la identificación de entidades, relaciones e indicadores claves (KPI, por sus siglas en inglés de "Key Performance Indicator"); **clasificación mediante redes neuronales**, etc. Estas herramientas aunque útiles para ciertas tareas, han demostrado limitaciones importantes para capturar la complejidad de los "funcionamientos discursivos" que son esenciales para el análisis social (Gindin and Busso, 2018). Estas limitaciones incluyen dificultades para gestionar el contexto completo, mantener la atención en partes relevantes del texto, identificar relaciones complejas entre conceptos y adecuar la representación a tareas de análisis específicas, problemas que a menudo surgían con algoritmos previos a las generaciones actuales de inteligencia artificial generativa (en adelante, IAG) basadas en arquitectura transformer (Vaswani et al., 2017).

Por lo tanto, es necesario desarrollar nuevas estrategias de análisis que su-

peren estas restricciones, permitiendo acceder al nivel de los funcionamientos discursivos complejos y aplicando categorías y operaciones propias de diversas disciplinas científicas sociales.

Siguiendo con estos lineamientos, se plantea la hipótesis de la utilización de los servicios de IAG en un proceso original, combinados con técnicas de ciencias de datos y análisis de texto para organizar el análisis de noticias y representación de los resultados. La IAG se presenta como una tecnología prometedora, capaz de generar texto similar al humano y de capturar y reproducir relaciones complejas entre unidades lingüísticas, además de poder proveer cierto nivel de explicaciones sobre el significado de los patrones identificados en los textos analizados. Su aplicabilidad, calidad y confiabilidad se garantizará a través de un framework que involucre la supervisión, configuración, análisis y ecualizaciones por parte de expertos del dominio en diversas áreas de la investigación social.

Para su implementación se utilizarán los siguientes datos de entrada para la construcción de los "prompts" de la IAG a utilizar:

1. Los datos empíricos de las noticias con el agregado de información de contexto. Ejemplo: tipo de portal, georeferencia, importancia de la noticia, ubicación, etc.

2. Los objetivos requeridos para el análisis. Ejemplo: alcance, paradigma de análisis, referencias a tener en cuenta, etc.

3. El tipo de salida o representación del análisis. Ejemplo: narrativa, tablas, gráficos, etc.

1.2. Objetivos

A partir de la motivación y el estado del arte relevados, se define el siguiente objetivo general para el presente trabajo:

> **Diseñar e implementar un framework basado en el estilo arquitectónico de Tubos y Filtros que automatice el análisis de noticias de portales digitales mediante inteligencia artificial generativa, manteniendo trazabilidad del flujo de datos, control editorial por parte del experto del dominio y reproducibilidad del proceso analítico.**

Para la concreción del objetivo general se plantean los siguientes objetivos específicos:

OE1: **Diseñar e implementar un módulo *addon* para Odoo Community que integre el ciclo completo de extracción de noticias, composición de *prompts* mediante *templates* parametrizables, invocación de servicios generativos, verificación editorial de resultados y consolidación analítica, satisfaciendo los requisitos de gobernanza y trazabilidad documental identificados en la revisión sistemática.**

El módulo se integra como *addon* sobre la suite *Odoo Community*[1] de código abierto, aprovechando la infraestructura preexistente del entorno CAE-TI/CIM. Provee servicios de gestión para los valores de parametrización de los algoritmos que intervienen en el pipeline, y expone al experto del dominio —a través de formularios— la configuración de portales, reglas de extracción, *templates* de análisis y criterios de verificación. Se desarrollarán integraciones con los módulos de informes y tableros de Odoo para la visualización de los resultados consolidados.

OE2: **Modelar el ciclo de análisis como un mapa de procesos inspirado en la notación BPMN, que documente la secuencia de actividades, los roles intervinientes —experto del dominio, implementador y componentes del sistema— y las entradas y salidas de cada etapa, favoreciendo la reproducibilidad procedimental y estableciendo la correspondencia entre los procesos del ciclo y los filtros de la arquitectura propuesta.**

Este objetivo formaliza la hoja de ruta procedimental que organiza el trabajo del experto del dominio y del sistema, y constituye el puente entre la caracterización arquitectónica (Capítulo 3) y la operación concreta del framework. Las actividades se representan mediante diagramas inspirados en

[1] `https://www.odoo.com/es_ES`

la notación BPMN y se documentan con sus objetivos, alcance, entradas y salidas.

OE3: **Caracterizar el pipeline de procesamiento generativo bajo el estilo arquitectónico de Tubos y Filtros, especificando los filtros, conectores, sintonizadores y tipos de datos del sistema, y verificando formalmente los invariantes esenciales del estilo sobre el diseño resultante.**

El pipeline de procesamiento se fundamenta en el estilo de Tubos y Filtros (Garlan and Shaw, 1994; Shaw and Garlan, 1996), adaptado al caso concreto mediante las especializaciones de *pipeline* y *tubos tipados*, junto con la incorporación de un repositorio común como desviación controlada del estilo. La caracterización formal de cada filtro, de los sintonizadores que parametriza el experto y de los contratos de datos entre etapas —junto con la verificación de los cuatro invariantes esenciales del estilo— constituye el núcleo conceptual del presente trabajo.

OE4: **Validar el framework mediante un caso de aplicación concreto sobre noticias de portales digitales regionales, analizando la calidad de los resultados obtenidos y discutiendo las limitaciones de la propuesta en términos de escalabilidad, fidelidad documental y costo computacional.**

El caso de aplicación materializa el ciclo completo sobre un escenario empírico delimitado —selección de portales, configuración del *template*, ejecución del pipeline y entrega de resultados—, y permite discutir la adecuación del framework a los objetivos planteados, las limitaciones observadas y las oportunidades de trabajo futuro.

1.3. Responsabilidades transversales del framework

El objetivo general compromete al framework con tres garantías (trazabilidad del flujo de datos, control editorial por parte del experto del dominio y

reproducibilidad del proceso analítico) que ningún componente satisface por sí solo: cada una se honra en varios puntos de la arquitectura, del módulo, del mapa de procesos y del caso de aplicación. Para que el lector pueda seguir cada garantía a través de los capítulos sin depender de los nombres propios de cada vista, el documento emplea seis marcadores estables, [A] a [F], que identifican las responsabilidades transversales que el framework asume (Tabla 1.1). Los marcadores nombran obligaciones, no elementos: no son etapas del pipeline —que se identifican por sus filtros y puertos en el Capítulo 3— ni procesos del mapa —numerados en el Capítulo 5—, y una misma responsabilidad se paga en varios de ellos. Cada marcador enlaza al registro maestro de la Tabla 3.2, que consolida su realización capítulo por capítulo.

Tabla 1.1: Responsabilidades transversales del framework, obligación que cada una nombra y objetivos específicos de los que deriva. Los marcadores identifican responsabilidades, no componentes ni etapas.

Marcador	**Responsabilidad**	**Obligación que nombra**	**OE**
[A]	Intervención del experto	Toda decisión de dominio la toma el experto del dominio por una vía explícita, separada del flujo de datos.	OE1, OE2
[B]	Independencia del estudio	El framework no conoce el estudio: tema, pregunta, paradigma, esquema de respuesta y criterios de validez y consolidación se declaran como datos del estudio, no como código del framework.	OE3, OE4
[C]	Frontera generativa	La dependencia de los servicios de modelos externos —generativo y de *embedding*— queda aislada y sustituible, y su salida ingresa al sistema de manera agnóstica al proveedor.	OE1, OE3
[D]	Control editorial	Ninguna afirmación generada llega al experto sin un juicio de fidelidad, una medida de confianza y la evidencia que la sustenta; la decisión editorial última es humana.	OE1, OE4
[E]	Procedencia	Desde cualquier resultado puede reconstruirse a partir de qué datos, con qué configuración y cuándo se produjo, y nada de esa historia se elimina.	OE1, OE3
[F]	Infraestructura preexistente	Queda delimitado qué reutiliza el framework del entorno institucional preexistente y qué construye la presente tesina.	OE1

Capítulo 2

Marco teórico y revisión sistemática de la literatura

En el contexto de un ecosistema mediático caracterizado por la hipermediatización y la proliferación de datos, los métodos tradicionales de análisis de contenido enfrentan desafíos sin precedentes. Este capítulo aborda las bases teóricas y técnicas necesarias para fundamentar una infraestructura que automatice el análisis de noticias digitales. Se presenta primero el marco teórico que define el campo conceptual; luego se documenta la revisión sistemática de la literatura (RSL) que guió la selección de fuentes, junto con las preguntas de investigación, la metodología empleada y los estudios incluidos; finalmente se analiza el estado del arte en tres ejes temáticos, se sintetizan las brechas detectadas y se cierra con las implicancias para esta tesina.

2.1. Marco Teórico

Según Luhmann (2000, p. 1): "*Todo lo que sabemos acerca de nuestra sociedad, o más aún, acerca del mundo en el que vivimos, lo conocemos a través de los medios de comunicación masiva.*"[1] Esta afirmación no es una observación trivial sobre el flujo de información: Luhmann (2000) sostiene que los medios masivos constituyen un sistema social funcionalmente diferenciado que *produce* la realidad social en lugar de meramente reflejarla. Si lo que conocemos sobre el mundo nos llega mediado por instituciones cuyo criterio de selección

[1]Las traducciones de citas originalmente en otros idiomas son propias del autor, salvo indicación contraria.

obedece a su propia lógica operativa, entonces el estudio de esa mediación se vuelve política y científicamente ineludible.

La forma más influyente en que se ha estudiado empíricamente este fenómeno es la teoría del *agenda-setting*. McCombs and Shaw (1972) formularon la hipótesis de que "*los medios masivos establecen la agenda de cada campaña política, influyendo en la prominencia de las actitudes hacia los temas políticos*". Recuperando una observación previa de Cohen, los autores sintetizaron este efecto en una fórmula que se volvió canónica: la prensa "*puede no ser exitosa la mayor parte del tiempo en decirle a la gente qué pensar, pero es asombrosamente exitosa en decirle a sus lectores sobre qué pensar*". El estudio empírico de McCombs and Shaw (1972) sobre la elección presidencial estadounidense de 1968 en Chapel Hill encontró correlaciones cercanas a la unidad entre el énfasis temático de los medios y los temas que los votantes consideraban importantes, estableciendo así el primer nivel de la agenda-setting: la transferencia de prominencia (*salience*) desde la agenda mediática hacia la agenda pública.

Dos décadas más tarde, McCombs et al. (1997) extendieron la teoría al notar que los objetos sobre los que los medios construyen agenda –candidatos, temas, eventos– poseen *atributos*, tanto sustantivos como afectivos. Así, "*el primer nivel de la agenda-setting es la transmisión de la prominencia de los objetos; el segundo nivel de la agenda-setting es la transmisión de la prominencia de los atributos*" (McCombs et al., 1997, p. 704). Esta extensión vincula la agenda-setting con el concepto de *framing* formalizado por Entman (1993): enmarcar es "*seleccionar algunos aspectos de una realidad percibida y hacerlos más prominentes en un texto comunicativo, de modo de promover una definición particular del problema, una interpretación causal, una evaluación moral y/o una recomendación de tratamiento para el asunto descrito*" (Entman, 1993, p. 52). Enmarcar, entonces, es seleccionar qué atributos de un objeto se vuelven prominentes en un texto comunicativo. La famosa fórmula de Cohen se reescribe en consecuencia: los medios no solo nos dicen "*sobre qué pensar, sino que también nos dicen cómo pensar al respecto*" (McCombs et al., 1997, p. 716). Este segundo nivel sustenta los estudios contemporáneos que miden computacionalmente el sesgo y el sentimiento en la cobertura mediática, analizados más adelante en la Sección 2.3.

El paisaje en el que opera la agenda-setting cambió radicalmente desde McCombs y Shaw. Couldry and Hepp (2017) caracterizan la situación contemporánea como *deep mediatization*, un régimen marcado por tres procesos simultáneos: la *datafication* (cada encuentro mediado deja un rastro

de datos), la fragmentación profunda de las audiencias y la incorporación de plataformas y algoritmos a la construcción misma del acontecimiento mediático. En palabras de los autores, "*las construcciones algorítmicas de la realidad pasan a formar parte de la construcción de los eventos mediáticos*" (Couldry and Hepp, 2017, p. 3), transformación que obliga a analizar los procesos mediáticos contemplando el rol de datos y algoritmos. En sintonía con esta caracterización, Carlón (2020) sitúa el concepto de *sociedad hipermediatizada* como el contexto general en el que esta tesina inscribe su objeto de estudio.

Esta transformación del objeto exige una transformación correlativa de los métodos. Berry (2011) identifica este desplazamiento como un *computational turn*: la creciente dependencia de las ciencias sociales y humanidades respecto de técnicas computacionales para acceder a sus objetos. Berry advierte que las técnicas computacionales "*no son meramente un instrumento empuñado por los métodos tradicionales; más bien, tienen efectos profundos sobre todos los aspectos de las disciplinas*" (Berry, 2011, p. 13): introducen nuevos modos de análisis –como el *distant reading* de Moretti recuperado por Berry (2011), según el cual los corpus literarios masivos no pueden comprenderse sumando lecturas individuales sino tratándolos como sistemas colectivos– y reconfiguran las preguntas que las disciplinas pueden formular. Meunier (2021) extiende esta línea hacia la semiótica computacional, ofreciendo un sustrato teórico complementario para la articulación entre código, signos y significado.

El giro computacional aplicado al análisis del lenguaje natural cuenta con un punto de inflexión técnico preciso: la arquitectura *Transformer* propuesta por Vaswani et al. (2017). El aporte distintivo de esta arquitectura no reside en la representación vectorial de las unidades lingüísticas —los *embeddings* eran ya un recurso consolidado del NLP anterior, con familias como Word2Vec (Mikolov et al., 2013), GloVe y FastText— sino en el mecanismo de *self-attention*, una función que relaciona todas las posiciones de una secuencia en paralelo, permitiendo capturar dependencias de largo alcance que las arquitecturas recurrentes y convolucionales previas procesaban de manera secuencial o local. Sobre esta base se construyen los modelos de lenguaje extenso contemporáneos —GPT-3, GPT-4, LaMDA y sus sucesores— capaces de traducir, clasificar, resumir y generar respuestas analíticas complejas a partir de instrucciones en lenguaje natural.

Estos desarrollos teóricos y técnicos encuentran un antecedente empírico relevante en el trabajo de tesis doctoral de Pablo Rey-Mazón, "Color of

Corruption" (Rey-Mazón, 2023), que articula varias herramientas y una metodología novedosas para medir el proceso de *agenda-setting* en un ecosistema mediático complejo. Según Rey-Mazón, la metodología tradicional de análisis de contenido se basa en el supuesto de que el contenido ya está completo y estático para su estudio, lo que ignora el flujo constante de noticias y su influencia sobre la recepción. En contraste, su software **Homepagex** rastrea la posición de los elementos noticiosos en las páginas de inicio durante varios días, permitiendo reconstruir los flujos y ciclos de noticias en los sitios web mediante el cruce entre una base de datos construida por scraping horario y análisis automatizado de contenido. La herramienta complementaria **Color of Corruption** consiste en una base de datos de imágenes de noticias sobre corrupción etiquetada por tono emocional, enfoque narrativo y presencia de actores específicos, soporte de la metodología denominada *Visual Agenda-Setting Analysis*. La cercanía conceptual y metodológica del trabajo de Rey-Mazón con la propuesta de esta tesina lo vuelve una referencia central.

Disponer de modelos generativos no alcanza, sin embargo, para construir una infraestructura utilizable: es necesario orquestar las tecnologías mediante estándares formales de procesos de negocio. Trabajos como el de Prades et al. (2013) establecen metodologías para implementar flujos de trabajo en BPMN de acuerdo con el estándar ANSI/ISA-95, proporcionando un vocabulario normalizado para especificar cada etapa del proceso y los roles intervinientes. La incorporación de modelos generativos dentro de pipelines de procesos automatizados —con el consiguiente desafío de combinar invocación no determinista, verificación de resultados y composición de etapas— es una línea de trabajo activa en ingeniería de software. La articulación entre marcos teóricos, instrumentos computacionales y orquestación procedimental que se propone en esta tesina se inscribe en esa misma línea.

2.2. Revisión Sistemática de la Literatura

Sobre la base del marco teórico anterior, se llevó a cabo una revisión sistemática de la literatura (RSL) orientada a identificar trabajos empíricos y de ingeniería que combinen análisis de noticias, web scraping, embeddings y modelos de lenguaje extenso. Esta sección presenta las preguntas que guiaron la revisión, la metodología de búsqueda y selección, y el conjunto final de estudios incluidos.

2.2.1. Preguntas de Investigación

La revisión se orientó a responder las siguientes preguntas:

PR1: ¿Cuáles son las limitaciones de las técnicas tradicionales de Procesamiento de Lenguaje Natural (NLP) frente a la complejidad del análisis discursivo en medios digitales?

PR2: ¿Qué metodologías y herramientas informáticas se están utilizando para evaluar el agenda-setting y el impacto mediático en la actualidad?

PR3: ¿Cuáles son las brechas estructurales y procedimentales en los pipelines de IAG actuales al integrar web scraping y embeddings para la investigación social?

2.2.2. Metodología de Búsqueda y Selección

La presente revisión se condujo siguiendo las directrices del protocolo PRISMA 2020 (Page et al., 2021), con el propósito de identificar estudios que combinaran análisis de noticias, web scraping, embeddings y modelos de lenguaje extenso (LLM).

La recolección bibliográfica se ejecutó durante mayo de 2026 sobre cinco bases de datos y agregadores académicos: SciSpace (búsqueda básica), SciSpace con filtro de texto completo, SciSpace Library, Google Scholar y ArXiv. A estos se sumó una búsqueda complementaria por *snowballing* sobre la biblioteca personal del autor, utilizada para recuperar trabajos conocidos previamente por el equipo de investigación y evitar la omisión de referencias clave no bien indexadas en los agregadores. La cadena de búsqueda exacta aplicada se detalla en la Figura 2.1. Los conteos por fuente fueron: SciSpace (n=100), SciSpace texto completo (n=100), SciSpace Library (n=8), Google Scholar (n=20), ArXiv (n=20) y *snowballing* sobre biblioteca personal (n=6), totalizando 254 registros identificados.

Los **criterios de inclusión** aplicados fueron:

CI1: Artículos, tesis o actas de conferencias.

CI2: Publicados entre 2020 y 2025.

CI3: Escritos en español o inglés.

CI4: Presentan detalles técnicos sobre la arquitectura de software propuesta.

CI5: Combinan al menos dos de las áreas centrales de la revisión: web scraping, embeddings, LLMs/IAG, agenda mediática.

Complementariamente, los **criterios de exclusión** aplicados fueron:

CE1: Trabajos puramente teóricos sin componente de software.

CE2: Basados en tecnologías pre-transformer u obsoletas.

CE3: Metodología no replicable o sin descripción técnica suficiente.

CE4: Fuera del dominio de análisis de medios digitales o noticias.

CE5: Texto completo no disponible públicamente.

El proceso de selección se desarrolló en cuatro fases ejecutadas por el autor. Primero se eliminaron los duplicados entre bases (n=20), reduciendo el corpus a 234 registros. A continuación se realizó un cribado por título y resumen aplicando los criterios de inclusión, excluyendo 130 registros por baja relevancia y conservando 104. Sobre estos se buscó recuperar el texto completo, descartando 54 informes por no contar con acceso público al documento (CE5). Finalmente, los 50 informes recuperados fueron evaluados a texto completo aplicando los criterios de exclusión, descartando 41: 20 por estar fuera del tema principal (CE4), 12 por carecer de metodología replicable (CE3) y 9 por idioma no admitido (CI3 incumplido). El conjunto final quedó conformado por 9 estudios, presentados en la Sección 2.2.3 y analizados en la Sección 2.3.

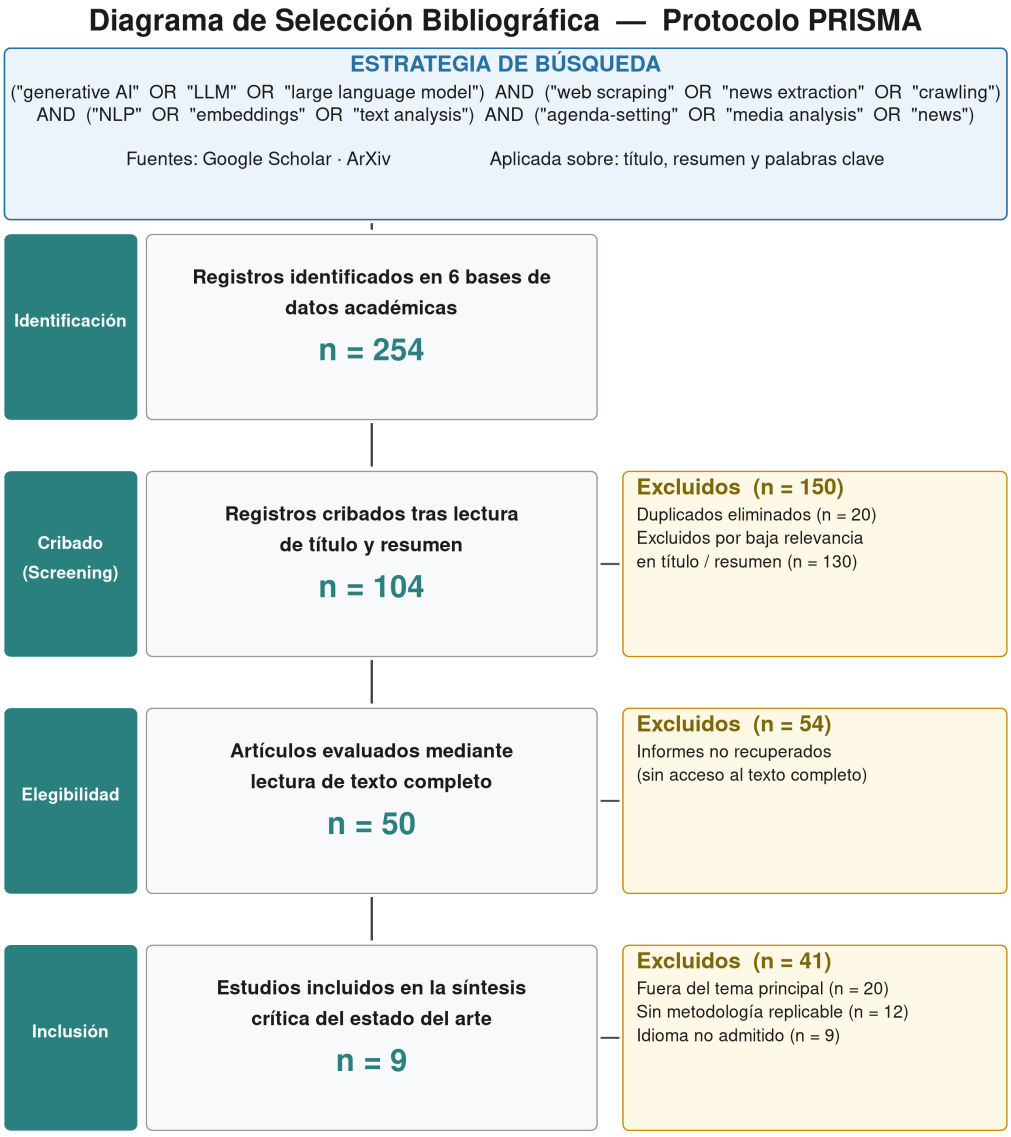


Figura 2.1: Diagrama de selección bibliográfica según el protocolo PRISMA (Page et al., 2021).

2.2.3. Estudios Incluidos

El proceso de selección descrito en la sección anterior arrojó nueve estudios incluidos en la revisión, distribuidos de manera desigual entre los tres ejes temáticos: dos trabajos abordan las limitaciones del NLP tradicional (PR1), dos las aplicaciones computacionales del *agenda-setting* (PR2) y cinco las arquitecturas de software con IAG para el procesamiento de noticias (PR3). Este desbalance refleja la madurez relativa de cada línea: mientras que las dos primeras presentan estudios más bien fundacionales o de validación metodológica, la tercera concentra la mayor parte de la producción reciente, donde proliferan prototipos y sistemas aplicados que combinan extracción, vectorización e inferencia.

Una lectura transversal del corpus revela un patrón arquitectónico recurrente: la mayoría de los trabajos articula una cadena de procesamiento que va de la ingesta de contenido –ya sea por *scraping* directo o por consumo de APIs noticiosas– a la representación semántica mediante *embeddings* o esquemas RAG, y de allí a la inferencia con modelos de lenguaje extenso (Neupane et al., 2025; Behbahanian, s.f.; Somasundharam et al., 2025; Ameer, 2025). Las variantes no afectan la estructura general sino los énfasis: algunos priorizan deduplicación y agregación (Behbahanian, s.f.), otros la entrega multimedia o personalizada (Ameer, 2025; Begum J and D, 2024), y otros la trazabilidad de fuentes (Somasundharam et al., 2025). Esta convergencia técnica confirma que el patrón existe como herencia disponible, pero también vuelve más visibles las dimensiones que ningún estudio resuelve por completo.

En particular, las limitaciones que reportan los propios autores conforman un mapa de zonas no cubiertas que se repiten a lo largo del corpus: la ausencia de validación con usuarios o expertos (Behbahanian, s.f.; Neupane et al., 2025), el sacrificio de la profundidad interpretativa en favor de la distribución o la *performance* (Ameer, 2025; Somasundharam et al., 2025), la dependencia de muestras acotadas o casos únicos (Pavlyshenko, 2024; Samuel et al., 2025) y la falta de estandarización cross-plataforma (Haider et al., 2025). La Tabla 2.1 sintetiza este panorama agrupado por pregunta de investigación; la columna "Estudio" incorpora entre paréntesis el sistema o stack distintivo de cada trabajo.

Tabla 2.1: Estudios incluidos en la revisión sistemática, agrupados por pregunta de investigación.

Estudio	**Aporte central**	**Limitación**
PR1 — Limitaciones del NLP tradicional frente al discurso mediático		
Milbauer et al. (2023) (*NewsSense*)	Evidencia el límite del NLP clásico para sintetizar perspectivas contradictorias y verificar sin referencias externas	Foco en *sensemaking* del lector; no escala a corpus masivos
Pavlyshenko (2024) (RAG GPT-4o + bayesiano)	La IAG supera al NLP estadístico en análisis cualitativo, pero requiere modelado bayesiano para cuantificar la incertidumbre	Muestra acotada; caso único (elecciones US 2024)
PR2 — Aplicaciones computacionales del agenda-setting de segundo nivel		
Haider et al. (2025) (*Media Bias Detector*)	Medición sistemática de selección y encuadre mediático a escala	Sistema en curso; falta estandarización cross-plataforma de etiquetas
Samuel et al. (2025) (ML + LLMs sobre corpus)	Traza dinámicas emocionales (*AI phobia*) propagadas por la cobertura mediática	Tema único; corpus específico
PR3 — Arquitecturas de software con IAG para el procesamiento de noticias		
Behbahanian (s.f.) (*GlimzAI*)	Deduplicación semántica y agregación end-to-end de noticias	Sin validación con usuarios; foco en distribución
Neupane et al. (2025) (Node.js + GPT-4o)	Clustering temático y generación automática de contenido para portales autónomos	Sin validación con expertos
Ameer (2025) (*NexGen MediaCraft*)	Broadcasting de noticias en tiempo real con presentador virtual	Foco en presentación; no aborda análisis interpretativo
Begum J and D (2024) (GPT-3.5 + BERTweet)	Reportes diarios personalizados con resumen en dos niveles y análisis de sentimiento	Foco en personalización individual
Somasundharam et al. (2025) (LangChain + LLaMA-3)	RAG con trazabilidad de fuentes y baja latencia para consultas multi-URL	Foco en performance; no aborda análisis interpretativo

2.3. Estado del Arte

Sobre la base del Marco Teórico (Sección 2.1), esta sección analiza estudio por estudio cómo el corpus incluido aborda cada una de las preguntas de investigación. Las subsecciones siguen el orden de las tres PR.

2.3.1. Limitaciones del Procesamiento de Lenguaje Natural (NLP) Tradicional

Esta subsección aborda la PR1 examinando las limitaciones que las técnicas tradicionales de NLP enfrentan al analizar el discurso mediático contemporáneo. El análisis de grandes volúmenes de noticias ha dependido históricamente de técnicas de NLP centradas en la detección de palabras clave y análisis de sentimiento superficial. Sin embargo, autores como Milbauer et al. (2023) sostienen que estas técnicas fallan al capturar el *sensemaking* cross-documental, es decir, la capacidad de contrastar y sintetizar perspectivas contradictorias presentes en múltiples artículos. Mientras que el NLP tradicional trata los documentos como entidades aisladas, la necesidad actual de investigadores sociales exige interfaces que evalúen el apoyo o contradicción de la evidencia en tiempo real (Milbauer et al., 2023).

Por otro lado, Pavlyshenko (2024) evalúa el uso de modelos GPT frente a métodos estadísticos tradicionales, concluyendo que, aunque la IAG ofrece una capacidad interpretativa superior para el análisis cualitativo, aún presenta incertidumbres significativas en la cuantificación de datos. Esta postura contrasta con enfoques puramente algorítmicos al sugerir que la IAG no reemplaza al NLP estadístico, sino que requiere de marcos complementarios (como la regresión bayesiana) para interpretar los sesgos inherentes en las salidas generadas (Pavlyshenko, 2024).

2.3.2. Aplicaciones Computacionales del Agenda-Setting de Segundo Nivel

Esta subsección aborda la PR2 examinando cómo trabajos recientes han operacionalizado computacionalmente el agenda-setting de segundo nivel propuesto por McCombs et al. (1997), particularmente en lo relativo a dos atributos centrales: el sesgo y el encuadre (*framing*) por un lado, y el sentimiento por otro. Haider et al. (2025) proponen un framework para detectar el sesgo

mediático a escala, subrayando que la selección y el encuadre pueden medirse sistemáticamente mediante pipelines de scraping integrados con LLMs. Este enfoque permite una vigilancia continua de la responsabilidad mediática, algo que los métodos manuales de codificación no podían alcanzar por limitaciones de escala (Haider et al., 2025).

Complementariamente, el estudio de Samuel et al. (2025) sobre la difusión del miedo hacia la inteligencia artificial evidencia cómo la cobertura mediática amplifica sentimientos públicos. Al emplear una combinación de aprendizaje automático y LLMs, los autores logran trazar dinámicas emocionales en el corpus noticioso, demostrando que la infraestructura técnica es capaz de revelar no solo qué temas están en la agenda, sino qué emociones colectivas se están propagando a través de ella (Samuel et al., 2025).

2.3.3. Arquitecturas de Software con IAG para el Procesamiento de Noticias

Esta subsección aborda la PR3 examinando las arquitecturas de software emergentes que integran *web scraping*, *embeddings* y modelos generativos en *pipelines* reproducibles para sistematizar el procesamiento y análisis de noticias. La implementación de infraestructuras técnicas ha evolucionado hacia sistemas modulares y agentes autónomos. Behbahanian (s.f.) describe el sistema *GlimzAI*, que utiliza flujos de trabajo "agentic" y *embeddings* de Sentence-BERT para la deduplicación semántica. Este enfoque es fundamental para el análisis de noticias, ya que permite gestionar la redundancia informativa de múltiples portales.

Asimismo, Neupane et al. (2025) demuestra la eficacia de combinar un backend de scraping basado en Node.js con modelos como GPT-4o para orquestar pipelines que van desde la extracción y clustering semántico hasta la publicación automatizada de contenido (Neupane et al., 2025).

En conjunto, estos trabajos confirman la consolidación de un patrón común –scraping, indexación vectorial e inferencia con LLMs– pero exhiben divergencias significativas en sus objetivos finales (entrega multimedia, reporte personalizado, recuperación trazable) y en los criterios de evaluación aplicados.

La Tabla 2.2 sintetiza los nueve estudios incluidos, agregando la columna "Relación con la presente tesina" que explica el vínculo de cada trabajo con los aportes que esta investiga. Esta tabla complementa la Tabla 2.1 (pág. 19),

que organiza los mismos trabajos por pregunta de investigación con foco en aportes y limitaciones.

La lectura transversal de la tabla confirma el argumento presentado: existe un patrón arquitectónico consolidado (scraping, *embeddings*, RAG e inferencia con LLMs) que ninguna de las obras relevadas completa con las cinco dimensiones que la presente tesina aborda de manera integral —tipos de datos contractuales entre filtros, trazabilidad documental de extremo a extremo, verificación editorial explícita con calibración de incertidumbre, parametrización por *templates* gestionados, y carácter auditable por aplicación del estilo Tubos y Filtros.

2.4. Síntesis Crítica y Brechas en la Investigación

Al contrastar las fuentes, se observa una convergencia clara en el uso de *embeddings* para la representación semántica del contenido y pipelines modulares de extracción, procesamiento y publicación (Neupane et al., 2025; Behbahanian, s.f.). No obstante, surgen divergencias en cuanto a la prioridad: mientras algunos autores se enfocan en la eficiencia de la entrega de contenidos (Behbahanian, s.f.), otros priorizan la verificabilidad y la explicabilidad del rastro documental (Milbauer et al., 2023).

Se han identificado las siguientes brechas que el framework propuesto en esta tesina busca llenar:

- **Escalabilidad y consistencia de anotaciones:** Aunque se busca medir la selección y el encuadre (agenda-setting), falta un estándar interoperable de etiquetas (tópico, tono, tipo de cobertura, tal como lo describe Haider et al. (2025)) y formas de evaluar su fiabilidad cross-plataforma.

- **Robustez del Scraping:** La mayoría de los prototipos documentados no abordan estrategias de mantenimiento ante cambios estructurales en portales dinámicos (Haider et al., 2025).

- **Trazabilidad en Ciencias Sociales:** Existe poca evaluación sobre la fidelidad de la evidencia extraída. Hacen falta mecanismos uniformes que integren los *embeddings* con los metadatos del scraping para auditar

Tabla 2.2: Comparativa de los estudios incluidos en la revisión sistemática.

Estudio	**Enfoque**	**Caso de aplicación**	**Relación con la presente tesina**
Milbauer et al. (2023) (NewsSense)	NLP cross-document + verificación de evidencia	*Sensemaking* del lector sobre noticias contradictorias	Antecedente directo del filtro de Verificación: motiva el juicio de fidelidad con sustentación documental.
Pavlyshenko (2024) (RAG + bayesiano)	IAG + modelado bayesiano para cuantificar incertidumbre	Elecciones US 2024	Respalda la necesidad de calibrar incertidumbre (brecha 4) con el campo `confianza` del tipo `Veredicto`.
Haider et al. (2025) (Media Bias Detector)	Pipeline de scraping + LLMs para sesgo y encuadre a escala	Corpus multi-portal en desarrollo	Justifica la robustez del scraping (brecha 2) y los esquemas declarados por *template*.
Samuel et al. (2025) (ML + LLMs)	Combinación de ML clásico y LLMs para trazar dinámicas emocionales	Tema único: "AI phobia"	Valida el uso de modelos generativos para el agenda-setting de segundo nivel (atributos afectivos).
Behbahanian (s.f.) (GlimzAI)	Pipeline end-to-end con deduplicación semántica por *embeddings*	Agregación de noticias general	Patrón arquitectónico cercano al filtro de Enriquecimiento semántico; diverge en la ausencia de gobernanza editorial.
Neupane et al. (2025) (Node.js + GPT-4o)	Clustering temático + generación automática de contenido	Portal autónomo	Patrón scraping → *embedding* → LLM análogo al pipeline propuesto, sin verificación editorial.
Ameer (2025) (NexGen MediaCraft)	Broadcasting time-real con presentador virtual 3D	Noticias en tiempo real	Muestra alcance alternativo del sumidero; la presente tesina prioriza el análisis sobre la entrega multimedia.
Begum J and D (2024) (GPT-3.5 + BERTweet)	Reportes diarios con resumen en dos niveles + análisis de sentimiento	Personalización individual por correo	Inspiración para el análisis multi-nivel (resumen + análisis) del filtro de Normalización.
Somasundharam et al. (2025) (LangChain + LLaMA-3)	RAG con FAISS + trazabilidad de fuentes	Consultas multi-URL de baja latencia	Antecedente del patrón Claim Check que inspira el repositorio común (§ 3.2.5).

(y justificar ante el investigador social) qué fragmentos apoyan cada afirmación generada.

- **Calibración de Incertidumbre:** Pocos frameworks integran la cuantificación de incertidumbre en los scores de análisis de sentimiento o tópicos, un aspecto crítico destacado por Pavlyshenko (2024).
- **Gobernanza de publicación y responsabilidad editorial:** Existen riesgos por la publicación automática de análisis sin filtros de veracidad o atribución, siendo críticos los procedimientos *human-in-the-loop* ante controversias.

Cada una de estas brechas se aborda de manera explícita desde los objetivos específicos formulados en la Sección 1.2. La Tabla 2.3 fija la correspondencia entre cada brecha identificada y los objetivos que la atienden, junto con el mecanismo concreto mediante el cual la propuesta la aborda y la responsabilidad transversal (Sección 1.3) que ese mecanismo realiza.

Tabla 2.3: Trazabilidad entre brechas identificadas en la revisión sistemática, objetivos específicos que las atienden y responsabilidades transversales que las realizan. Marcadores de responsabilidades: [A] experto · [B] estudio · [C] generativa · [D] control editorial · [E] procedencia · [F] preexistente (Tabla 3.2).

Brecha	**OE**	**Capítulo**	**Mecanismo de abordaje**	**Resp.**
Escalabilidad y consistencia de anotaciones	OE3	Cap. 3	Tipos de datos con esquemas declarados por *template* (§ 3.2.4 y Apéndice A).	[B]
Robustez del *scraping*	OE3	Cap. 3	Sustituibilidad del filtro de Extracción: puede reemplazarse sin afectar el resto del pipeline (Subsección 3.1.3).	[F]
Trazabilidad en ciencias sociales	OE1, OE3	Caps. 3, 5	Grafo de procedencia de extremo a extremo desde `Resultado` hasta `Noticia`, consultable por *backward reachability* (Apéndice A).	[E]
Calibración de incertidumbre	OE3	Cap. 3	Filtro de Verificación con campo `confianza` explícito en el tipo `Veredicto` (Subsección 3.2.2).	[D]
Gobernanza editorial *human-in-the-loop*	OE1, OE2	Caps. 3, 5	Compuerta blanda en Verificación + representación de la intervención del experto en el mapa de procesos (Subsecciones 3.2.2 y 5.4).	[A], [D]

Propuesta de valor: Ante estas brechas, un framework modular integrado proporcionará *scraping* resistente, indexación por *embeddings*, control sobre la incertidumbre y gobernanza editorial explícita, mitigando los riesgos de la automatización ciega y aumentando la reproducibilidad y la trazabilidad para los investigadores.

2.5. Conclusión del Capítulo

La revisión sistemática presentada en este capítulo permite delimitar con precisión el punto de partida de esta tesina, tanto en lo que la literatura ya ha consolidado como en lo que aún permanece sin resolver.

En el plano de lo consolidado, los nueve estudios analizados confirman que el patrón arquitectónico que combina *web scraping*, indexación mediante *embeddings* e inferencia con modelos de lenguaje extenso es técnicamente viable y reproducible en distintos dominios de aplicación (Behbahanian, s.f.; Neupane et al., 2025; Somasundharam et al., 2025). Asimismo, queda establecido que la inteligencia artificial generativa supera al NLP estadístico tradicional en tareas interpretativas (Milbauer et al., 2023; Pavlyshenko, 2024) y que es posible operacionalizar computacionalmente el *agenda-setting* de segundo nivel a escala (Haider et al., 2025; Samuel et al., 2025). Esta base no constituye, por lo tanto, un riesgo a explorar en este trabajo, sino el suelo técnico sobre el que se construye.

En el plano de lo no resuelto, la Síntesis Crítica (Sección 2.4) identificó cinco brechas que los sistemas existentes no abordan de manera integral: la ausencia de estándares interoperables para anotar tópico, tono y encuadre de forma comparable entre plataformas; la fragilidad del *scraping* ante cambios estructurales de los portales; la falta de trazabilidad documental entre la evidencia extraída y las afirmaciones generadas por los modelos; la escasa calibración de la incertidumbre en los *scores* de análisis; y la carencia de procedimientos de gobernanza editorial *human-in-the-loop*. Estas brechas configuran el espacio problemático específico desde el cual arranca esta tesina.

A partir de ese punto de partida, los cuatro objetivos específicos definidos en el Capítulo 1 responden a brechas concretas detectadas en la revisión. El OE1 (módulo *addon* de Odoo) institucionaliza el flujo extracción-análisis-visualización como aplicación gestionable y auditable, atendiendo a las carencias de gobernanza y trazabilidad. El OE2 (mapa de procesos) formaliza la hoja de ruta procedimental que dota de reproducibilidad y robustez a las etapas de recolección y análisis. El OE3 (caracterización del pipeline bajo el estilo Tubos y Filtros) busca consistencia de anotaciones y trazabilidad de inferencias en la capa generativa. El OE4 (caso de aplicación) somete la propuesta a validación empírica sobre un escenario concreto, cerrando el ciclo entre diseño y evidencia. En conjunto, el framework propuesto no aspira a reemplazar el patrón arquitectónico consolidado por la literatura, sino a completarlo en las dimensiones procedimentales y de rigor científico que los prototipos relevados aún no resuelven.

El capítulo siguiente desarrolla la primera capa de esta propuesta: la aplicación de *scraping* que materializa el componente de extracción del framework.

Capítulo 3

Arquitectura

Este capítulo presenta la arquitectura de la infraestructura propuesta. A diferencia del Capítulo 5, donde los procesos se describen de manera agnóstica a la implementación, aquí se aborda la dimensión técnica: las decisiones de estilo arquitectónico, los componentes que materializan cada capa del sistema y el stack tecnológico sobre el que se construye.

La propuesta no se concibe en el vacío. Esta tesina se inscribe en una línea de investigación interdisciplinaria desarrollada por el Centro de Altos Estudios en Tecnología Informática (CAETI) de la Universidad Abierta Interamericana en articulación con el Centro de Investigaciones en Mediatizaciones (CIM) de la Universidad Nacional de Rosario, en cuyo marco se ha venido construyendo una infraestructura técnica para la extracción y análisis de noticias de portales digitales mediante inteligencia artificial generativa. Esa infraestructura preexistente provee el sustrato sobre el cual esta tesina opera: una plataforma Odoo Community con módulos para la configuración y monitoreo de portales digitales, una herramienta de scraping basada en spiders parametrizables que combina librerías como BeautifulSoup, Selenium y Newspaper3k, y una capa inicial de procesamiento mediante *retrieval-augmented generation* (RAG) y modelos de lenguaje de gran escala (Raimondo Anselmino et al., 2024). Su caracterización detallada, y la delimitación precisa de qué se reutiliza de ella, se desarrollan en la Sección 4.1.

Sobre esa base, el aporte original de la presente tesina se concentra en tres artefactos técnicos. En primer lugar, un módulo *addon* específico para Odoo que integra y unifica las capacidades necesarias para automatizar el ciclo completo de análisis, satisfaciendo el objetivo específico OE1. En segundo lugar, un panel de creación de *templates* de *prompt* que reifica como subsiste-

ma gestionado lo que en aplicaciones convencionales basadas en modelos de lenguaje suele resolverse de manera *ad hoc*, habilitando la configuración, parametrización y reutilización de plantillas por parte de expertos del dominio: da cuerpo a la dimensión de *prompt engineering* del objetivo específico OE3 y, en la arquitectura que se caracteriza más adelante, constituye la interfaz de configuración del filtro de Composición de contexto. En tercer lugar, un pipeline de procesamiento orientado a inteligencia artificial generativa diseñado bajo el estilo arquitectónico de Tubos y Filtros (Garlan and Shaw, 1994; Shaw and Garlan, 1996), que organiza la transformación progresiva del input —desde los datos empíricos extraídos hasta el resultado analítico final— como una composición de filtros desacoplados con formatos de mensaje explícitos entre etapas, atendiendo a la dimensión arquitectónica de ese mismo objetivo específico OE3.

La elección de Tubos y Filtros como estilo articulador del procesamiento generativo no es arbitraria. Refleja, por una parte, una tendencia documentada en la literatura sobre la integración de modelos generativos dentro de mapas de procesos tecnológicos automatizados (Bertagnolio et al., 2023), donde la modularización del flujo se ha consolidado como condición para la mantenibilidad, la auditabilidad y la sustituibilidad de los componentes algorítmicos. Por otra parte, atiende de manera específica a brechas identificadas en la revisión sistemática del Capítulo 2, según se desarrolla en la Sección 3.1.3.

El capítulo se organiza en tres secciones. La Sección 3.1 presenta el estilo de Tubos y Filtros: sus componentes, conectores, invariantes y modelo computacional, y justifica su elección como estilo articulador del procesamiento generativo de noticias. La Sección 3.2 aplica ese estilo al caso concreto de la presente tesina, caracterizando el framework como un sistema de Tubos y Filtros cuya fuente son los portales digitales de noticias y cuyo sumidero es la capa de presentación de resultados, y siguiendo el esquema descriptivo del catálogo de Cristiá and Pomponio (2025). Finalmente, la Sección 3.3 sintetiza el capítulo y anticipa cómo los elementos arquitectónicos aquí descriptos se materializan como módulo *addon* de Odoo en el Capítulo 4 y se operativizan procesualmente en el Capítulo 5.

3.1. El estilo Tubos y Filtros

3.1.1. Componentes, conectores y patrón estructural

El estilo de Tubos y Filtros define dos clases de elementos. Los *filtros* son los componentes activos del sistema: unidades de procesamiento independientes que reciben datos por uno o más *puertos* de entrada y producen datos sobre uno o más *puertos* de salida (Cristiá and Pomponio, 2025). Cada filtro encapsula una transformación local sobre el flujo que recibe, sin compartir estado con otros filtros y sin conocer su identidad. Los *tubos* son los conectores: canales unidireccionales que transportan datos del puerto de salida de un filtro al puerto de entrada de otro, preservando el orden de los elementos transmitidos y sin modificar su contenido (Garlan and Shaw, 1994; Shaw and Garlan, 1996).

La configuración estructural canónica del estilo se organiza como un grafo dirigido acíclico que va desde una o más *fuentes* hasta uno o más *sumideros*. Las fuentes son los componentes externos al sistema que alimentan datos al pipeline —típicamente entradas estándar, archivos, dispositivos o servicios externos—; los sumideros son los componentes externos que reciben los datos emitidos por el sistema. El flujo de información se propaga progresivamente entre filtros adyacentes a través de los tubos, sin retroceso ni ciclos en su forma canónica (Cristiá and Pomponio, 2025).

Una propiedad estructural distintiva del estilo es su *cierre composicional*: un sistema entero de Tubos y Filtros constituye en sí mismo un filtro, en la medida en que admite ser tratado como una unidad de procesamiento con sus propios puertos de entrada y salida (Cristiá and Pomponio, 2025, p. 24). Esta propiedad habilita el diseño jerárquico —en el que un filtro puede a su vez descomponerse internamente en una subred de filtros— y formaliza la noción de que el sistema completo es una composición funcional de los filtros que lo integran (Garlan and Shaw, 1994). La descripción detallada de los componentes específicos del framework propuesto, así como de las fuentes y sumideros concretos sobre los que opera, se presenta en la Sección 3.2.

3.1.2. Modelo computacional e invariantes

Cada filtro de un sistema de Tubos y Filtros opera de manera independiente: no comparte memoria con los demás filtros y procesa los datos disponibles en sus puertos de entrada sin coordinación explícita con el resto del sistema. La

literatura distingue dos modalidades de filtros según cómo accedan al flujo entrante (Cristiá and Pomponio, 2025). Los filtros *activos* revisan periódicamente sus puertos de entrada en busca de datos disponibles y, cuando los detectan, los consumen y producen la salida correspondiente sobre sus puertos de salida; los filtros *pasivos* exponen una interfaz que es invocada por los tubos para comunicarles o solicitarles datos. La elección entre una y otra modalidad responde a consideraciones de implementación —asociadas, por ejemplo, al uso de hilos de ejecución, a la sincronización entre productores y consumidores y al manejo de buffers limitados— y no altera la semántica del estilo.

El estilo se define formalmente por cuatro *invariantes esenciales* que cualquier diseño que se reclame como Tubos y Filtros debe verificar (Cristiá and Pomponio, 2025, p. 23); las tres primeras formalizan las invariantes que Garlan and Shaw (1994) establecen para el estilo:

1. Ningún filtro conoce la identidad ni la existencia de los demás filtros del sistema.

2. Ningún filtro depende del estado interno de los demás filtros, ni puede observarlo o modificarlo.

3. La corrección de la salida producida por el sistema no depende del orden en que cada filtro realice sus cálculos individuales.

4. Los tubos transmiten los datos entre filtros sin alterar su contenido.

Estas invariantes constituyen, en conjunto, una condición de *independencia funcional* entre los componentes del sistema. Tres consecuencias arquitectónicas se desprenden directamente de ellas. En primer lugar, la *sustituibilidad* de cada filtro: cualquier filtro puede ser reemplazado por otro que respete su interfaz de puertos y los tipos de datos transmitidos, sin necesidad de modificar el resto del sistema (Garlan and Shaw, 1994). En segundo lugar, la *auditabilidad lineal* del flujo: las transformaciones que sufren los datos al atravesar el sistema pueden inspeccionarse filtro por filtro, sin reconstruir interacciones implícitas. En tercer lugar, la *posibilidad de ejecución concurrente*: en la medida en que el orden de cómputo no incide sobre la corrección del resultado, los filtros pueden ejecutarse en paralelo o distribuirse entre máquinas distintas sin afectar la semántica del estilo. Conviene precisar que el invariante 3 refiere al orden de *scheduling* interno con que cada filtro realiza

sus cálculos individuales, no a las dependencias causales que la topología del pipeline establece entre ellos: un filtro no puede consumir datos que aún no fueron emitidos en el puerto al que está conectado, y esa restricción —propia del grafo de tubos— es una dependencia de flujo, no una dependencia de orden de cómputo en el sentido del invariante. La verificación de cómo se satisfacen estas condiciones en el diseño propuesto se aborda en la Subsección 3.2.7; el análisis de cuál de las propiedades arquitectónicas resulta más relevante para el caso de la presente tesina se discute en la Subsección 3.1.3.

3.1.3. Por qué Tubos y Filtros para el procesamiento generativo de noticias

La elección de un estilo arquitectónico para el procesamiento generativo de noticias debe responder a las necesidades específicas del problema, tal como se las delimitó en los capítulos previos. La revisión sistemática presentada en el Capítulo 2 identificó cinco brechas que los sistemas existentes no abordan de manera integral; de ellas, dos caen dentro del alcance de las decisiones de estilo arquitectónico y orientan la elección que se adopta en esta tesina: la *fragilidad del scraping* ante cambios estructurales de los portales, que demanda poder modificar o reemplazar la etapa de extracción sin reescribir el resto del sistema; y la *falta de trazabilidad documental* entre la evidencia extraída y las afirmaciones generadas por los modelos, que exige que cada transformación de los datos resulte localizable e inspeccionable. De las tres brechas restantes, la ausencia de estándares interoperables de anotación se atiende en el modelo de datos del módulo, mientras que la escasa calibración de la incertidumbre y la carencia de procedimientos de gobernanza editorial *human-in-the-loop* se abordan mediante una etapa de verificación que el framework incorpora al propio pipeline —el filtro de Verificación (Subsección 3.2.2)—, complementada por el mapa de procesos del Capítulo 5. A estas dos brechas se suman dos requisitos adicionales derivados del ecosistema tecnológico en el que la propuesta opera: la necesidad de poder sustituir los modelos generativos accesibles por API ante una evolución acelerada del mercado, y la necesidad de recombinar las etapas de procesamiento para acomodar distintos casos de estudio sin reescribir la infraestructura.

El estilo de Tubos y Filtros se ajusta a este perfil de problema. El catálogo de Cristiá and Pomponio (2025, p. 21) identifica entre los dominios canónicos de aplicación del estilo el procesamiento de cadenas, el procesa-

miento de señales y, en general, los sistemas cuyo flujo de información se conciba como una secuencia de transformaciones sobre un *stream* de datos. El análisis automatizado de noticias mediante modelos generativos, en tanto procesamiento progresivo de texto que se inicia en la extracción y culmina en una representación analítica enriquecida, encuadra naturalmente en esta familia. Garlan and Shaw (1994) mencionan, además, los sistemas de transformación de información y los entornos distribuidos como dominios donde el estilo ha resultado históricamente efectivo.

La propiedad de *sustituibilidad* de los filtros, derivada de las invariantes 1 y 2 del estilo, atiende de manera conjunta a la brecha de fragilidad del scraping y al requisito de sustitución de modelos generativos. En la medida en que ningún filtro conoce la identidad de los demás ni accede a su estado interno, la etapa de extracción puede actualizarse —para adaptarse a un rediseño del portal monitoreado— sin afectar a las etapas posteriores; del mismo modo, el filtro responsable de invocar al modelo generativo puede ser reemplazado por otro que utilice un proveedor distinto, o un modelo distinto del mismo proveedor, sin necesidad de reescribir el resto del pipeline. La misma propiedad habilita la sustitución del filtro de enriquecimiento semántico —reemplazando, por ejemplo, una técnica de *embedding* por otra— sin alterar las etapas siguientes. Esta capacidad de evolución localizada resulta particularmente valiosa frente a infraestructuras cuyas dependencias externas evolucionan con frecuencia. En términos de las responsabilidades transversales de la Sección 1.3, la sustituibilidad se paga en dos costuras: la frontera con los servicios de modelos externos ([C]) y la frontera con la infraestructura preexistente ([F]).

La *auditabilidad lineal* del flujo, segunda consecuencia derivada de las invariantes del estilo, responde directamente a la brecha de trazabilidad documental identificada en la revisión: la dificultad de reconstruir el camino que sigue la evidencia desde su extracción hasta las afirmaciones que el modelo generativo produce sobre ella. Cada transformación que los datos sufren al atravesar el sistema queda localizada en un filtro identificable, con una entrada y una salida definidas y sin interacciones implícitas mediadas por estados compartidos. Para una investigación que pretenda fundamentar afirmaciones sobre la cobertura mediática o el sesgo discursivo en términos verificables, esta propiedad no es accesoria: constituye una condición de posibilidad para la revisión metodológica y para la reproducibilidad del análisis. La separación explícita entre las etapas de extracción, enriquecimiento, composición de contexto e invocación generativa permite, además, distinguir las decisiones

técnicas que afectan al corpus de aquellas que afectan a la interpretación, dimensión sobre la que el experto del dominio debe poder ejercer un control directo. La auditabilidad lineal es, así, la propiedad del estilo sobre la que se apoya la responsabilidad de procedencia ([E]).

Por último, el *cierre composicional* del estilo —el hecho de que un sistema de Tubos y Filtros sea en sí mismo un filtro— y la modularidad consiguiente atienden al requisito de diversidad de los casos analíticos. La caracterización del problema como una composición funcional de filtros habilita la construcción de pipelines especializados para distintos casos de estudio, en los que las etapas de procesamiento pueden recombinarse, agregarse o suprimirse en función del objetivo concreto del experto del dominio, sin alterar las garantías estructurales del estilo (Hohpe and Woolf, 2003); es la propiedad que sostiene la independencia del framework respecto del estudio ([B]). La adopción de Tubos y Filtros es coherente, además, con la forma en que los sistemas contemporáneos de aprendizaje automático y de modelos generativos se construyen por composición de componentes modulares: bibliotecas como *scikit-learn* organizan el flujo como una composición de transformaciones que constituye en sí misma una unidad reutilizable (Buitinck et al., 2013), y arquitecturas como la generación aumentada por recuperación combinan un modelo generativo con un componente de recuperación en un *pipeline* entrenado de extremo a extremo (Lewis et al., 2020). Esta convergencia no se limita al ámbito de la ingeniería clásica de software: los surveys más recientes sobre agentes basados en modelos de lenguaje —Wang et al. (2024); Xi et al. (2023)— describen los sistemas representativos —AutoGen (Wu et al., 2024), ChatDev (Qian et al., 2024), Generative Agents (Park et al., 2023)— como composiciones de etapas especializadas comunicadas mediante mensajes, con puntos explícitos de parametrización análogos a los sintonizadores del framework y reemplazabilidad entre implementaciones; es decir, rasgos que se corresponden con las invariantes canónicas del estilo (Garlan and Shaw, 1994). Incluso en un plano más bajo del pipeline, el patrón fundacional *ReAct* (Yao et al., 2023) —alternancia explícita entre razonamiento interno y acción externa sobre el entorno— es una instancia elemental del mismo principio de composición de etapas separadas que el presente framework aplica a escala de sistema. Del mismo modo, la literatura contemporánea sobre procesamiento de noticias documenta pipelines automatizados con filtros explícitos: el marco de predicción en tiempo real del impacto de noticias financieras sobre los mercados de Chen et al. (2023) organiza el tratamiento del flujo noticioso como una cadena de etapas de procesamiento desacopladas, mostrando

que el patrón de Tubos y Filtros es una elección técnica consolidada para la orquestación de componentes de tratamiento de datos heterogéneos en el dominio mediático.

Conviene anticipar que el diseño que se adopta no constituye una instancia pura del estilo, sino una variante *heterogénea* que combina la red de tubos y filtros con un repositorio compartido. Cada uno de los dos ingredientes de esta variante cuenta con respaldo independiente en la literatura: la admisión de un repositorio compartido junto a la red de tubos configura una *arquitectura heterogénea* en el sentido de Garlan and Shaw (1994), mientras que el mecanismo de transportar por el conector una referencia al contenido —y no la carga útil— corresponde al patrón *Claim Check* de Hohpe and Woolf (2003), tratado además como patrón de integración de primer orden en la literatura revisada por pares (Ritter, 2014). La combinación específica de ambos —tubos que transportan *claim checks* mientras los datos residen en un almacén separado— no constituye, hasta donde alcanza esta revisión, un patrón consolidado en la literatura académica, sino una síntesis propia de este trabajo; su adopción como práctica de ingeniería está documentada, no obstante, en catálogos de patrones de la industria, como el del *Microsoft Azure Architecture Center* (Microsoft, 2025). La caracterización detallada de esta variante, su fundamento y su compatibilidad con los invariantes del estilo se desarrollan en las Subsecciones 3.2.5 y 3.2.7.

3.2. El framework como sistema de Tubos y Filtros

Esta sección caracteriza el framework propuesto como un sistema de Tubos y Filtros, aplicando para ello el esquema descriptivo canónico documentado por Cristiá and Pomponio (2025). Las subsecciones que siguen presentan, en este orden, la vista de conjunto del sistema, los filtros que lo componen, los tubos que los conectan, las especializaciones del estilo que se adoptan en el diseño, las deformaciones canónicas aplicables, la verificación de los invariantes esenciales sobre el diseño resultante y un análisis de las ventajas y limitaciones de la elección.

3.2.1. Vista de conjunto y patrón estructural

El diagrama canónico de la Figura 3.1 resume la configuración del sistema: un pipeline lineal de siete filtros que va desde la fuente —el conjunto de portales digitales monitoreados— hasta el sumidero, conformado por la capa de presentación de resultados. Los filtros se denominan Extracción, Enriquecimiento semántico, Composición de contexto, Invocación generativa, Decodificación, Verificación y Normalización; cada uno encapsula una transformación específica sobre la representación de los datos que recibe y la entrega al siguiente filtro a través de un tubo dedicado. Cada tubo transporta un *recordset* tipado —en la terminología de Odoo, un conjunto ordenado de referencias a los registros resultantes de la transformación—, mientras que el contenido completo de esos registros reside en un repositorio común compartido por todos los filtros; los puertos de entrada y salida de cada filtro se numeran en la figura y se detallan, junto con su tipo y su cardinalidad, en la tabla de referencia que la acompaña.

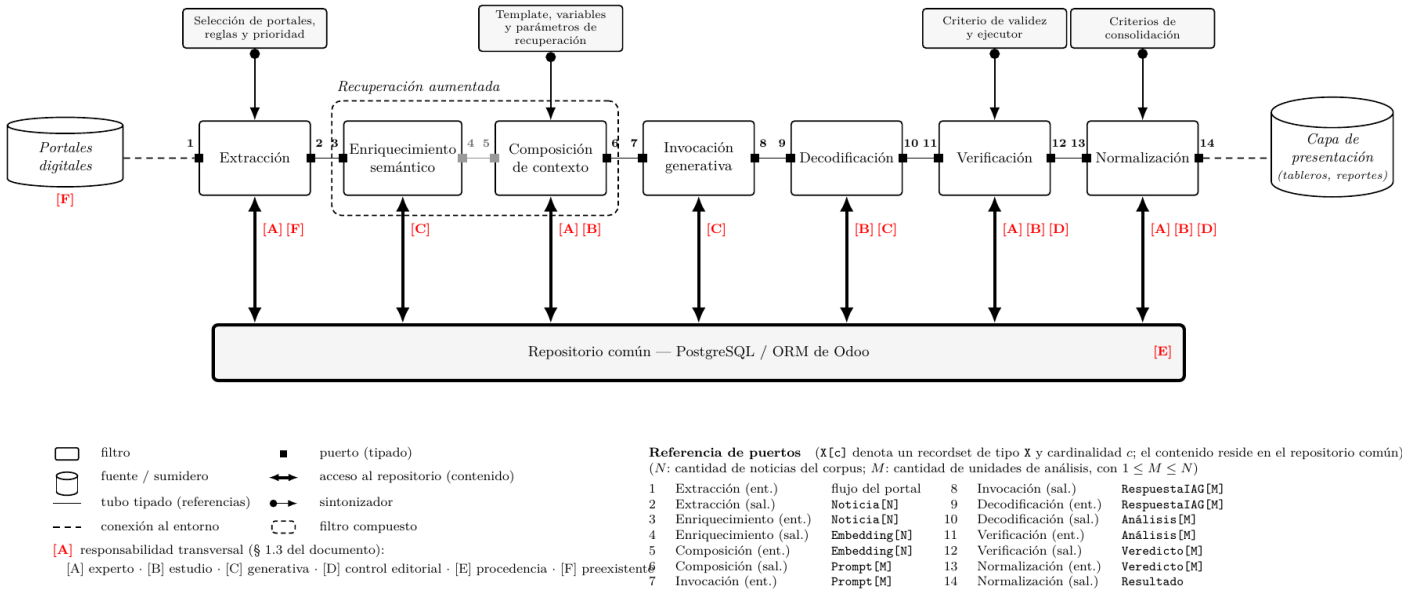


Figura 3.1: Diagrama canónico del framework propuesto como sistema de Tubos y Filtros. La fuente son los portales digitales de noticias y el sumidero es la capa de presentación de resultados (tableros y reportes). Los filtros se comunican mediante tubos tipados que transportan *recordsets* —conjuntos de referencias a registros—, cuyos puertos se numeran y se detallan en la tabla de referencia inferior; el contenido completo de esos registros reside en el repositorio común. Siguiendo la convención del estilo, las conexiones al entorno se representan con línea discontinua y los tubos entre filtros con línea continua —ambos sin punta, dado que el flujo se asume de izquierda a derecha—, en tanto que los sintonizadores —modulares, propios de cada filtro configurable— se grafican con flecha originada en un círculo y el acceso al repositorio con flecha bidireccional. El recuadro de trazo discontinuo que engloba los filtros de Enriquecimiento semántico y Composición de contexto indica que ambos conforman un filtro compuesto, denominado Recuperación aumentada. Los marcadores en rojo bajo cada filtro, junto a la fuente y sobre el repositorio señalan las responsabilidades transversales que cada elemento realiza. Marcadores de responsabilidades: [A] experto · [B] estudio · [C] generativa · [D] control editorial · [E] procedencia · [F] preexistente (Tabla 3.2).

La figura incorpora ya las especializaciones y deformaciones del estilo que el diseño adopta, cuya justificación se desarrolla en las subsecciones siguientes. La configuración se organiza como un *pipeline* —cada filtro posee un único puerto de entrada y un único puerto de salida— de *tubos tipados*, en el que cada tubo declara el tipo del recordset que transporta; ambas especializaciones se fundamentan en la Subsección 3.2.4. Incorpora, además, un repositorio común de datos sobre el cual los filtros leen y escriben el contenido que los tubos no transportan, deformación que se discute en la Subsección 3.2.5. Por último, expone en los filtros que lo requieren *sintonizadores* —procedimientos invocados desde la capa de presentación que incorporan la configuración del experto del dominio sin participar del flujo de datos del pipeline—, tratados en la Subsección 3.2.6.

Tres observaciones sobre la configuración resultan oportunas antes de abordar el detalle de cada componente. En primer lugar, el flujo de datos por la red de tubos es estrictamente acíclico: avanza desde la fuente hacia el sumidero sin ramificaciones ni ciclos. El acceso al repositorio común, de naturaleza bidireccional, no introduce ciclos entre filtros, ya que constituye un mecanismo de almacenamiento compartido y no un tubo que vincule etapas del pipeline entre sí. La iteración del experto del dominio sobre los resultados —fenómeno que tiene lugar entre ejecuciones sucesivas del sistema— es propia del proceso de uso, no de la arquitectura interna del pipeline, y por lo tanto queda fuera del alcance de la presente caracterización. En segundo lugar, fuente y sumidero son componentes externos al framework: los portales digitales no pertenecen al sistema sino que constituyen su entorno de entrada, y la capa de presentación, encargada de la visualización y la generación de reportes, corresponde al entorno de consumo de los resultados. Esta delimitación es coherente con la convención del estilo, según la cual el sistema queda definido por la red de filtros y tubos y no por sus interlocutores externos. En tercer lugar, la configuración exhibe la composicionalidad propia del estilo, presentada en la Subsección 3.1.1: dos de los filtros —Enriquecimiento semántico y Composición de contexto— se agrupan en un filtro compuesto, denominado Recuperación aumentada, que hacia el exterior se comporta como un único filtro. Esta misma propiedad habilita, además, la inclusión, eliminación o sustitución de etapas para acomodar variantes analíticas particulares; la caracterización aquí presentada corresponde al pipeline base que cubre el ciclo completo de análisis sin compromiso con ningún caso de estudio específico.

3.2.2. Componentes: los filtros del framework

Esta subsección caracteriza cada uno de los siete filtros —dos de los cuales, según se anticipó, se agrupan en el filtro compuesto Recuperación aumentada— siguiendo el esquema que el catálogo de Cristiá and Pomponio (2025) prescribe para el componente de un sistema de Tubos y Filtros: para cada filtro se especifican sus puertos de entrada y salida con su tipo, los sintonizadores que expone —si los tiene—, su acceso al repositorio común y una especificación funcional de la transformación que realiza. La numeración de los puertos remite a la tabla de referencia de la Figura 3.1; la estructura interna de los tipos que circulan por ellos se especifica en el Apéndice A.

Todos los filtros comparten un mismo patrón de operación, consecuencia del uso del repositorio común: reciben por su puerto de entrada un *recordset*, recuperan del repositorio el contenido asociado a esas referencias, aplican su transformación, persisten los resultados y emiten por su puerto de salida un nuevo *recordset* que referencia los registros producidos. Salvo el filtro de Extracción —que es un filtro activo, disparado por las acciones planificadas de la plataforma según la prioridad de monitoreo asignada a cada portal—, los restantes operan a demanda dentro de la ejecución de un análisis, consumiendo los datos disponibles en su puerto de entrada.

Este patrón admite una formulación más precisa, que extiende la notación funcional con que el catálogo describe a los filtros. Cristiá and Pomponio (2025, p. 24) conciben un filtro como una función de sus puertos de entrada en sus puertos de salida, $F : \Gamma^n \to \Gamma^m$, donde n y m son la cantidad de puertos de entrada y de salida y Γ el tipo de dato con que trabajan. En el presente diseño esa firma se particulariza por las dos especializaciones adoptadas (Subsección 3.2.4): por la de *pipeline*, cada filtro tiene un único puerto de cada lado, de modo que $n = m = 1$; por la de tubos tipados, el tipo de la entrada y el de la salida difieren. El dato de cada puerto es, además, un *recordset*, y su firma $T[c]$ declara el tipo T de los registros referenciados y su cardinalidad c (Subsección 3.2.3). Un filtro del pipeline en el estilo puro es, entonces, una función $f : T[c] \to T'[c']$ de su puerto de entrada en su puerto de salida. Conviene no confundir los exponentes n y m del catálogo, que cuentan puertos, con las cardinalidades N y M de la Subsección 3.2.3, que cuentan registros. En la variante heterogénea aquí adoptada, en cambio, la transformación del contenido ocurre en el repositorio común, de modo que el estado que se transforma es el par flujo–repositorio:

$$f : (T[c],\ Rep) \to (T'[c'],\ Rep') \qquad \text{con } Rep' = Rep \cup \Delta_f,$$

donde *Rep* es el estado del repositorio al activarse el filtro y Δ_f el conjunto de registros que éste persiste. El *recordset* emitido por el puerto de salida es, precisamente, el conjunto de referencias a ese delta —$refs(\Delta_f)$—: el tubo transporta la referencia a la transformación que el repositorio absorbió. Dos consecuencias se siguen de esta formulación. La primera es la *monotonía* del repositorio respecto del flujo de datos: cada filtro únicamente agrega registros —$Rep \subseteq Rep'$—, ninguno modifica ni elimina los que otro produjo, propiedad de la que depende la trazabilidad extremo a extremo del sistema (Apéndice A). La segunda es de método: al admitir el repositorio común, el propio catálogo exige documentar y explicar el conector adicional que la deformación introduce (Cristiá and Pomponio, 2025, §3.13.1), y la firma extendida constituye esa documentación en los mismos términos funcionales con que el catálogo describe el estilo puro. Cabe acotar su alcance: la firma describe la forma de la transformación, no su semántica —no supone determinismo, pues la invocación generativa es estocástica, ni pureza funcional—. Las siete firmas concretas del pipeline se consignan en el Apéndice A.

Extracción. Interfaz:

- **Puerto de entrada** (1): flujo del portal, como conexión al entorno desde la fuente.
- **Puerto de salida** (2): `Noticia[]`.
- **Sintonizador**: selección de los portales a monitorear, definición de las reglas de extracción mediante términos combinados por disyunción, conjunción y negación, y asignación de la prioridad de monitoreo.
- **Acceso al repositorio**: escribe los registros `Noticia` (título, copete, texto y enlace de la publicación, junto con sus metadatos de procedencia; Apéndice A).
- **Responsabilidades**: [A] —la selección de portales y reglas es decisión del experto—; [F] —el filtro se apoya en el scraper preexistente del entorno institucional—.

Especificación funcional. Accede a los portales registrados, recorre su estructura mediante técnicas de *web scraping* de tipo *spider* y recupera las publicaciones que satisfacen las reglas de extracción configuradas. De cada publicación extrae los elementos relevantes, normaliza el texto y lo persiste

como un registro `Noticia` en el repositorio. Emite por su puerto de salida el *recordset* con la totalidad de las noticias que conforman el corpus del análisis. La descripción detallada de esta etapa, que se apoya en la herramienta de scraping preexistente del entorno institucional, se desarrolla en la Sección 4.1.

Recuperación aumentada (filtro compuesto). Los filtros de Enriquecimiento semántico y Composición de contexto no operan de manera independiente: el primero genera las representaciones vectoriales que el segundo consume para recuperar el material empírico relevante. En conjunto implementan el patrón de *recuperación aumentada* (*retrieval-augmented generation*, RAG), definido por Lewis et al. (2020) como la combinación de un componente de recuperación en un espacio vectorial con un modelo generativo que consume el material recuperado como contexto para producir la salida. La taxonomía contemporánea de Gao et al. (2024) distingue tres variantes principales dentro del RAG —*naive*, *advanced* y *modular RAG*— y taxonomiza las fases del pipeline en etapas *pre-retrieval* (limpieza, segmentación e indexación del corpus) y *post-retrieval* (reordenamiento, compresión del contexto y ensamblado del prompt). El filtro compuesto del presente framework implementa exactamente esta bipartición: Enriquecimiento semántico corresponde a la fase *pre-retrieval* (indexación vectorial del corpus de noticias); Composición de contexto corresponde a la fase *post-retrieval* (recuperación semántica y ensamblado del prompt con las noticias relevantes). La presente caracterización descompone el RAG canónico en dos sub-filtros —uno de indexación semántica (Enriquecimiento) y otro de composición del contexto (Composición)— para mantener explícitos los puntos de sustituibilidad: reemplazar el modelo de *embedding* sustituye Enriquecimiento sin afectar la Composición, y viceversa. Esta frontera arquitectónica coincide, por lo tanto, con la frontera que la literatura contemporánea ha consolidado para sistemas RAG modulares. La etapa de generación propiamente dicha queda fuera del compuesto y corresponde al filtro de Invocación generativa siguiente. Sus dos sub-filtros se describen a continuación.

Enriquecimiento semántico (sub-filtro). Interfaz:

- **Puerto de entrada** (3): `Noticia[]`.
- **Puerto de salida** (4): `Embedding[]`.

- **Acceso al repositorio**: lee el texto de las noticias recibidas y escribe los registros `Embedding` —la representación vectorial asociada a cada una—.
- **Responsabilidades**: [C] —el modelo de *embedding* es una dependencia externa sustituible cuya identidad no viaja en el contrato—.

Especificación funcional. Para cada noticia del *recordset* de entrada, recupera su contenido textual del repositorio y computa su representación vectorial mediante el modelo de *embedding* configurado, persistiéndola como un registro `Embedding` que referencia a la noticia que codifica. Preserva la cardinalidad —emite un `Embedding` por noticia— pero transforma el tipo que circula: su salida no es el conjunto de noticias sino el de sus representaciones semánticas, disponibles para las operaciones de recuperación que realiza el filtro siguiente. Este filtro no expone sintonizadores al experto del dominio: su configuración —el modelo de *embedding* y la estrategia de fragmentación— es de carácter técnico y la fija el implementador, al margen de todo sintonizador. Conviene distinguir la *generación* del *embedding*, que ocurre aquí sin intervención del experto, de su posterior *uso* en la recuperación, que sí admite parametrización de dominio y se realiza en el filtro de Composición de contexto.

Composición de contexto (sub-filtro). Interfaz:

- **Puerto de entrada** (5): `Embedding[]`.
- **Puerto de salida** (6): `Prompt[]`.
- **Sintonizador**: selección del *template* de *prompt*, customización de las variables de contexto, ingreso de texto libre, ajuste de los parámetros del modelo generativo (temperatura, longitud de salida) y de los parámetros de recuperación (cantidad de documentos a recuperar y umbral de similitud).
- **Acceso al repositorio**: lee los vectores recibidos y las noticias que estos referencian —para la recuperación— y los *templates* disponibles; escribe los registros `Prompt`.
- **Responsabilidades**: [A] —núcleo de la intervención del experto—; [B] —el estudio ingresa como dato a través del *template*—.

Especificación funcional. Constituye el núcleo de la intervención del experto del dominio. A partir del *template* seleccionado y de las variables de contexto configuradas, realiza la recuperación contextual (*retrieval*): a partir de los `Embedding` recibidos consulta el repositorio para obtener las noticias semánticamente relevantes según similitud vectorial, e integra ese material empírico con el *template*, las aclaraciones de texto libre y los parámetros definidos para componer el *prompt* completo destinado al modelo generativo. La cardinalidad puede transformarse en esta etapa: a partir del *recordset* recibido, el filtro produce uno o varios *prompts* según la estrategia analítica —un único *prompt* agregado sobre el corpus, o un *prompt* por unidad de análisis—. Persiste los registros `Prompt` y emite el *recordset* correspondiente. El diseño del panel de *templates* que da soporte a este sintonizador se desarrolla en la Subsección 3.2.6.

Invocación generativa. Interfaz:

- **Puerto de entrada** (7): `Prompt[]`.
- **Puerto de salida** (8): `RespuestaIAG[]`.
- **Acceso al repositorio**: lee los *prompts* y escribe las respuestas crudas del modelo.
- **Responsabilidades**: [C] —aísla la dependencia del servicio generativo externo—.

Especificación funcional. Para cada *prompt* del *recordset* de entrada, establece la conexión con la interfaz de programación (API) del modelo generativo configurado, ejecuta la petición y recibe la respuesta producida. Persiste cada respuesta cruda como un registro `RespuestaIAG` y emite el *recordset* resultante. La selección del proveedor, del modelo y de las credenciales de acceso a la API es configuración técnica del implementador, por lo que el filtro no expone sintonizadores al experto. El aislamiento de esta etapa en un filtro propio —cuyo único cometido es la interacción con el servicio generativo externo— es lo que materializa la propiedad de sustituibilidad analizada en la Subsección 3.1.3: cambiar de proveedor o de modelo se reduce a reemplazar este filtro, sin impacto sobre el resto del pipeline.

Decodificación. Interfaz:

- **Puerto de entrada** (9): `RespuestaIAG[]`.
- **Puerto de salida** (10): `Análisis[]`.
- **Acceso al repositorio**: lee las respuestas crudas y escribe los análisis estructurados.
- **Responsabilidades**: [B] —valida contra el esquema declarado por el estudio, sin conocerlo—; [C] —absorbe el formato de salida del proveedor, manteniendo la etapa agnóstica—.

Especificación funcional. Transforma la salida del modelo generativo —típicamente texto en un lenguaje de marcado— en una estructura de datos del dominio apta para su tratamiento posterior. Para cada respuesta cruda del *recordset* de entrada, interpreta su formato —aísla el fragmento estructurado de la prosa circundante y lo convierte de texto en objeto—, valida su conformidad con la estructura esperada —el esquema declarado por el *template* del estudio, que el filtro alcanza siguiendo los punteros de procedencia que van de la respuesta a su *prompt* y de éste a su *template*, sin requerir por ello configuración de dominio propia—, canonicaliza los valores hacia los tipos del dominio y persiste el resultado como un registro `Análisis`. La validación opera como una compuerta: la respuesta que no conforma el esquema no se persiste como `Análisis` —la política ante la no conformidad (reintentar la invocación, reparar, descartar) es configuración técnica del implementador—, de modo que las etapas posteriores consumen exclusivamente estructuras válidas. Emite el *recordset* de análisis estructurados. Esta separación entre la obtención de la respuesta (filtro anterior) y su decodificación aísla el manejo del formato de salida del modelo, de modo que un cambio en dicho formato no afecte a las etapas previas.

El término encierra un compromiso arquitectónico que conviene explicitar. En el procesamiento de lenguaje natural, *decoding* designa la estrategia con que el modelo selecciona cada token durante la generación —búsqueda voraz, *beam search*, muestreo— (Wiher et al., 2022); la decodificación de este filtro es, en cambio, la interpretación estructural de la salida ya generada. Una técnica contemporánea difumina esa frontera: la *generación restringida* (*constrained decoding*) hace que una gramática enmascare el vocabulario del modelo en cada paso, garantizando por construcción la validez sintáctica de

la salida (Willard and Louf, 2023; Geng et al., 2025). El diseño no la excluye, pero no le confía el contrato: esa garantía se acopla al proveedor —que soporta sólo un subconjunto de los esquemas expresables (Geng et al., 2025)—, mientras que validar aguas abajo, en un filtro propio, mantiene la etapa agnóstica del proveedor y preserva la sustituibilidad de Invocación generativa (Subsección 3.1.3). Ambas son componibles: la validación estructural, la canonicalización y la compuerta de no conformidad permanecen aquí aun si la invocación emplea generación restringida, pues exceden lo que una gramática garantiza. En suma, Decodificación asegura que el análisis sea *estructuralmente* conforme —que tenga la forma que el esquema declara—, pero no que su *contenido* sea fiel al material empírico del que dice derivar; esa segunda garantía, de otra naturaleza, la asume el filtro siguiente.

Verificación. Interfaz:

- **Puerto de entrada** (11): `Análisis[]`.
- **Puerto de salida** (12): `Veredicto[]`.
- **Sintonizador**: definición del criterio de validez —qué se considera un análisis fiel a la evidencia— y selección del ejecutor de la verificación (Subsección 3.2.6).
- **Acceso al repositorio**: lee cada análisis estructurado y, siguiendo sus punteros de procedencia, el material empírico que lo sustenta —las noticias que su *prompt* incorporó por recuperación semántica—; escribe los registros `Veredicto`.
- **Responsabilidades**: [A] —criterio de validez y ejecutor los fija el experto—; [B] —el criterio es dato del estudio—; [D] —compuerta blanda: juzga, cuantifica y señala sin suprimir—.

Especificación funcional. Contrasta cada `Análisis` con la evidencia empírica de la que dice derivar y emite un juicio sobre su fidelidad. Para cada análisis del *recordset* de entrada, recupera del repositorio las noticias que su *prompt* incorporó por recuperación semántica y evalúa en qué medida las afirmaciones del análisis están efectivamente sostenidas por ese material —señalando, en particular, las afirmaciones no ancladas en la evidencia, esto es, las alucinaciones del modelo generativo—. Persiste el resultado como un registro

`Veredicto` que referencia al análisis juzgado, califica su estado —válido, observado o no sustentado—, cuantifica la confianza asociada y enumera los fragmentos de evidencia que respaldan o contradicen cada afirmación. Preserva la cardinalidad —emite un `Veredicto` por análisis— pero transforma el tipo que circula: aguas abajo ya no viaja el análisis sino el juicio sobre su fidelidad, del que aquél pende por referencia.

La verificación opera como una *compuerta blanda*: a diferencia de la compuerta de Decodificación —que descarta lo que no conforma el esquema—, aquí ningún análisis se suprime; los que no superan el criterio de validez se marcan y se propagan señalados, de modo que la decisión editorial última quede en manos del analista y no de un automatismo. Esta política responde de manera directa a tres brechas identificadas en la revisión sistemática (Sección 2.4): la fidelidad de la evidencia —si la auditabilidad lineal del estilo vuelve inspeccionable el recorrido de la evidencia (Subsección 3.1.3), la verificación da el paso siguiente y evalúa si ese recorrido efectivamente sostiene cada afirmación—, la calibración de la incertidumbre en los *scores* de análisis —que aquí se reifica en la confianza que el veredicto cuantifica— (Pavlyshenko, 2024), y la gobernanza editorial mediante procedimientos *human-in-the-loop* ante contenidos controvertidos. Que la compuerta sea blanda, y no una criba automática, es lo que la hace coherente con esta última brecha: el filtro no publica ni censura, sino que provee al experto el juicio fundamentado sobre el cual decidir.

Dos precisiones sobre su lugar en la arquitectura. En primer lugar, la verificación no reintroduce ciclos en la red de tubos: cuando un veredicto adverso motiva regenerar un análisis, esa reejecución es un fenómeno del proceso de uso —que tiene lugar entre corridas sucesivas del sistema, según se precisó en la Subsección 3.2.1— y no un tubo de retorno que vincule este filtro con los anteriores; el flujo interno permanece estrictamente acíclico. En segundo lugar, el filtro es agnóstico respecto de quién ejecuta la verificación: por la sustituibilidad que las invariantes 1 y 2 garantizan (Subsección 3.1.3), su realización interna —un procedimiento automático que asiste a un revisor humano, o un conjunto de agentes generativos que confrontan sus juicios— puede reemplazarse sin alterar su interfaz de puertos ni el resto del pipeline. El criterio de validez y la elección del ejecutor se incorporan por sintonizador, no por el flujo de datos, de modo que el aporte del experto —o la orquestación de los agentes— se mantiene fuera de la red de tubos, en línea con la frontera que la Subsección 3.2.6 establece para toda configuración de dominio. Cuando el ejecutor se apoya en agentes generativos que confrontan

sus juicios —caso documentado como *LLM-as-a-judge* (Zheng et al., 2023) y como verificación multiagente en la literatura de agentes basados en modelos de lenguaje (Xi et al., 2023)—, la sustituibilidad del filtro habilita reemplazar el procedimiento automático o el conjunto de agentes sin alterar el resto del pipeline.

Una última caracterización es importante. La literatura contemporánea sobre seguridad de LLMs denomina *guardrails* a los filtros especializados sobre la salida del modelo (Rebedea et al., 2023); la taxonomía de NeMo —*topical rails*, *fact-checking rails*, *safety rails*— se mapea directamente con los criterios de validez que el sintonizador de Verificación del presente framework admite configurar. El filtro de Verificación implementa, en términos arquitectónicos, un *guardrail* con la semántica adicional de trazabilidad documental (`sustento`) y calibración de incertidumbre (`confianza`). La necesidad de este filtro dentro del pipeline no es opcional: la investigación sobre vulnerabilidad de los sistemas LLM-integrados ante inyección de *prompts* (Liu et al., 2023) muestra que los pipelines de LLM, al exponer interfaces de texto entre filtros, quedan expuestos a salidas no fiables o no sustentadas que deben ser controladas en puntos explícitos de la cadena de procesamiento. La decisión de modelar ese control como un filtro propio —en lugar de confiar al modelo generativo la fiabilidad de su propia salida— cuenta además con respaldo empírico directo: el patrón *Self-Refine* de Madaan et al. (2023) demuestra que un LLM que se auto-reflexiona para mejorar su propia salida produce resultados sustancialmente más fiables; Shinn et al. (2023) extiende este patrón con memoria episódica de fallos previos (*Reflexion*), lo que sugiere que la iteración entre corridas que la presente arquitectura modela —iteración del experto, registrada en el grafo de procedencia del Apéndice A— es un fenómeno análogo al que la literatura contemporánea documenta a escala del LLM individual.

Normalización. Interfaz:

- **Puerto de entrada** (13): `Veredicto[]`.
- **Puerto de salida** (14): `Resultado`.
- **Sintonizador**: revisión del analista, definición de los criterios de consolidación —la política de tratamiento de los análisis observados por la verificación, la condición de inclusión que restringe la consolidación a los análisis cuya estructura toma un valor dado y la selección de las

claves del esquema que se consolidan— y definición de la variable de comparación contra la cual se cruzan los resultados: una agrupación que el corpus ya trae —portal, adscripción geográfica, fecha— o una que el experto declara mediante una tabla de asignación sobre esas mismas claves de procedencia (por ejemplo, portal → tipo de prensa), pues la agrupación pertinente para una pregunta es un dato del estudio y no del corpus ni del framework.

- **Acceso al repositorio**: lee los veredictos, por referencia los análisis que estos juzgan y, por *backward reachability* sobre el grafo de procedencia hasta las noticias, los metadatos de origen —portal, adscripción geográfica, fechas— sobre los que se define la variable de comparación; escribe el resultado consolidado del estudio, con los criterios y la variable aplicados materializados en su contenido.

- **Responsabilidades**: [A] —revisión del analista—; [B] —los criterios de consolidación y la variable de comparación son datos del estudio—; [D] —aplica la política editorial sobre los análisis observados—; [E] —materializa en el resultado los criterios con que consolidó—.

Especificación funcional. Consolida el *recordset* de veredictos —y los análisis verificados que estos referencian— en un único registro `Resultado` que representa el producto del estudio. Documenta el recorrido del proceso, organiza los datos de manera coherente, aplica la política de consolidación definida por el analista —que determina cómo se integran los análisis observados por la verificación—, compara los análisis con las variables definidas y produce la agregación que da sentido al análisis del *agenda-setting* a escala de corpus. Esa agregación toma la forma de la distribución de cada clave consolidada y de su cruce contra la variable de comparación —la proporción de cada categoría en cada grupo—, acompañada del resumen de los veredictos por grupo; una clave cuya categoría dominante concentra casi todas las unidades se señala como poco informativa, para que el analista la reporte sin interpretarla. La consolidación es determinística —no interviene ningún modelo generativo— y la lectura de los cruces queda en manos del analista. Es en este filtro donde la cardinalidad se reduce de un conjunto de veredictos a un resultado único consolidado, que se entrega a la capa de presentación —el sumidero del sistema— para su visualización en tableros y reportes.

3.2.3. Conectores: tubos tipados

Los tubos son los conectores del sistema: canales unidireccionales que enlazan el puerto de salida de un filtro con el puerto de entrada del siguiente, preservan el orden de los elementos que transportan y no modifican su contenido (Subsección 3.2.7). Cada tubo está tipado —transporta un *recordset* de un tipo declarado (Subsección 3.2.4)—: un conjunto ordenado de referencias cuyo contenido reside en el repositorio común (Subsección 3.2.5).

La caracterización de un tubo no se agota, sin embargo, en el tipo de sus elementos: comprende también su *cardinalidad*, esto es, el tamaño del *recordset* que transporta. La cardinalidad no se conserva a lo largo de la cadena: la mayoría de las etapas la preservan, la composición de contexto la redefine —al producir uno o varios *prompts* a partir del conjunto de noticias— y la normalización la reduce a la unidad —al consolidar los análisis en un único resultado—. Por ello, la firma de cada tubo se expresa como `Tipo[c]`, donde `c` denota la cardinalidad del *recordset* transportado.

El pipeline comprende seis tubos internos, además de las conexiones de la fuente y el sumidero con el entorno. La cardinalidad recorre la cadena $N \to N \to M \to M \to M \to M \to 1$, cuyos tramos se detallan a continuación con remisión a los puertos de la Figura 3.1.

Extracción produce el corpus: emite `Noticia[N]` (puerto 2), donde N es la cantidad de noticias recolectadas; la conexión previa —del portal al puerto 1— transporta el flujo crudo del portal, sin estructura de *recordset*, y queda por ello fuera de la notación tipada. El primer tubo (2→3) lleva ese `Noticia[N]` a Enriquecimiento semántico, que preserva la cardinalidad pero transforma el tipo: emite un `Embedding` por cada noticia (puerto 4), de modo que el tubo interno del filtro compuesto *Recuperación aumentada* (4→5) transporta `Embedding[N]` —las N representaciones vectoriales, cada una con la referencia a la noticia que codifica—. En este tramo la cardinalidad es la invariante y la representación es lo que cambia: cada noticia sigue presente en el flujo, pero mediada por su codificación semántica.

Composición de contexto redefine la cardinalidad: a partir de los N `Embedding` recibidos emite `Prompt[M]` (puerto 6), con $1 \leq M \leq N$, donde M es la cantidad de *unidades de análisis* que fija la estrategia —desde un único *prompt* agregado sobre el corpus ($M = 1$) hasta un *prompt* por noticia ($M = N$)—. Cada `Prompt` incorpora, por recuperación semántica, el subconjunto de noticias relevante a su unidad; la relación entre noticias y *prompts* es, por lo tanto, de muchos a muchos, y se registra en el propio

`Prompt` (Subsección 3.2.2).

Los tres tubos siguientes preservan M: Invocación generativa emite una `RespuestaIAG` por cada *prompt* (`Prompt[M]` → `RespuestaIAG[M]`, puerto 8), Decodificación una `Análisis` por cada respuesta (`RespuestaIAG[M]` → `Análisis[M]`, puerto 10) y Verificación un `Veredicto` por cada análisis (`Análisis[M]` → `Veredicto[M]`, puerto 12). Que la cardinalidad se mantenga en M a lo largo de estas etapas expresa que el análisis procede como un *map* sobre las unidades definidas en la composición: cada unidad se invoca, se decodifica y se verifica de manera independiente.

Finalmente, Normalización reduce la cardinalidad a la unidad: consolida los M `Veredicto` —y los análisis verificados que referencian— en un único registro `Resultado` (puerto 14) que, por ser un registro único y no un *recordset*, se tipa sin cardinalidad. Esta reducción es el *reduce* que clausura el *map* de las etapas previas. Vista en conjunto, la dinámica de cardinalidad del pipeline —seleccionar M unidades sobre N noticias, procesarlas de forma independiente y consolidarlas en un único resultado— es lo que permite escalar el análisis a corpus que no cabrían en una sola invocación del modelo generativo.

3.2.4. Especializaciones aplicadas: pipelines y tubos tipados

El diseño adopta dos de las especializaciones canónicas que el estilo admite —documentadas tanto en la formulación seminal (Garlan and Shaw, 1994) como en el catálogo de Cristiá and Pomponio (2025, §3.12)—: la de *pipelines* y la de *tubos tipados*, aplicadas de manera combinada.

La especialización de *pipeline* restringe la topología a una secuencia lineal de filtros, cada uno con un único puerto de entrada y un único puerto de salida (Cristiá and Pomponio, 2025, §3.12.1); en los términos de Garlan and Shaw (1994), "pipelines…restrict the topologies to linear sequences of filters". Su adopción responde a la naturaleza del problema: el análisis de noticias procede como una cadena de transformaciones sucesivas —extracción, enriquecimiento, composición, invocación, decodificación y normalización— sin ramificaciones ni puntos de decisión en su configuración base, de modo que la restricción a un único puerto de entrada y de salida por filtro no supone pérdida de expresividad para el ciclo completo de análisis. Las variantes analíticas que requieran recombinar etapas se acomodan por la composicionalidad del

estilo (Subsección 3.1.1), no por la ramificación de la topología.

La especialización de *tubos tipados* exige que cada tubo transporte un dato de tipo bien definido —"typed pipes, which require that the data passed between two filters have a well-defined type" (Garlan and Shaw, 1994)—, de manera que un puerto de tipo `T` sólo pueda conectarse a un tubo del mismo tipo, sin que el conector realice conversión alguna (Cristiá and Pomponio, 2025, §3.12.2). En el framework, cada tubo declara el tipo del *recordset* que transporta —`Noticia[]`, `Embedding[]`, `Prompt[]`, `RespuestaIAG[]`, `Análisis[]`, `Veredicto[]` y `Resultado`, según la tabla de referencia de la Figura 3.1; su estructura interna se especifica en el Apéndice A—. El tipado cumple una doble función: documenta el contrato entre etapas —qué produce cada filtro y qué espera el siguiente— y permite detectar tempranamente errores de composición, al volver la conexión de dos filtros incompatibles un error de tipo antes que un fallo en tiempo de ejecución. Conviene señalar el compromiso que supone: al declarar tipos concretos, los filtros dependen más del contexto que en la formulación no tipada, lo que reduce la flexibilidad de recomposición a cambio de mayor seguridad y de un tratamiento más ordenado de los datos (Cristiá and Pomponio, 2025, §3.12.2).

3.2.5. Deformación aplicada: repositorio común sobre PostgreSQL

La caracterización del framework incorpora una deformación canónica del estilo: un *repositorio común* de datos, materializado sobre PostgreSQL a través del ORM de Odoo, del cual todos los filtros leen y sobre el cual escriben el contenido que los tubos no transportan (Cristiá and Pomponio, 2025, §3.13.1). Por los tubos no circula el contenido completo de las noticias, los *prompts* o los análisis, sino *recordsets* —conjuntos ordenados de referencias a los registros—, mientras que el contenido reside en el repositorio y se recupera bajo demanda.

Esta separación entre referencia y contenido no es una imposición sobre la plataforma, sino la semántica nativa del ORM de Odoo. La documentación oficial define que "every model instance is a 'recordset', i.e., an ordered collection of records of the model" y que el método `browse` "returns a recordset for the ids provided as parameter" —de modo que un *recordset* se representa por sus identificadores: `browse([7, 18, 12])` produce `res.partner(7, 18, 12)`—. El contenido de los campos no viaja con la referencia: la propia

plataforma “maintains a cache for the fields of the records, so that not every field access issues a database request”, recuperándolo del repositorio sólo al accederlo (Odoo S.A., 2024). Transmitir recordsets por los tubos equivale, entonces, a transmitir referencias tipadas cuyo contenido se materializa contra el repositorio común.

A nivel del conector, este esquema instancia el patrón *Claim Check* (Hohpe and Woolf, 2003): “store message data in a persistent store and pass a Claim Check to subsequent components”. El patrón no se agota en una recomendación de catálogo: la literatura académica sobre patrones de integración empresarial lo ha reificado. Al reexpresar el catálogo de Hohpe and Woolf (2003) en una notación de modelado formal, Ritter (2014) caracteriza el Claim Check como un *call by reference* sobre la carga útil del mensaje —el contenido se almacena en un repositorio de datos y sólo la referencia se transmite al siguiente elemento del flujo—. El mismo principio —transportar una referencia y no la carga útil— es práctica corriente en los orquestadores de *pipelines* de datos y de aprendizaje automático: Apache Airflow recomienda comunicar por su canal sólo mensajes pequeños y referenciar a un almacenamiento remoto los datos mayores, empujando la ruta del dato y no el dato (Apache Software Foundation, 2026), y Kubeflow Pipelines distingue los parámetros, pasados por valor, de los *artifacts*, que son una referencia (`.uri`) al objeto en un almacén externo (The Kubeflow Authors, 2026).

El alcance de esta decisión merece una precisión. En los sistemas mencionados, el reparto entre referencia y valor suele gobernarse por un umbral de tamaño —los datos pequeños viajan en línea por el conector y sólo los grandes se referencian al almacén—, configurando arquitecturas híbridas. El diseño aquí presentado adopta, en su configuración base, el paso uniforme por referencia, decisión que se apoya en que el repositorio común constituye la única fuente de verdad del sistema y en que la propia plataforma representa sus colecciones como recordsets referenciales. La adopción de un repositorio junto a la red de tubos no abandona el estilo de Tubos y Filtros, sino que configura una *arquitectura heterogénea* en el sentido de Garlan and Shaw (1994), en la que los componentes acceden a un repositorio por parte de su interfaz pero interactúan a través de tubos.

3.2.6. Sintonizadores

El catálogo de Cristiá and Pomponio (2025, §3.13.2) documenta los *sintonizadores* como una deformación canónica del estilo: procedimientos que un

filtro expone para que un componente externo —típicamente una interfaz gráfica— altere o ajuste su comportamiento sin participar del flujo de datos del sistema. El diseño reifica esta deformación para modelar la intervención del experto del dominio, que parametriza el comportamiento de ciertos filtros sin inyectar datos en el pipeline.

Esta caracterización no es exclusiva del catálogo: tiene un precedente directo en la formulación seminal del estilo. Al resolver la interacción con el usuario en su caso de estudio del osciloscopio, Garlan and Shaw (1994) asocian a cada filtro una *control interface* "that allows an external entity to set parameters of operation for the filter", y modelan así los filtros como funciones de orden superior "for which the configuration parameters determine what data transformation the filter will perform"; su virtud es que "decouples the signal processing functions…from the actual user interface". El sintonizador reifica esa interfaz de control: desacopla la parametrización que aporta el experto del procesamiento que realiza el filtro. La misma separación entre configuración y datos reaparece en las bibliotecas modernas de aprendizaje automático —en *scikit-learn*, el constructor de un estimador "does not see any actual data…All it does is attach the given parameters to the object" (Buitinck et al., 2013)—.

En el framework exponen sintonizador únicamente los cuatro filtros cuyo comportamiento depende de decisiones de dominio —y son, por ello, los puntos donde se realiza la responsabilidad de intervención del experto, [A]—: Extracción (selección de portales, reglas de extracción y prioridad de monitoreo), Composición de contexto (selección del *template*, variables de contexto, texto libre y parámetros de recuperación y del modelo generativo), Verificación (criterio de validez de los análisis y selección del ejecutor de la verificación) y Normalización (criterios de consolidación y variables de comparación). Los filtros restantes —Enriquecimiento, Invocación y Decodificación— no lo hacen: su configuración es de carácter técnico y la fija el implementador, al margen de todo sintonizador (Subsección 3.2.2). Todos los sintonizadores se invocan desde la capa de presentación —nunca desde otro filtro— y se grafican en la Figura 3.1 con la flecha originada en un círculo que apunta a cada filtro configurable.

El más elaborado es el de Composición de contexto, y es el único que se realiza sobre dos superficies de configuración distintas. La primera es un panel de creación de *templates* de *prompt*: un subsistema gestionado que habilita al experto configurar, parametrizar y reutilizar plantillas en lugar de resolver esa lógica de manera *ad hoc*, como es habitual en las aplicaciones basadas en

modelos de lenguaje. Cada *template* declara el objetivo del análisis —alcance, paradigma de análisis, referencias a considerar—, las instrucciones parametrizadas con variables de contexto y el esquema de la respuesta esperada, reificando los datos de entrada para la construcción de *prompts* anticipados en el Capítulo 1; su estructura se especifica en el Apéndice A. La segunda es el formulario de sintonía del estudio, donde el experto selecciona la plantilla, liga valores a las variables que ésta declara, agrega las aclaraciones de texto libre y fija los parámetros de recuperación y del modelo generativo. La partición responde a que ambas configuraciones tienen ciclos de vida distintos: el *template* es una entidad persistente del repositorio común, reutilizable entre estudios y ajena a la monotonía del flujo (Apéndice A.9), mientras que los valores sintonizados pertenecen a una corrida y quedan fijados con ella; editar la plantilla dentro del estudio obligaría a bifurcar una copia por corrida y disolvería la reutilización que el panel persigue. Que la sintonía se reparta en dos formularios no la aleja del estilo: el sintonizador se define por lo que parametriza y por el momento en que el filtro lo lee —una única vez, al invocarse—, no por la descomposición que adopte la capa de presentación; es, precisamente, el desacople entre el procesamiento y la interfaz de usuario que Garlan and Shaw (1994) señalan como virtud de la interfaz de control. La práctica corriente de la ingeniería de *prompts* converge con esa separación: se recomienda declarar variables en la plantilla y ligar sus valores por fuera, conservar el *prompt* en un artefacto separado del código, y documentar cada intento contra una plantilla nombrada y versionada, registrando por corrida el modelo, la temperatura, la longitud de salida y los parámetros de recuperación (Boonstra, 2024). Su realización concreta en el módulo se retoma en la Sección 4.3.

Que la intervención del experto se canalice por sintonizadores —y no por el flujo de datos— es lo que la mantiene compatible con el estilo: el sintonizador ajusta *cómo* transforma un filtro, sin alterar la red de tubos ni agregar un puerto al pipeline. La configuración así incorporada es un parámetro del filtro, no un dato que circule por la arquitectura.

3.2.7. Verificación de los invariantes esenciales

Esta subsección verifica que el diseño resultante —un *pipeline* de tubos tipados con repositorio común— satisface los cuatro invariantes esenciales enunciados en la Subsección 3.1.2.

Identidad de los filtros (invariante 1). Ningún filtro referencia a otro por

su identidad. La comunicación se produce exclusivamente a través de los puertos tipados —por los que circulan recordsets— y del repositorio común, sobre el que cada filtro lee y escribe registros sin conocer qué filtro los produjo ni cuál los consumirá. La sustituibilidad analizada en la Subsección 3.1.3 es consecuencia directa de esta propiedad.

Estado interno de los filtros (invariante 2). Ningún filtro accede al estado interno de otro. El repositorio común es un almacén externo compartido —conforme al patrón Claim Check— y no el estado en ejecución de ningún filtro; que se materialice sobre la base de datos de Odoo es una particularidad de la implementación que no incide en esta propiedad.

Orden de cómputo (invariante 3). La corrección del resultado no depende del orden en que cada filtro realiza sus cálculos internos. Las dependencias que la topología establece —un filtro no puede materializar un contenido que el filtro anterior aún no persistió— son dependencias de flujo propias del grafo de tubos, no dependencias del *scheduling* interno, según se precisó en la Subsección 3.1.2. Disponibles sus entradas, cada filtro produce el mismo resultado con independencia del orden relativo de cómputo.

Integridad del transporte (invariante 4). Los tubos transportan recordsets —referencias— sin alterarlos; toda transformación del contenido ocurre dentro de los filtros y queda persistida en el repositorio. El conector no realiza conversión ni modificación alguna sobre lo que transmite, en línea con la especialización de tubos tipados adoptada en la Subsección 3.2.4.

3.2.8. Ventajas y limitaciones de la elección

La elección del estilo se apoya en tres ventajas ya fundamentadas en la Subsección 3.1.3, que aquí sólo se recapitulan: la *sustituibilidad* de los filtros —que permite actualizar la extracción o reemplazar el modelo generativo sin reescribir el resto del pipeline—, la *auditabilidad lineal* del flujo —que localiza cada transformación en un filtro identificable, condición para la trazabilidad de la evidencia— y el *cierre composicional* —que habilita recombinar etapas para distintos casos de estudio—. Sobre esas ventajas se construye el aporte del capítulo; resta señalar las limitaciones y compromisos que la elección concreta acarrea.

En primer lugar, el diseño no es una instancia pura del estilo, sino una arquitectura heterogénea que incorpora un repositorio común (Subsección 3.2.5). El repositorio es un componente del que todos los filtros dependen: acopla el pipeline a la plataforma de persistencia —Odoo sobre PostgreSQL— y

concentra en un único almacén la disponibilidad del contenido. Este acoplamiento es el precio de las ventajas que el repositorio aporta —evitar la copia de grandes volúmenes por los tubos y disponer de una única fuente de verdad— y debe asumirse como una decisión de diseño antes que como una propiedad del estilo canónico.

En segundo lugar, la decisión de transportar *siempre* referencias por los tubos —y no un reparto híbrido por tamaño, como es habitual en los orquestadores relevados (Subsección 3.2.5)— privilegia la uniformidad y la coherencia con el modelo referencial de Odoo, pero introduce un acceso al repositorio en cada etapa aun cuando el volumen del dato no lo exigiría. Es un compromiso favorable en el contexto de la plataforma elegida, no una optimización universal.

En tercer lugar, el framework se aparta del caso de uso paradigmático del estilo —el procesamiento incremental de un *stream*—: opera sobre *recordsets* que representan corpus completos, y varias de sus etapas no son incrementales (la composición agrega la cardinalidad y la normalización la reduce a un único resultado). Garlan and Shaw (1994) señalan que, en su organización *batch*, el estilo resulta menos apropiado para interacciones incrementales; la presente elección es coherente con un procesamiento por lotes —acorde al análisis de un corpus de noticias— y no con la vigilancia en tiempo real, escenario que excedería la configuración base aquí caracterizada.

En cuarto lugar, la etapa de Verificación introduce un compromiso propio cuando su ejecutor es un revisor humano: la compuerta blanda que habilita la gobernanza editorial *human-in-the-loop* (Subsección 3.2.2) supone, en ese caso, una dependencia síncrona de un actor externo que añade latencia y limita el rendimiento del pipeline, en tensión con la operación autónoma por lotes que las demás etapas asumen. El rigor editorial que la etapa aporta tiene, así, un costo de *throughput*; que ese costo se pague —o se difiera hacia un ejecutor automático— es, precisamente, lo que la agnosticidad respecto del ejecutor deja como decisión configurable (Subsección 3.2.6).

3.2.9. Contratos de datos del pipeline

El pipeline transporta por los tubos *recordsets* tipados cuyo contenido reside en el repositorio común (Subsección 3.2.5). El framework define ocho tipos —siete que circulan por los tubos y uno que se mantiene en el repositorio como ancla de la definición del estudio—, cada uno con su firma de procedencia y su cardinalidad. Estos tipos constituyen los *contratos de datos* del

sistema: fijan qué produce cada filtro y qué puede esperar el siguiente, con independencia de la plataforma sobre la que el framework se materialice.

Tabla 3.1: Tipos de datos del pipeline, cardinalidades y responsabilidades transversales que cada contrato realiza. Marcadores de responsabilidades: [A] experto · [B] estudio · [C] generativa · [D] control editorial · [E] procedencia · [F] preexistente (Tabla 3.2).

Tipo	**Producido por**	**Tubo**	**Card.**	**Contenido principal**	**Resp.**
Noticia	Extracción	2→3	*N*	Título, texto, link, medio, fecha, reglas	[E] [F]
Embedding	Enriquecimiento	4→5 (interno)	*N*	Vector + ref. a noticia	[C] [E]
Prompt	Composición de contexto	6→7	*M*	Texto del prompt, parámetros, refs. a noticias y *template*	[B] [E]
RespuestaIAG	Invocación	8→9	*M*	Respuesta cruda del modelo + ref. al prompt	[C] [E]
Análisis	Decodificación	10→11	*M*	Estructura analítica validada + ref. a respuesta	[B] [E]
Veredicto	Verificación	12→13	*M*	Estado, confianza, sustento + ref. al análisis	[D] [E]
Resultado	Normalización	14 (salida)	1	Producto consolidado + refs. a veredictos	[B] [D] [E]
Template	(repositorio)	—	—	Objetivo, instrucciones, esquema de respuesta	[A] [B]

Dos criterios transversales gobiernan el diseño de estos contratos. En pri-

mer lugar, cada tipo contiene únicamente la carga útil que algún filtro posterior consume, junto con la procedencia del dato —cuándo se produjo y a partir de qué registros—; toda clave adicional volvería al contrato más exigente de lo que el pipeline requiere. En segundo lugar, donde el contenido depende del caso analítico —`Análisis`, `Resultado`—, el framework fija el sobre (procedencia y carga estructurada) y delega el esquema interno a la definición del *template* del estudio. La aplicación uniforme de ambos criterios, así como la especificación detallada de cada tipo, se presentan en el Apéndice A.

Leídos en conjunto, los siete tipos que circulan por los tubos definen sobre el repositorio un *grafo de procedencia* en el sentido del modelo PROV del W3C (Moreau and Missier, 2013): un grafo dirigido cuyos vértices son los registros persistidos y cuyas aristas —las claves de procedencia de cada tipo— van de cada registro a aquellos de los que se derivó, de modo que el grafo recorre el pipeline en sentido inverso. Reconstruir el origen de un resultado o de un veredicto es entonces una consulta de *backward reachability* sobre ese grafo, el tipo de consulta al que la literatura de procedencia reduce las preguntas sobre qué datos intervinieron en la producción de otro (Cuevas-Vicenttín et al., 2012): desde el `Resultado` final puede obtenerse, como el conjunto de sus ancestros, qué veredictos de verificación lo componen, sobre qué análisis se pronuncia cada uno, de qué respuestas crudas provienen esos análisis, con qué *prompts* —y bajo qué *template* y parámetros— se generaron, sobre qué noticias se apoyó cada composición y de qué portal fue extraída cada noticia, cuándo, y bajo qué reglas de extracción. Esta propiedad estructural del sistema de tipos materializa, a nivel del dato, la auditabilidad lineal que motivó la elección del estilo (Subsección 3.1.3) y resuelve la brecha de trazabilidad documental identificada en la revisión sistemática (Sección 2.4); es la realización, en la capa de datos, de la responsabilidad de procedencia [E]. Que todos los tipos del flujo la compartan —y sólo el *template*, que no circula, quede fuera— se refleja en la columna de responsabilidades de la Tabla 3.1.

3.3. Síntesis del capítulo

La Tabla 3.2 consolida el registro maestro de las responsabilidades transversales introducidas en la Sección 1.3: para cada marcador, la obligación que nombra y los elementos que la realizan en la arquitectura de este capítulo, en

el módulo de Odoo (Capítulo 4), en el mapa de procesos (Capítulo 5) y en el caso de aplicación (Capítulo 6). Los marcadores conservan su identidad a través de las vistas mientras cambia su realización: la misma responsabilidad es un sintonizador en la arquitectura, un formulario en el módulo, una actividad con actor en el mapa de procesos y una decisión documentada en el caso. El registro es la fuente única a la que remite cada marcador del documento; su función es hacer inspeccionable la traza desde las garantías comprometidas en el objetivo general hasta el lugar donde cada una se cumple, no acreditar por sí mismo que se cumplan.

Tabla 3.2: Registro maestro de responsabilidades transversales: realización de cada marcador en la arquitectura, el módulo, el mapa de procesos y el caso de aplicación. Las celdas marcadas *(por completar)* corresponden a capítulos en redacción.

M.	**Responsabilidad**	**Arquitectura (Cap. 3)**	**Módulo Odoo (Cap. 4)**	**Procesos (Cap. 5)**	**Caso (Cap. 6)**
[A]	Intervención del experto	Sintonizadores de Extracción, Composición, Verificación y Normalización (§ 3.2.6); invocados sólo desde la capa de presentación; *template* como configuración editada.	*(por completar)*	Actividades con actor Experto de dominio: 02.01, 02.02, 02.05, 03.03–03.05; actor Investigador o analista: 04.03.	*(por completar)*

continúa en la página siguiente

Tabla 3.2 (continuación)

M.	Responsabilidad	Arquitectura (Cap. 3)	Módulo Odoo (Cap. 4)	Procesos (Cap. 5)	Caso (Cap. 6)
[B]	Independencia del estudio	`Template` (objetivo, esquema de respuesta); Decodificación valida contra el esquema del estudio sin configuración propia; sobre fijo y esquema delegado en `Análisis`, `Veredicto.sustento` y `Resultado` (Apéndice A, criterio 3).	*(por completar)*	Proceso 03: 03.01 creación de *templates*, 03.03 selección, 03.04 customización; 03.06 composición.	*(por completar)*
[C]	Frontera generativa	Invocación y Decodificación como filtros propios; Enriquecimiento sustituible; los contratos omiten proveedor y modelo (Apéndice A, criterio 2).	*(por completar)*	Proceso 04: 04.01 procesamiento IAG, 04.02 decoder; proceso 02: 02.04 *embedding*.	*(por completar)*
[D]	Control editorial	Verificación como compuerta blanda; `Veredicto` (estado, confianza, sustento); política de consolidación de los análisis observados en Normalización.	*(por completar)*	04.03 normalización (comparación y análisis de resultados); procesos 05 y 06.	*(por completar)*

continúa en la página siguiente

Tabla 3.2 (continuación)

M.	Responsabilidad	Arquitectura (Cap. 3)	Módulo Odoo (Cap. 4)	Procesos (Cap. 5)	Caso (Cap. 6)
[E]	Procedencia	Auditabilidad lineal (§ 3.1.3); monotonía del repositorio; claves de referencia y `fecha_registro` en los siete tipos; cadena `Resultado` → `Noticia` (§ 3.2.9).	*(por completar)*	04.03: documentación del recorrido completo del proceso.	*(por completar)*
[F]	Infraestructura preexistente	Extracción sobre el scraper preexistente; repositorio sobre PostgreSQL y el ORM de Odoo (§ 3.2.5).	*(por completar)*	Proceso 02: 02.03 (Aplicación de Scraping).	*(por completar)*

Capítulo 4

Implementación del framework en Odoo

Este capítulo presenta la realización del diseño del Capítulo 3 como módulo *addon* de Odoo Community, en cumplimiento del objetivo específico OE1. Lo que cambia respecto del capítulo anterior es el nivel de descripción, no el objeto descripto: los mismos filtros, tubos, tipos y sintonizadores que allí se caracterizaron con independencia de la plataforma aparecen aquí como modelos, campos, vistas y parámetros de un sistema concreto. La arquitectura fijó los contratos —qué recibe y qué emite cada filtro, qué configura el experto y qué decide el implementador, qué se persiste y qué circula por referencia—; este capítulo muestra cómo la plataforma los realiza, qué mecanismos nativos de Odoo se aprovechan para ello y qué compromisos introduce esa realización.

El capítulo se organiza alrededor de una regla de correspondencia que se enuncia de antemano y se sostiene a lo largo de todas sus secciones: *cada elemento de la arquitectura tiene exactamente una realización nombrada en el módulo*. Cada filtro es un modelo abstracto que expone una única operación —recibe el *recordset* de su puerto de entrada y la configuración de su sintonizador, persiste sus registros y devuelve el *recordset* de su puerto de salida—; cada tipo del flujo es un modelo persistente cuyos campos son las claves del contrato correspondiente del Apéndice A; cada tubo es un campo relacional que conserva el *recordset* emitido por el puerto que le corresponde; cada sintonizador es un formulario desde el que el experto del dominio fija sus parámetros; el repositorio común es el ORM de Odoo sobre PostgreSQL; la fuente es el scraper preexistente del entorno institucional, y el sumidero, las

vistas de tablero nativas de la plataforma. La Tabla 4.1 consolida esa correspondencia y constituye la guía de lectura del capítulo: para cada elemento del Capítulo 3 indica su realización en el módulo, la superficie desde la que se lo configura —y quién lo configura— y las responsabilidades transversales de la Sección 1.3 que esa realización honra. Los marcadores [A] a [F] son, aquí como en el resto del documento, la columna vertebral de la exposición: cada sección del capítulo señala qué responsabilidad realiza y con qué mecanismo concreto, y la síntesis final (Sección 4.4) vuelca ese recorrido en la columna correspondiente del registro maestro (Tabla 3.2).

Tabla 4.1: Correspondencia entre los elementos de la arquitectura (Capítulo 3) y su realización en el módulo `noticias_iag`. La columna *Superficie de configuración* distingue los sintonizadores, que fija el experto del dominio, de la configuración técnica, que fija el implementador; los elementos sin superficie propia no exponen configuración. Marcadores de responsabilidades: [A] experto · [B] estudio · [C] generativa · [D] control editorial · [E] procedencia · [F] preexistente (Tabla 3.2).

Elemento (Cap. 3)	**Puertos**	**Realización en el módulo**	**Superficie de configuración**	**Resp.**
Filtros				
Extracción	1→2	Scraper preexistente `wsfnoticias`: modelos de portales y reglas, acción planificada de la plataforma (Sección 4.1).	Sintonizador: formularios de portales, reglas y términos de `wsfnoticias`; selección del corpus del estudio sobre lo ya extraído.	[A] [F]
Enriquecimiento semántico	3→4	`iag.filtro.enriquecimiento`	Configuración técnica: modelo de *embedding* y largo máximo del texto codificado.	[C]

continúa en la página siguiente

Tabla 4.1 (continuación)

Elemento (Cap. 3)	**Puertos**	**Realización en el módulo**	**Superficie de configuración**	**Resp.**
Composición de contexto	5→6	`iag.filtro.composicion`; ensamblador único del *prompt*, compartido con la previsualización del panel de *templates*.	Sintonizador: panel de *templates* y solapa *Composición* del estudio (plantilla, variables, texto libre, estrategia, parámetros de recuperación y del modelo). Configuración técnica: largo máximo de cada noticia en el material empírico.	[A] [B]
Invocación generativa	7→8	`iag.filtro.invocacion`; servicio `iag.proveedor` con adaptadores intercambiables.	Configuración técnica: adaptador, servicio, modelo, credenciales, tiempo de espera y modo de salida estructurada.	[C]
Decodificación	9→10	`iag.filtro.decodificacion`; validador del esquema y canonicalización.	Configuración técnica: política ante la no conformidad (reparar o descartar).	[B] [C]
Verificación	11→12	`iag.filtro.verificacion`; ejecutor generativo (juez) o humano.	Sintonizador: solapa *Verificación* del estudio (criterio de validez, ejecutor); formulario de veredicto para el revisor humano.	[A] [B] [D]
Normalización	13→14	`iag.filtro.normalizacion`; consolidación determinística.	Sintonizador: solapa *Normalización* del estudio (política sobre los observados, variable de comparación, condición de inclusión, claves a consolidar).	[A] [B] [D] [E]
Tipos del flujo (un modelo por contrato del Apéndice A)				
`Noticia`	2→3	`wsf_noticias_resultados` (preexistente).	—	[E] [F]

continúa en la página siguiente

Tabla 4.1 (continuación)

Elemento (Cap. 3)	**Puertos**	**Realización en el módulo**	**Superficie de configuración**	**Resp.**
`Embedding`	4→5	`iag.embedding`	—	[C] [E]
`Prompt`	6→7	`iag.prompt`	—	[B] [E]
`RespuestaIAG`	8→9	`iag.respuesta`	—	[C] [E]
`Análisis`	10→11	`iag.analisis`	—	[B] [E]
`Veredicto`	12→13	`iag.veredicto`, con el sustento como registros propios.	—	[D] [E]
`Resultado`	14	`iag.resultado`	—	[B] [D] [E]
Elementos fuera del flujo de datos				
`Template`	—	`iag.template`	Panel de *templates*: objetivo, instrucciones, variables, esquema de respuesta, previsualización.	[A] [B]
Repositorio común	—	ORM de Odoo sobre PostgreSQL; los *recordsets* de los tubos son referencias a sus registros.	—	[E] [F]
Configuración del pipeline	—	`iag.estudio`: la instancia de la red de tubos, sus sintonizadores, su estado de avance y la documentación de la corrida.	Formulario de estudio: reúne los sintonizadores en solapas y conserva la configuración técnica vigente al momento de la corrida.	[A] [E]
Sumidero	14	Proyección de sólo lectura `iag.resultado.fila` sobre el resultado, con vistas de tabla dinámica y gráfico de la plataforma; ficha del estudio.	—	[D] [E]

La tabla exhibe una asimetría que conviene explicitar desde el inicio, porque delimita el aporte de la presente tesina frente a lo que reutiliza del entorno institucional ([F]). El módulo se construye como un *addon* nuevo,

denominado `noticias_iag`, que declara su dependencia del *addon* preexistente `wsfnoticias` y no lo modifica: el filtro de Extracción y el tipo `Noticia` son los de la infraestructura preexistente, tal como se caracterizan en la Sección 4.1, y los seis filtros restantes, los seis tipos que producen, el *template*, el estudio y el sumidero son construcción propia. Que la frontera entre ambos coincida exactamente con el primer tubo del pipeline no es casual: es la sustituibilidad del estilo puesta en acto, pues el resto del sistema depende del scraper únicamente a través del contrato `Noticia`, y podría montarse sobre otro extractor que lo respetara sin modificación alguna. En el mismo espíritu de delimitación, el capítulo distingue en todo momento tres niveles de evidencia: lo *implementado*, que es lo que la tabla consigna y las pruebas de la Sección **??** ejercitan; lo *previsto con la misma interfaz*, esto es, sustituciones que el diseño admite sin alterar los contratos pero que no se construyeron —un índice vectorial en la base de datos en lugar del cómputo de similitud en memoria, otros adaptadores de proveedor, la ejecución de los filtros en segundo plano—; y lo que queda *fuera de alcance*. Nada de lo que se describe como implementado carece de correlato en el código.

Un elemento de la tabla no tiene contraparte nominal en el Capítulo 3 y merece presentación aparte: el *estudio*. La arquitectura caracteriza el estilo y su pipeline base; el estudio es una *instancia* de esa red —en el sentido en que Garlan and Shaw (1994) distinguen el estilo de los sistemas que lo instancian—: fija qué *recordset* ocupa cada tubo, con qué valores de sintonizador se invocó cada filtro configurable, en qué etapa se encuentra la corrida y qué configuración técnica regía al ejecutarla. Es, por lo tanto, la unidad de corrida del framework y el objeto sobre el que el experto opera: crea un estudio, selecciona su corpus, sintoniza los filtros, ejecuta el pipeline y lee el resultado; una nueva iteración es otro estudio, enlazado al anterior, que hereda su configuración y vuelve a recorrer la red. Al igual que el *template*, el estudio no es un tipo del flujo ni circula por tubo alguno: pertenece a la configuración, no a los datos, y queda por ello fuera de la monotonía del repositorio (Apéndice A.10). Su papel como realización de la procedencia ([E]) es, sin embargo, central: mientras el grafo de procedencia entre los tipos reconstruye *a partir de qué datos* se produjo un resultado, el estudio conserva *con qué configuración y cuándo*, y documenta la corrida —cardinalidades por tubo, descartes, tiempos— en una ficha generada exclusivamente a partir de lo persistido.

La tabla anticipa también una partición que atraviesa todo el capítulo: la de las dos superficies de configuración. Los sintonizadores del Capítulo 3

se realizan como formularios accesibles al experto del dominio —el panel de *templates* y las solapas del estudio—, y son la vía explícita por la que ingresa toda decisión de dominio ([A]). La configuración técnica —proveedor, modelos, credenciales, fragmentación, modo de salida estructurada, política ante la no conformidad— se realiza como parámetros del sistema editables únicamente desde los ajustes de administración de la plataforma, y es potestad del implementador ([C]). Que ambas superficies sean distintas, estén sujetas a permisos distintos y queden registradas por separado en cada estudio es lo que permite afirmar que el experto controla el estudio sin conocer el proveedor, y que el implementador controla el proveedor sin conocer el estudio ([B]).

4.1. Infraestructura preexistente del entorno CAETI/CIM

Esta sección caracteriza la infraestructura sobre la que el módulo se monta y cumple, con ello, la obligación de delimitación que nombra la responsabilidad [F]: dejar establecido qué reutiliza el framework del entorno institucional y qué construye la presente tesina. La caracterización es factual y se apoya en dos fuentes: la descripción publicada por el propio equipo que la desarrolló (Raimondo Anselmino et al., 2024) y el código fuente del *addon* institucional, en la versión de su repositorio a la que esta tesina tuvo acceso. Donde ambas difieren, se consigna lo que el código hace, porque es sobre el código, y no sobre su descripción, que el módulo se apoya.

Conviene anticipar que la infraestructura preexistente comprende dos cosas de naturaleza distinta, y que la sección las trata por separado. Por un lado, un *sustrato*: la plataforma, el *addon* institucional con sus modelos, permisos y menús, y la base de datos que el Capítulo 3 caracterizó como repositorio común. El sustrato no transforma ni decide nada durante el ciclo de análisis; es el suelo sobre el que corre el pipeline. Por otro lado, un *filtro*: la Extracción, que el pipeline reutiliza tal como está, con sus puertos, su sintonizador y su modo de disparo. Es sobre esta segunda parte que recae la verificación de que el contrato `Noticia` del Apéndice A se cumple.

4.1.1. La plataforma y el *addon* institucional

La infraestructura fue desarrollada en el Centro de Altos Estudios en Tecnología Informática (CAETI) de la Universidad Abierta Interamericana, sede Rosario, y se emplea en la línea de investigación sobre discursos mediatizados que ese centro sostiene en articulación con el Centro de Investigaciones en Mediatizaciones (CIM) de la Universidad Nacional de Rosario. Raimondo Anselmino et al. (2024) la describen como una arquitectura de seis componentes: los portales de noticias como fuente primaria, registrados en el sistema con una prioridad de monitoreo; las bibliotecas de Python para el acceso y la extracción; los módulos de tareas planificadas de Odoo, que disparan la extracción según periodicidad y prioridad; el propio framework Odoo Community, como entorno de ejecución que provee base de datos PostgreSQL, capa de servidor y *frontend* web; el módulo *addon* de la aplicación, que concentra la configuración de portales y reglas, la extracción y la visualización de resultados; y un módulo de recuperación aumentada, concebido como capa semántica de análisis sobre los datos recolectados. De esos seis componentes, los cinco primeros son el sustrato y el filtro que esta tesina reutiliza; el sexto es la capa que el framework del Capítulo 3 formaliza y reemplaza, según se precisa al final de esta subsección.

El *addon* institucional tiene por nombre técnico `wsfnoticias` y depende únicamente de los módulos base y de contactos de la plataforma. Sus modelos de datos se organizan en tres grupos. El primero configura la extracción: los *portales* (`wsf_noticias_medios`), cada uno asociado a un contacto de la plataforma —el *partner* que identifica al medio— y provisto de la dirección de su sitio, la de su canal RSS si lo tiene, un límite de publicaciones por corrida, una prioridad de monitoreo, un estado de activación y una adscripción geográfica (`departamento`); las *reglas* (`wsf_noticias_reglas`), que combinan tres conjuntos de *términos* (`wsf_noticias_terminos`) en las relaciones lógicas de disyunción, conjunción y negación, con un estado de activación propio; y una *secuencia* de monitoreo (`wsf_noticias_secuencia`) que registra, por prioridad, hasta qué portal llegó la última pasada. El segundo grupo persiste el producto de la extracción: los *resultados* (`wsf_noticias_resultados`), donde cada registro es una publicación extraída, y las *categorías* con que un operador puede etiquetarlos manualmente. El tercer grupo es de soporte: un registro de eventos, un modelo de exclusiones y una vista de mapa por coordenadas de los portales, además del *laboratorio* que se describe más abajo. La seguridad define dos grupos de usuarios —*Analista de Noticias*,

con acceso completo, y *Operadores de Noticias*, restringidos por una regla de dominio a las publicaciones ya valoradas— y un menú raíz, *Noticias*, bajo el cual cuelgan todas las vistas. El módulo de esta tesina reutiliza esos dos grupos y ese menú raíz sin definir otros, de modo que un analista del entorno institucional encuentra el pipeline generativo como una rama más del sistema que ya opera.

El repositorio común del Capítulo 3 es, en esta infraestructura, la base de datos PostgreSQL de la instancia de Odoo, accedida a través del ORM de la plataforma. No se trata de un componente que el *addon* institucional provea, sino de un servicio del framework Odoo del que aquél ya hace uso: los registros de portales, reglas y resultados residen allí, y el módulo de esta tesina agrega sus propios modelos a la misma base y bajo el mismo ORM. La consecuencia arquitectónica —que los *recordsets* de los tubos sean referencias a registros de un único almacén— se desarrolló en la Subsección 3.2.5 y no depende de nada que el *addon* institucional haga; depende de la plataforma.

4.1.2. Realización del filtro de Extracción

La Extracción es el único filtro del pipeline que la presente tesina no construye. El Capítulo 3 la caracterizó por su interfaz —un puerto de entrada conectado al flujo de los portales, un puerto de salida que emite `Noticia[N]`, un sintonizador de portales y reglas, y escritura sobre el repositorio— y por su modo de operación, como filtro activo disparado por las acciones planificadas de la plataforma. Esta subsección describe cómo el *addon* institucional realiza esa interfaz.

Sintonizador. La selección de los portales a monitorear y la definición de las reglas de extracción se realizan desde los formularios de portales, reglas y términos del *addon* institucional, que son por lo tanto la superficie del sintonizador de Extracción. Una regla se satisface cuando la publicación contiene al menos uno de sus términos de disyunción —o no declara ninguno—, contiene todos sus términos de conjunción y no contiene ninguno de sus términos de negación; la comparación es por inclusión de subcadena, sin distinguir mayúsculas, sobre el título, el cuerpo y la descripción de la publicación. La descripción publicada de la herramienta habla de expresiones regulares (Raimondo Anselmino et al., 2024); el motor que el código implementa es de coincidencia por subcadena, y esa diferencia no es menor para el caso de

aplicación, porque una subcadena breve engancha toda palabra que la contenga. La prioridad de monitoreo se asigna por portal, de una escala de diez valores, y determina a qué pasada de extracción pertenece cada portal.

Disparo. La extracción es un proceso continuo y no una operación bajo demanda. Una acción planificada de la plataforma invoca cada ochenta minutos la extracción de los portales de una prioridad determinada; cada pasada toma los portales activos de esa prioridad de a siete, en orden y de manera circular, registrando en la secuencia de monitoreo dónde quedó. Las restantes prioridades se disparan desde *scripts* externos a la plataforma, ejecutados por el planificador del sistema operativo contra la interfaz de procedimientos remotos de Odoo. Dos precisiones respecto de la caracterización del Capítulo 3 se siguen de aquí. La frecuencia no se configura por portal sino por prioridad: es el grupo el que tiene una cadencia, y el portal la hereda al pertenecer a él. Y el filtro es activo en el sentido del estilo —revisa periódicamente su fuente sin que nadie lo invoque—, pero su ciclo es independiente del ciclo de análisis: el pipeline no dispara la extracción, sino que encuentra en el repositorio lo que ésta ya depositó.

Especificación funcional realizada. Para cada portal de la pasada, el filtro privilegia el canal RSS si el portal lo declara: recorre sus entradas, descarga y analiza cada artículo con la biblioteca Newspaper3k y aplica las reglas activas; si no hay canal RSS, construye el índice del sitio con la misma biblioteca y recorre los artículos que encuentra. Una publicación se descarta si ninguna regla la satisface, si su enlace ya existe en el repositorio —el enlace es clave única del modelo de resultados— o si su fecha de publicación tiene más de tres días de antigüedad. De las que superan esos filtros se persisten el título, el cuerpo, el enlace, la fecha de publicación cuando la biblioteca logra extraerla, la descripción a modo de copete cuando se ingresa por la vía del índice del sitio, el portal y su adscripción geográfica, la fecha y hora del registro, y los nombres de las reglas que motivaron su ingreso; a ello se agregan los enriquecimientos de la capa semántica ya descriptos. Las publicaciones se acumulan en memoria durante la pasada y se graban en lote al finalizarla, con una notificación a un canal de mensajería instantánea por cada una.

La ventana de tres días define el alcance del filtro con más fuerza que cualquier otro parámetro: la Extracción vigila el presente de los portales y no permite recuperar lo publicado antes de que una regla estuviera activa. Los

propios autores de la herramienta documentan esa limitación y la resolución que adoptaron para su investigación —*scripts* complementarios de extracción retrospectiva, ajenos a la plataforma— (Raimondo Anselmino et al., 2024). Para el framework de esta tesina la consecuencia es de diseño: el corpus de un estudio no es lo que se extrae *para* el estudio, sino lo que la Extracción depositó en el repositorio mientras las reglas pertinentes estuvieron activas; por eso el estudio incorpora una selección sobre lo ya extraído como segunda superficie del sintonizador de Extracción, según se desarrolla en la Sección 4.3.

4.1.3. El contrato `Noticia` sobre el modelo preexistente

Que el resto del pipeline pueda apoyarse sobre el filtro reutilizado depende de una sola condición: que el modelo de resultados del *addon* institucional satisfaga el contrato `Noticia` del Apéndice A.2. La Tabla 4.2 verifica esa condición clave por clave.

Tabla 4.2: Correspondencia entre las claves del contrato `Noticia` y los campos del modelo preexistente `wsf_noticias_resultados`. Cada clave se realiza en un campo homónimo del modelo, salvo la última. Las claves marcadas con `?` son opcionales en el contrato.

Clave del contrato	**Tipo Odoo**	**Observación**
`titulo`	Char	El scraper elimina las comillas del título.
`copete?`	Char	Se llena sólo por la vía del índice del sitio; vacío por la vía RSS.
`texto`	Text	
`link`	Char	Restricción de unicidad en la base de datos: la clave única que el contrato exige.
`medio`	Many2one	Referencia al contacto (*partner*) del portal; su configuración vive en el modelo de portales.
`departamento`	Char	Copiado del portal en el momento de la extracción; es una etiqueta del portal, no del contenido.
`fecha_hora?`	Datetime	Fecha de publicación según la biblioteca de extracción; puede faltar.
`fecha_registro`	Datetime	La escribe el scraper al grabar.
`reglas_aplicadas`	Char	Único campo cuyo nombre difiere: se llama `regla2`. Guarda los nombres de las reglas satisfechas, separados por coma.

El modelo tiene, además, campos que ningún filtro del pipeline consume:

la polaridad de sentimiento, la nube de términos, el campo de entidades, la valoración manual del operador, las categorías, las coordenadas y los campos derivados de día y hora. Su presencia no contradice el contrato —que fija lo que el pipeline requiere, no lo que el modelo puede contener— y es coherente con el criterio de contrato mínimo del Apéndice A: el pipeline lee las nueve claves de la tabla y nada más. La conclusión es la que la introducción anticipó: el primer tubo del pipeline existe antes de que el módulo de esta tesina se instale. `Noticia[N]` es un *recordset* de `wsf_noticias_resultados`, y el módulo se limita a heredar ese modelo para fijar el título como nombre visible del registro, sin agregar, modificar ni eliminar campo alguno.

4.1.4. Delimitación de lo reutilizado

Lo anterior permite cerrar la delimitación que la responsabilidad [F] exige, en tres niveles. Se *reutiliza*, sin modificación, el filtro de Extracción con su sintonizador y su disparo; el modelo de resultados como realización del tipo `Noticia`; el repositorio común, que es la base de datos de la plataforma; y los grupos de seguridad y el menú raíz del *addon* institucional. Se *reemplaza* la capa semántica inicial por el pipeline de seis filtros que el resto del capítulo describe, dejando intactos los valores que aquélla persiste. Y *no se modifica* el *addon* institucional: el módulo declara la dependencia y hereda un único modelo para un fin cosmético; toda otra interacción ocurre a través del contrato `Noticia`.

Esa delimitación no queda confiada a la prosa: es verificable en cuatro declaraciones del módulo, que la Figura 4.1 reúne y que constituyen la totalidad de su superficie de contacto con el *addon* institucional. El manifiesto declara la dependencia, con lo que la plataforma garantiza el orden de carga sin que el módulo copie ni intervenga el código ajeno. La herencia del modelo de resultados no agrega columnas ni redefine método alguno: fija el campo de presentación para que las referencias desde los tipos del pipeline resulten legibles, y es el fin cosmético al que el párrafo anterior alude. El menú propio cuelga del menú raíz institucional, y los permisos de todos los modelos del módulo se otorgan sobre los grupos que el *addon* preexistente ya define, en lugar de crear un esquema de seguridad paralelo. Que la extensión completa se agote en cuatro renglones —y que ninguno de ellos escriba sobre el *addon* institucional— es lo que permite sostener la afirmación de la responsabilidad [F] sin apelar a la buena fe del lector: el inventario íntegro de archivos del módulo es el de la Figura 4.2, y `noticia.py` es el único que nombra a

`wsfnoticias`.

```
# __manifest__.py --- declara la dependencia, no copia el addon
'depends': ['base', 'wsfnoticias'],

# models/noticia.py --- único toque sobre el addon institucional
class Noticia(models.Model):
    _inherit = 'wsf_noticias_resultados'
    _rec_name = 'titulo'

# views/menus.xml --- el menú propio cuelga del menú institucional
<menuitem id="menu_iag_root" parent="wsfnoticias.wsf_noticias_root"/>

# security/ir.model.access.csv --- reutiliza los grupos existentes
access_iag_estudio_analista,...,wsfnoticias.noticias_analista,1,1,1,1
```

Figura 4.1: Superficie de contacto completa entre el módulo `noticias_iag` y el *addon* institucional `wsfnoticias`: la declaración de dependencia, la herencia del modelo de resultados, el anclaje del menú y la reutilización de los grupos de seguridad. Se abrevia la fila del archivo de permisos: el módulo declara una por cada modelo y grupo, con combinaciones que varían según el modelo —los tipos del flujo se otorgan al analista con lectura y creación, pero sin escritura ni borrado— y que la Sección **??** retoma al verificar la monotonía del repositorio.

4.2. Stack tecnológico

El stack sobre el que se realiza el módulo está determinado en su mayor parte por la infraestructura preexistente; las decisiones propias de esta tesina se concentran en la frontera con los servicios de modelos y en el criterio de dependencias. Esta sección las presenta en cuatro planos: la plataforma, las bibliotecas, los servicios de modelos y el entorno de desarrollo y pruebas.

Plataforma. El módulo se realiza sobre Odoo Community en su versión 16, la misma que corre la instancia institucional, con PostgreSQL como base de datos —versión 15 en el entorno de desarrollo—. La versión de Python es la que provee la imagen oficial de la plataforma, 3.9, lo que fija el subconjunto del lenguaje disponible. Del framework se emplean tres servicios. El ORM, que provee la semántica de *recordsets* sobre la que se apoya el transporte

por referencia de los tubos (Subsección 3.2.5) y el mecanismo de herencia de modelos por el que un *addon* extiende o reemplaza el comportamiento de otro, que es el mecanismo con el que el módulo realiza la sustituibilidad de los filtros (Sección 4.3). El campo de tipo JSON, incorporado por la plataforma en esta versión y persistido como `jsonb` de PostgreSQL, con el que se representan los valores cuya estructura no fija el framework sino el estudio: el vector de cada `Embedding`, el esquema de respuesta y las variables de cada *template*, el contenido de cada `Análisis` y de cada `Resultado`, y la configuración técnica y el informe de cada corrida. La elección tiene una consecuencia que el módulo debe absorber: `jsonb` no conserva el orden de las claves de un objeto, de modo que todo orden significativo —el de las claves del esquema, el de los grupos de un cruce— se representa con listas y no con objetos. Y, por último, las vistas nativas de tabla dinámica y gráfico, con las que se realiza el sumidero sin construir una capa de presentación propia.

Bibliotecas. Las dependencias de Python se reparten según la frontera de la Sección 4.1. Las de la extracción son las que el *addon* institucional importa —*feedparser* para los canales RSS, Newspaper3k para la descarga y el análisis de los artículos, TextBlob para la polaridad de sentimiento, BeautifulSoup como soporte— y el módulo no las usa directamente. La única dependencia propia del módulo es *requests*, el cliente HTTP con que se accede a los servicios de modelos, que la propia plataforma ya incluye. La decisión de no incorporar bibliotecas numéricas ni de búsqueda vectorial es deliberada: el cómputo de similitud coseno entre vectores se realiza en Python puro, con un recorrido lineal sobre los *embeddings* del corpus, lo que resulta suficiente para corpus de cientos a pocos miles de noticias y mantiene el módulo instalable sobre una instancia estándar de Odoo sin dependencias binarias. Un índice vectorial en la base de datos —por ejemplo, la extensión *pgvector*— es una sustitución prevista con la misma interfaz de filtro, no una carencia del diseño.

Servicios de modelos. El módulo accede al modelo generativo y al modelo de *embedding* a través de un único servicio interno con dos operaciones —*embeber* una lista de textos y *completar* un *prompt* con una temperatura, una longitud de salida y, opcionalmente, un esquema de restricción— del que existen dos adaptadores intercambiables: uno *simulado*, determinístico y sin red, que sostiene las pruebas automatizadas, y uno para la interfaz de programación compatible con OpenAI, que cubre con el mismo código al

servicio de OpenAI, a servidores locales como Ollama o vLLM y a intermediarios como Groq u OpenRouter, por sus puntos de acceso de *chat completions* y de *embeddings*. El adaptador implementa la salida estructurada mediante el parámetro de formato de respuesta, con degradación automática al modo menos restrictivo siguiente si el servicio rechaza el pedido, y reintenta ante fallas transitorias de red o del servidor. Esta capa es la realización concreta de la frontera generativa ([C]): los filtros que necesitan un modelo —Enriquecimiento, Composición, Invocación y Verificación— invocan el servicio interno y no conocen qué adaptador lo realiza; cambiar de proveedor es cambiar un parámetro, y agregar un proveedor con otra interfaz de programación es agregar un adaptador sin tocar filtro alguno. Qué modelos concretos se emplearon en el caso de aplicación es configuración de la corrida y se consigna en el Capítulo 6.

Entorno de desarrollo y pruebas. El módulo se desarrolló y probó en un entorno reproducible definido con Docker Compose, que comprende la base de datos, una imagen propia construida sobre la imagen oficial de Odoo 16 con las dependencias de la extracción fijadas a versiones determinadas, y un servicio opcional de Ollama que sirve modelos locales sin costo de API ni salida de datos del entorno. El *addon* institucional se monta desde su propio repositorio, sin copiarlo ni modificarlo, en consonancia con la delimitación de la Sección 4.1. Las pruebas automatizadas se escriben con el marco de pruebas de la plataforma, corren con el adaptador simulado y deshacen sus efectos al terminar, de modo que no dependen de ningún servicio externo ni dejan datos en la base; su cobertura se presenta en la Sección **??**.

4.3. Materialización del pipeline en el módulo

Esta sección desarrolla la correspondencia que la Tabla 4.1 enunció. Comienza por lo que todos los filtros comparten —la interfaz que los define como filtros, el modo en que reciben y emiten *recordsets*, y el mecanismo con que se los sustituye—, sigue con el estudio, que instancia la red de tubos y la recorre, y recién entonces trata cada filtro. Para cada uno se sigue un esqueleto fijo: el contrato que el Capítulo 3 le fijó, el modelo o servicio que lo realiza, la superficie desde la que se lo configura y el rol que puede accionarla, y las responsabilidades que esa realización honra. Cierran la sección las dos superficies de configuración tomadas en conjunto y el sumidero.

La Figura 4.2 presenta la estructura de archivos del módulo anotada contra los elementos de la arquitectura, y funciona como mapa de las subsecciones que siguen: cada filtro, cada tipo del flujo y cada servicio nombrado allí se desarrolla más adelante en el orden del pipeline. La organización obedece a la regla de correspondencia enunciada en la introducción del capítulo, y su asimetría más visible es deliberada: los filtros ocupan un archivo cada uno, mientras que los siete tipos del flujo comparten uno solo. La razón es que el filtro es la unidad sustituible del estilo —un *addon* tercero hereda `iag.filtro.invocacion` y reemplaza su operación sin tocar nada más—, en tanto que los tipos no se sustituyen sino que se declaran, y comparten el mixin y las restricciones de monotonía que la Subsección 4.3.1 describe.

```
noticias_iag/
|-- __manifest__.py               # depende de wsfnoticias
|-- models/
|   |-- filtros/
|   |   |-- base.py               # iag.filtro: la interfaz
|   |   |-- enriquecimiento.py    # filtro  3->4
|   |   |-- composicion.py        # filtro  5->6
|   |   |-- invocacion.py         # filtro  7->8
|   |   |-- decodificacion.py     # filtro  9->10
|   |   |-- verificacion.py       # filtro 11->12
|   |   `-- normalizacion.py      # filtro 13->14
|   |-- tipos.py                  # los siete tipos y su mixin
|   |-- estudio.py                # iag.estudio: pipeline y runner
|   |-- template.py               # iag.template
|   |-- resultado_fila.py         # proyección del sumidero
|   |-- noticia.py                # hereda wsf_noticias_resultados
|   `-- res_config_settings.py    # configuración técnica
|-- services/
|   |-- proveedor.py              # frontera generativa: adaptadores
|   |-- esquema.py                # validación y canonicalización
|   |-- render.py                 # ensamblador único del prompt
|   `-- vectores.py               # similitud coseno
|-- views/                        # superficies del experto
|-- security/ir.model.access.csv  # permisos analista/operador
|-- data/templates.xml            # templates de ejemplo
`-- tests/test_pipeline.py        # 27 pruebas automatizadas
```

Figura 4.2: Estructura de archivos del *addon* `noticias_iag`, anotada con el elemento de la arquitectura que cada archivo realiza; los filtros se identifican por los puertos que conectan en la Figura 3.1. Se omiten los archivos de inicialización de paquete, los recursos estáticos y el entorno de desarrollo.

4.3.1. El patrón común de los filtros

Todos los filtros construidos por el módulo realizan una misma interfaz, definida por un modelo abstracto de Odoo, `iag.filtro`, que expone una única operación: *ejecutar*, que recibe el *recordset* del tubo de entrada y un diccionario con los valores del sintonizador —vacío para los filtros que no exponen ninguno—, persiste los registros que produce y devuelve el *recordset* del tubo de salida. Un tercer argumento, opcional, es un diccionario de *informe* en el que el filtro deja diagnósticos de su ejecución —descartes, similitudes calculadas, reparaciones, fallas del proveedor— y que no es un tubo ni un

sintonizador: no transporta datos hacia el filtro siguiente ni configura el comportamiento del filtro, sino que documenta la corrida para la ficha del estudio. Cada filtro concreto es un modelo abstracto que hereda de esa interfaz y la implementa: `iag.filtro.enriquecimiento`, `iag.filtro.composicion`, y así los seis. Que sean modelos *abstractos* significa, en la plataforma, que no tienen tabla propia —no persisten nada de sí mismos— pero sí participan del registro de modelos, con lo que se los alcanza por nombre y se los extiende por herencia como a cualquier otro modelo.

Esta última propiedad es la que realiza la sustituibilidad de los filtros. El mecanismo de herencia del ORM permite que un *addon* redefina el comportamiento de un modelo existente sin modificar su código fuente: un módulo tercero puede heredar `iag.filtro.invocacion` y reemplazar su operación *ejecutar* por otra que hable con un servicio distinto, y el resto del sistema —que alcanza al filtro por su nombre y sólo conoce su interfaz— no se entera. Es el equivalente, en la plataforma, de reemplazar un filtro por otro que respete sus puertos (Subsección 3.1.2), y se apoya en que ningún filtro importa ni invoca a otro: la única pieza del módulo que conoce el nombre de los seis filtros es la tabla de etapas del estudio que se presenta en la subsección siguiente.

Los *recordsets* que la operación recibe y devuelve son los tubos. La realización es literal respecto de la caracterización de la Subsección 3.2.5: el filtro recibe una colección ordenada de referencias a registros de un modelo dado, materializa su contenido contra la base de datos a medida que lo accede, crea los registros del tipo que produce y devuelve las referencias a esos registros. Nada más viaja entre filtros. Los tipos del flujo, por su parte, son modelos persistentes —uno por contrato del Apéndice A— que comparten un pequeño mixin, `iag.tipo.flujo`, con dos aportes: la fecha de registro que todos los contratos declaran, asignada automáticamente al crear y de sólo lectura, y la prohibición de eliminar, que hace fallar cualquier intento de borrar un registro del flujo. Las claves de procedencia de cada tipo se realizan como campos relacionales obligatorios con política de restricción en la base de datos: no se puede eliminar una noticia referenciada por un *prompt*, ni un *prompt* referenciado por una respuesta, y así hasta el resultado. Sobre estos dos mecanismos, y sobre los permisos que se describen más adelante, descansa la monotonía del repositorio que el Apéndice A.10 afirma; su verificación conjunta se retoma al final de la sección.

Resta un elemento que la arquitectura no nombra porque no es filtro, tubo ni sintonizador: el servicio de acceso a los modelos, `iag.proveedor`,

presentado en la Sección 4.2. Cuatro filtros lo invocan —Enriquecimiento y Composición para embeber, Invocación y Verificación para completar— y lo hacen a través de la interfaz común, que resuelve qué adaptador está configurado. Que un servicio sea compartido por varios filtros no los acopla entre sí: el servicio no tiene estado propio de una corrida, no transporta datos de un filtro a otro y no conoce qué filtro lo llama, del mismo modo en que el ORM es compartido por todos sin que por ello un filtro observe el estado interno de otro. Es infraestructura, en el mismo sentido en que lo es la base de datos, y su frontera es la que la responsabilidad [C] exige mantener aislada.

4.3.2. El estudio como configuración del pipeline

El estudio, modelo `iag.estudio`, es la instancia de la red de tubos que la introducción del capítulo presentó. Sus campos se agrupan en cuatro conjuntos: los tubos, los sintonizadores, la selección del corpus y la documentación de la corrida.

Tubos y máquina de estados. Cada tubo del pipeline es un campo relacional del estudio que conserva el *recordset* emitido por el puerto correspondiente: seis campos de relación múltiple para `Noticia[N]`, `Embedding[N]`, `Prompt[M]`, `RespuestaIAG[M]`, `Análisis[M']` y `Veredicto[M']`, y un campo de referencia única para el `Resultado` del puerto 14; sus etiquetas en la interfaz llevan el número de puerto de la Figura 3.1. Un campo de estado registra hasta qué tubo se llenó, y sus valores son la máquina de estados que recorre toda corrida: *borrador*, *corpus seleccionado*, *enriquecido*, *prompts compuestos*, *invocado*, *decodificado*, *en verificación*, *verificado* y *consolidado*. La red de tubos queda declarada en una única tabla de seis etapas, cada una de las cuales indica el estado previo, el filtro que se ejecuta, el campo del tubo de entrada, el campo del tubo de salida y el estado resultante:

```
ETAPAS = [
    # (estado previo,   filtro,
    #      tubo de entrada,  tubo de salida,   estado resultante)
    ('corpus',          'enriquecimiento',
         'noticia_ids',    'embedding_ids',  'enriquecido'),
    ('enriquecido',     'composicion',
         'embedding_ids',  'prompt_ids',     'compuesto'),
    ('compuesto',       'invocacion',
         'prompt_ids',     'respuesta_ids',  'invocado'),
    ('invocado',        'decodificacion',
```

```
        'respuesta_ids',  'analisis_ids',   'decodificado'),
    ('decodificado',    'verificacion',
        'analisis_ids',   'veredicto_ids',  'verificado'),
    ('verificado',      'normalizacion',
        'veredicto_ids',  'resultado_id',   'consolidado'),
]
```

Esta tabla es la topología del pipeline hecha dato: el orden de los filtros, qué tubo alimenta a cuál y qué tipo circula por cada uno están declarados en un solo lugar, y reordenar, agregar o suprimir una etapa —la recombinación que el cierre composicional del estilo admite (Subsección 3.1.1)— es modificar una tupla. El *runner* del estudio la recorre: determina la etapa cuyo estado previo coincide con el estado actual, resuelve el filtro por su nombre, le entrega el *recordset* del tubo de entrada y el diccionario del sintonizador que le corresponde, mide el tiempo, recoge el informe y escribe el *recordset* emitido en el tubo de salida junto con el nuevo estado. Cada etapa persiste su delta antes de avanzar, de modo que los estados intermedios son puntos de control: una corrida interrumpida se reanuda desde el último tubo lleno. Dos botones del formulario exponen el *runner*: *Ejecutar siguiente filtro*, que avanza una etapa, y *Ejecutar todo*, que encadena etapas hasta el estado consolidado o hasta el estado *en verificación*, en el que la corrida se detiene a esperar al revisor humano cuando el ejecutor de Verificación así lo indica.

Selección del corpus. El tránsito del estado inicial al de corpus seleccionado no ejecuta ningún filtro: llena el primer tubo con una selección sobre lo que la Extracción ya depositó en el repositorio, y es la segunda superficie del sintonizador de Extracción que la Sección 4.1 anticipó. Los criterios de selección son claves de procedencia del contrato `Noticia` y nada más: un rango de fechas —sobre la fecha de registro, que siempre existe, o sobre la de publicación, que puede faltar—, los portales de origen, las reglas de extracción que motivaron el ingreso de la noticia, la exclusión de las publicaciones que un operador archivó y un límite de cantidad. El experto obtiene una vista previa de las noticias que cumplen los criterios, puede quitar o agregar noticias a mano, y confirma; la confirmación congela la tabla como `Noticia[N]`, registra la fecha y guarda por separado las noticias que cumplían los criterios y fueron excluidas a mano, de modo que la curación manual quede documentada y no se confunda con el criterio. Los criterios y la tabla quedan de sólo lectura una vez confirmado el corpus.

Sintonizadores. Los valores de los sintonizadores de Composición, Verificación y Normalización son campos del estudio agrupados en solapas homónimas del formulario. El *runner* los entrega a cada filtro mediante un método que construye, para el filtro que se va a ejecutar, un diccionario con exclusivamente las claves de su sintonizador —la plantilla, los valores de las variables, el texto libre, la estrategia y los parámetros de recuperación y del modelo para Composición; el criterio de validez y el ejecutor para Verificación; la política sobre los observados, la agrupación, la variable del experto, la condición de inclusión y las claves a consolidar para Normalización—; ningún filtro recibe el estudio ni accede a campos de otro sintonizador. Cada solapa pasa a sólo lectura cuando el filtro que la consume ya se ejecutó, lo que realiza en la interfaz la propiedad de que el sintonizador se lee una única vez, al invocarse el filtro (Subsección 3.2.6). El detalle de cada sintonizador se presenta con su filtro.

Documentación de la corrida. Al ejecutarse la primera etapa, el estudio toma una copia de la configuración técnica vigente —adaptador, servicio, modelos, tiempo de espera, modo de salida estructurada, política de no conformidad, límites de longitud— y la conserva con la corrida, sin las credenciales. A ello se suman el tiempo insumido por cada filtro y los informes que éstos dejaron. Los tres se muestran en una solapa de la corrida y alimentan la ficha del estudio que se describe con el sumidero. Es la realización de la mitad de la procedencia que la cadena de tipos no cubre ([E]): con qué configuración, cuándo y con qué incidencias se produjo cada resultado.

Iteración y reinicio. La iteración del experto, que el Capítulo 3 dejó fuera del pipeline como fenómeno del proceso de uso, se realiza con el botón *Nueva iteración*: crea un estudio nuevo en estado inicial, enlazado al de origen, que hereda los criterios del corpus, la plantilla, los valores de las variables y los tres sintonizadores, pero ninguno de los tubos ni la documentación de la corrida, que se marcan como no copiables. El estudio derivado recorre la red desde el principio y produce su propio resultado; los estudios de una misma línea quedan enlazados en ambas direcciones. El botón *Volver a borrador*, por su parte, desvincula los *recordsets* de todos los tubos y reinicia el estado, pero no elimina ningún registro: las noticias, los *embeddings*, los *prompts* y los demás registros permanecen en el repositorio, referenciables desde otros estudios o desde la auditoría, en aplicación estricta de la monotonía.

Superficie y rol. El estudio es el objeto principal con que opera el experto del dominio, miembro del grupo de analistas del *addon* institucional, que tiene sobre él permiso completo; los operadores sólo lo leen. Su formulario reúne, en la cabecera, la barra de estados y los cinco botones —ejecutar siguiente, ejecutar todo, continuar tras la verificación humana, nueva iteración y volver a borrador—; en la caja de accesos, un botón por tubo que abre el *recordset* correspondiente con su cardinalidad, más el resultado y el tablero; y en las solapas, el corpus, los tres sintonizadores, el resultado, la corrida, la ficha y las notas del experto. Es la superficie donde se realizan la intervención del experto ([A]) y la procedencia de la configuración ([E]), y donde el estudio —tema, plantilla, criterios— entra al sistema como dato y no como código ([B]).

4.3.3. Los filtros del pipeline

Las siete subsecciones que siguen recorren los filtros en el orden del pipeline. Cada una abre con una ficha del módulo que realiza el filtro, construida según la documentación que el catálogo de Cristiá and Pomponio (2025, §3.11) prescribe para los componentes de un sistema de Tubos y Filtros. La ficha tiene dos partes. La primera es la interfaz del módulo en la notación 2MIL del propio catálogo, que para este estilo se extiende con las cláusulas de puertos de entrada y de salida —`inports` y `outports`— y documenta los sintonizadores, cuando los hay, como procedimientos de la cláusula `exports`; la semántica de las restantes cláusulas —`comprises`, `imports`, `inherits from`— es la del apunte de diseño de Cristiá (2026), del que la notación proviene; los parámetros de cada procedimiento se marcan con `i`, `o` o `io` según sean de entrada, de salida o de ambas, extendiendo con el tercer marcador la convención del apunte, que sólo emplea los dos primeros. Junto a cada nombre de módulo se consigna, entre paréntesis, el modelo o servicio de la plataforma que lo realiza y, tras dos puntos, las operaciones o parámetros que de él se usan; es el modo en que la ficha une el nombre de diseño, que es el de los contratos del Capítulo 3, con el nombre de código, que es el que el lector encuentra en el módulo.[1] La segunda es la entrada del módulo en la *Guía de*

[1] En la versión electrónica, los nombres de código enlazan al archivo que los define en el repositorio correspondiente. Para el *addon* institucional, `https://github.com/wsf/wsfnoticias`, en la revisión exacta a la que esta tesina tuvo acceso; para el módulo de esta tesina, `https://github.com/lbassani12/noticias_iag`, en el commit que corresponde a la versión aquí descripta.

Módulos —el documento que Parnas et al. (1985) proponen como sustento de la descomposición de un sistema— con sus dos apartados: la *función*, que aquí se escribe como contrato de la operación del filtro en el sentido de Meyer (1992) —lo que exige de su entrada, lo que garantiza sobre el repositorio y sobre su salida, y los errores que emite—, y el *secreto*, la decisión de diseño que el módulo oculta y el cambio previsto que la motiva. Completan la ficha la superficie desde la que se configura el filtro, con el rol que la acciona, y las responsabilidades que la realización honra. A continuación de la ficha, cada subsección desarrolla las decisiones de implementación que la especificación del Capítulo 3 dejaba abiertas.

Extracción

Interfaz del módulo.

```
Module    Extraccion                -- filtro activo; módulo lógico
comprises Scraper (wsf_noticias_medios)         -- addon institucional
          SeleccionCorpus (iag.estudio)         -- módulo propio
inports   portal   : flujo del portal           -- puerto 1
outports  noticias : Noticia[N]                 -- puerto 2
exports   Scraper.scrap_noticias(i importancia)
          Scraper.configurar(i portales, i reglas, i prioridad)
          SeleccionCorpus.seleccionar_corpus(i criterios, i curacion)
                                              : Noticia[N]
imports   Noticia (wsf_noticias_resultados)
          Reglas (wsf_noticias_reglas)
          Terminos (wsf_noticias_terminos)
          Secuencia (wsf_noticias_secuencia)
          Partner (res.partner)
comments  scrap_noticias() es el disparo del filtro activo.
          configurar() es el sintonizador en su superficie
          institucional: los formularios de portales, reglas y
          términos. seleccionar_corpus() es el sintonizador en la
          superficie del estudio: action_buscar_corpus() y
          action_seleccionar_corpus() de iag.estudio. El tubo 2->3
          es Tubo(Noticia): Scraper escribe, SeleccionCorpus lee.
          Ningún submódulo hereda de iag.filtro.
```

- **Función.** *Exige:* del lado de la escritura, al menos un portal activo en la prioridad disparada y al menos una regla activa, pues una publicación que no satisface regla alguna se descarta; del lado de la lectura, un estudio en estado borrador cuyos criterios sean claves del contrato

`Noticia` y nada más: un rango sobre la fecha de registro o sobre la de publicación, los portales de origen, las reglas que motivaron el ingreso, la exclusión de las publicaciones archivadas por un operador y un límite de cantidad. *Garantiza:* cada pasada de extracción deja $Rep' = Rep \cup \Delta$, donde Δ son las publicaciones de los portales de la pasada que satisfacen alguna regla activa, cuyo enlace no existía en el repositorio y cuya fecha de publicación no supera los tres días; cada registro de Δ cumple las nueve claves del contrato (Tabla 4.2); ningún registro anterior se modifica ni se elimina, y la secuencia de monitoreo avanza hasta el último portal procesado. La confirmación del corpus deja en el estudio `Noticia[N]` $\subseteq$ *Rep*: las noticias que cumplen los criterios —las más recientes primero, hasta el límite si éste es positivo— tal como quedaron en la tabla tras la curación manual; las que cumplían los criterios y no quedaron en la tabla se registran como excluidas; se fija la fecha del corpus, el estado pasa a corpus seleccionado y los criterios y la tabla quedan de sólo lectura. Esta segunda operación no crea ni modifica registro alguno del flujo —su Δ es vacío— y escribe únicamente campos del estudio. *Errores:* las publicaciones que no satisfacen regla, duplican enlace o exceden la ventana se omiten sin que el pipeline reciba cuenta de ellas, porque el filtro reutilizado no produce el diccionario de informe que los filtros construidos entregan al *runner* (Subsección 4.3.1); la confirmación del corpus falla con un error de usuario si el estudio no está en borrador o si ninguna noticia cumple los criterios.

- **Secreto.** El Scraper oculta cómo se accede a cada portal —canal RSS o índice del sitio, biblioteca de extracción, análisis de cada artículo—, el motor con que se evalúan las reglas —coincidencia de subcadena, sin distinguir mayúsculas—, la cadencia y el orden circular de las pasadas por prioridad y la ventana de tres días. SeleccionCorpus oculta la traducción de los criterios a una consulta sobre el repositorio —el dominio del ORM y la coincidencia exacta por nombre de regla sobre la clave `reglas_aplicadas`— y el tope de la vista previa. Los cambios previstos que esos secretos absorben son, respectivamente, reemplazar el extractor —otra biblioteca, otro motor de reglas, otra clase de fuente— sin tocar el módulo, mientras produzca registros con las nueve claves del contrato, y agregar criterios de selección sin tocar el *addon* institucional.

- **Superficie y rol.** Dos superficies con ciclos de vida distintos, según la partición que la Subsección 3.2.6 estableció: los formularios de portales, reglas y términos del *addon* institucional, que gobiernan la extracción continua y son configuración del entorno, compartida por todos los estudios y vigente antes de que cualquiera exista; y la solapa *Corpus* del estudio —vista previa, curación manual y confirmación—, que gobierna qué parte de lo extraído entra a una corrida y queda fijada con él. Ambas para el grupo de analistas; los operadores sólo leen. Las dos parametrizan la misma transformación sin inyectar datos en el pipeline, y la segunda permite lo que la extracción continua no ofrece: definir un corpus por su procedencia y curarlo dejando registradas las exclusiones, con lo que un mismo repositorio sirve a estudios distintos sin que ninguno modifique lo extraído.

- **Responsabilidades**: [A] —portales, reglas y selección del corpus son decisiones del experto—; [F] —el filtro se reutiliza del entorno institucional sin modificación—.

Decisión de implementación. La ficha presenta a la Extracción como módulo lógico: agrupa dos módulos físicos que viven en *addons* distintos y no tiene implementación propia. Esa ausencia es deliberada. El módulo no construyó un extractor ni un envoltorio sobre el institucional: envolver la Extracción en un modelo `iag.filtro.extraccion` habría dado uniformidad a la tabla de etapas, pero a costa de introducir una dependencia hacia el interior del *addon* institucional —sus métodos de extracción, su secuencia de monitoreo— que la delimitación de la Sección 4.1.4 excluye. De los módulos que la cláusula `imports` enumera, SeleccionCorpus usa sólo tres, y para leer: el contrato `Noticia`, las reglas y los contactos de los portales —que el estudio consulta para ofrecer los criterios— y, dentro del primero, la valoración manual del operador, que se consulta únicamente para excluir las publicaciones archivadas. Es la mínima superficie de contacto compatible con reutilizar el filtro, y es lo que permite afirmar que un extractor distinto que produjera registros con las nueve claves del contrato sustituiría al actual sin tocar el módulo. Por la misma razón el puerto 2 se realiza en dos tiempos —la escritura continua del Scraper y la lectura selectiva de SeleccionCorpus— sin que ninguno altere al otro: el scraper no sabe qué estudios existen, y el estudio no dispara ni configura la extracción, sino que lee por las claves de procedencia que el contrato garantiza.

Enriquecimiento semántico

Interfaz del módulo.

```
Module    Enriquecimiento (iag.filtro.enriquecimiento)
          inherits from iag.filtro                    -- filtro pasivo
inports   entrada : Noticia[N]                        -- puerto 3
outports  salida  : Embedding[N]                      -- puerto 4
exports   ejecutar(i entrada, i sintonizador, io informe)
                                                : Embedding[N]
                              -- redefine la operación de iag.filtro;
                              -- sin sintonizador (diccionario vacío)
imports   Noticia (wsf_noticias_resultados)
          Embedding (iag.embedding)
          Proveedor (iag.proveedor: embeber)
          ConfiguracionTecnica (ir.config_parameter: modelo_embedding,
                                max_chars_embedding)
comments  No importa ni invoca a otro filtro. No escribe en informe:
          no tiene descartes ni reparaciones que reportar. El modelo
          de embedding no viaja en Embedding: queda en la copia de la
          configuración técnica que el estudio toma al iniciar la
          corrida.
```

- **Función.** *Exige:* un *recordset* `Noticia[N]` —posiblemente vacío— y un proveedor disponible según la configuración técnica; el diccionario del sintonizador se ignora, porque el filtro no expone ninguno. *Garantiza:* si la entrada está vacía, devuelve el *recordset* vacío sin invocar al proveedor. En otro caso, para cada noticia de la entrada y en su mismo orden, compone el texto a codificar —título, copete y cuerpo, separados por una línea en blanco y truncados al largo máximo que fija la configuración técnica—, obtiene del proveedor su vector, en lotes de treinta y dos textos, y persiste un registro `Embedding` que referencia a la noticia y guarda el vector. Δ son exactamente los N registros creados y el *recordset* emitido es *refs*(Δ): la cardinalidad se preserva y el tipo cambia de `Noticia` a `Embedding`. La creación ocurre en una única operación al final, de modo que Δ es completo o vacío y ninguna falla deja un corpus codificado a medias. Ningún registro se modifica ni se elimina; la fecha de registro la asigna el mixin del flujo, y la referencia a la noticia es obligatoria y con política de restricción, con lo que la noticia codificada no puede eliminarse mientras exista su vector. El registro no guarda qué modelo lo produjo, conforme al contrato del Apéndice A.3;

esa identidad queda en la copia de la configuración técnica que el estudio toma al iniciar la corrida. *Errores:* falla con un error de usuario, sin persistir nada, si el proveedor configurado no está disponible, si el servicio de *embeddings* responde con error, o si la cantidad de vectores recibidos no coincide con la cantidad de textos enviados —condición que se verifica dos veces: en el adaptador, lote por lote, y en el filtro, sobre el total—.

- **Secreto.** Qué texto representa a una noticia —la concatenación de título, copete y cuerpo, y el truncamiento al largo máximo— y cómo se dialoga con el servicio de *embeddings* —el tamaño del lote y la verificación de cardinalidad—. El protocolo del servicio, por debajo, es secreto del adaptador y no del filtro (Subsección 4.3.1). Los cambios previstos que el secreto absorbe son alterar la composición del texto —codificar sólo el título, por ejemplo— o reemplazar el truncamiento por otra estrategia de fragmentación, sin que Composición ni ningún otro filtro se enteren, mientras se emita un `Embedding` por noticia. Cambiar de modelo de *embedding*, en cambio, no es un cambio interno sino la sustitución del filtro completo, porque los vectores de modelos distintos no son comparables (Apéndice A.3); por eso el modelo es configuración técnica y no sintonizador.

- **Superficie y rol.** Ninguna para el experto: el filtro no expone sintonizador. Su configuración —el modelo de *embedding* y el largo máximo del texto codificado— es técnica, se edita en los ajustes de administración de la plataforma y es potestad del implementador, con el grupo de administración del sistema; el estudio la copia, sin credenciales, al iniciar la corrida. Los analistas leen y crean registros `Embedding` —lo segundo, por intermedio del filtro— pero no los modifican ni los eliminan.

- **Responsabilidades**: [C] —el modelo de *embedding* es una dependencia externa sustituible cuya identidad no viaja en el contrato—.

Decisión de implementación. La estrategia de fragmentación que el Capítulo 3 dejó como configuración técnica se realiza, en esta versión, como un truncamiento: un único vector por noticia sobre los primeros caracteres de su texto, con el largo máximo como único parámetro. La alternativa —fragmentar cada noticia en varios segmentos y codificar cada uno— produciría más de un vector por noticia y obligaría a Composición a agregar

segmentos antes de comparar; como la recuperación de ese filtro ordena noticias enteras por su similitud con la consulta (Subsección 4.3.3), un vector por noticia mantiene `Embedding[N]` alineado con `Noticia[N]` y el recorrido de similitud lineal y simple. El largo máximo existe como salvaguarda frente a textos anómalos y frente al límite de contexto del modelo de *embedding*; su valor es configuración técnica y la ficha del estudio lo registra. El tamaño del lote, en cambio, es una constante del filtro y no configuración, porque no altera el resultado sino sólo el costo de obtenerlo. Por último, el filtro no reutiliza vectores calculados por corridas anteriores sobre la misma noticia: como el registro no guarda el modelo que lo produjo, reutilizarlo exigiría suponer que la configuración técnica no cambió entre corridas. Recalcular cuesta una llamada al servicio por lote y garantiza que todo `Embedding[N]` de un estudio proviene de un mismo modelo, el que su ficha documenta.

Composición de contexto

Interfaz del módulo.

```
Module    Composicion (iag.filtro.composicion)
          inherits from iag.filtro                    -- filtro pasivo
inports   entrada : Embedding[N]                      -- puerto 5
outports  salida  : Prompt[M]                         -- puerto 6
exports   ejecutar(i entrada, i sintonizador, io informe) : Prompt[M]
          sintonizar(i template, i variables, i texto_libre,
                     i estrategia, i top_k, i umbral, i temperatura,
                     i longitud_salida)       -- solapa Composición
imports   Embedding (iag.embedding), Noticia (wsf_noticias_resultados)
          Prompt (iag.prompt), Template (iag.template)
          Proveedor (iag.proveedor: embeber)
          Render (services/render.py: prompt)
          Vectores (services/vectores.py: coseno)
          ConfiguracionTecnica (ir.config_parameter:
                                max_chars_noticia)
comments  La primera superficie del sintonizador es el panel de
          templates (iag.template: objetivo, instrucciones, variables,
          esquema de respuesta); la segunda, la solapa Composición del
          estudio. En informe deja las similitudes de las noticias
          seleccionadas y la cantidad descartada por umbral.
```

- **Función.** *Exige:* un *template* en el sintonizador —es lo primero que se comprueba, antes que la entrada—; un *recordset* `Embedding[N]`, posiblemente vacío; un valor para cada variable de contexto que las instruc-

ciones del *template* usan, tomado del estudio o, en su defecto, del valor por defecto que el *template* declara; y un proveedor disponible, porque la consulta de recuperación se codifica con el mismo servicio que codificó las noticias. *Garantiza:* si la entrada está vacía, devuelve el *recordset* vacío. En otro caso sustituye las variables en las instrucciones; compone la consulta de recuperación con el objetivo del *template*, los valores de las variables y el texto libre; obtiene su vector; calcula la similitud coseno entre ese vector y el de cada `Embedding` recibido; conserva las noticias cuya similitud alcanza el umbral —un umbral no positivo significa sin umbral, porque la similitud coseno vive en $[-1, 1]$—, ordenadas de mayor a menor, y las recorta a las k primeras si el sintonizador fija un k positivo. Con la estrategia *por noticia* forma un grupo por noticia seleccionada; con la estrategia *agregado*, un único grupo con todas. Por cada grupo persiste un `Prompt` cuyo texto es el ensamblado completo —objetivo, instrucciones, aclaraciones, material empírico con una entrada por noticia (portal, fecha, título, copete y cuerpo truncado al largo máximo de la configuración técnica) e instrucción de formato con el esquema de respuesta—, con la temperatura y la longitud de salida del sintonizador, la referencia al *template* y las referencias a las noticias del grupo. Δ son los M registros creados en una única operación al final; $M = 1$ con la estrategia agregada y $M = k$ con la estrategia por noticia, donde $k \leq N$ es la cantidad de noticias seleccionadas; con umbral y k en sus valores por defecto, $M = N$. Deja en el informe de la corrida la similitud de cada noticia seleccionada y la cantidad de noticias descartadas por el umbral. Ningún registro se modifica ni se elimina. *Errores:* falla con un error de usuario, sin persistir nada, si no hay *template*, si alguna variable usada por las instrucciones queda sin valor, si el proveedor no puede codificar la consulta o si ninguna noticia alcanza el umbral.

- **Secreto.** Cómo se construye la consulta de recuperación y cómo se mide la relevancia —similitud coseno calculada en Python puro, con un recorrido lineal sobre los vectores del corpus—; el orden y el formato de las secciones del *prompt*, que fija un único ensamblador compartido con la previsualización del panel de *templates*; y el truncamiento de cada noticia en el material empírico. Los cambios previstos que el secreto absorbe son reemplazar la medida o incorporar un índice vectorial cuando el corpus lo exija, y cambiar el ensamblado del *prompt*;

ninguno altera lo que circula, porque el texto del `Prompt` materializa el ensamblado completo y los filtros siguientes no conocen su estructura interna (Apéndice A.4).

- **Superficie y rol.** El sintonizador tiene las dos superficies que la Subsección 3.2.6 estableció: el panel de *templates*, donde el experto define objetivo, instrucciones, variables y esquema de respuesta, con permiso completo para el grupo de analistas y lectura para los operadores; y la solapa *Composición* del estudio, donde elige el *template*, da valor a las variables, agrega aclaraciones y fija la estrategia, los parámetros de recuperación y los del modelo. La solapa pasa a sólo lectura cuando el filtro se ejecutó. La configuración técnica aporta un único parámetro, el largo máximo de cada noticia en el material empírico, potestad del implementador. Los analistas leen y crean registros `Prompt` —lo segundo, por intermedio del filtro—, sin modificarlos ni eliminarlos.
- **Responsabilidades**: [A] —el núcleo de la intervención del experto: qué se pregunta, sobre qué material y con qué parámetros—; [B] —el estudio ingresa como dato por el *template* y las variables, y el filtro no contiene ninguna decisión de dominio—.

Decisión de implementación. Tres decisiones fijan lo que la especificación dejaba abierto. La primera es que la consulta de recuperación se codifica con el mismo proveedor y el mismo modelo que codificaron las noticias, sin lo cual la similitud no tendría sentido (Apéndice A.3); la coherencia dentro de una corrida descansa en que la configuración técnica no cambie entre Enriquecimiento y Composición, y la copia que el estudio toma al iniciar la corrida la documenta. La segunda es el ensamblador único: el texto que el panel de *templates* previsualiza y el que Composición envía provienen de la misma función, de modo que lo que el experto ve al definir el estudio es, sección por sección, lo que recibirá el modelo, con el único hueco del material empírico, que pertenece al corpus y no al *template*. La tercera es que la recuperación no usa índice vectorial ni biblioteca numérica, por el criterio de dependencias de la Sección 4.2: para corpus de cientos a pocos miles de noticias el recorrido lineal es suficiente y mantiene al módulo sin dependencias propias. Las similitudes que el filtro deja en el informe no son un dato del flujo —ningún tipo las porta— sino documentación de la corrida: permiten reconstruir, en la ficha del estudio, por qué cada noticia entró o no al material

empírico ([E]).

Invocación generativa

Interfaz del módulo.

```
Module    Invocacion (iag.filtro.invocacion)
          inherits from iag.filtro                    -- filtro pasivo
inports   entrada : Prompt[M]                         -- puerto 7
outports  salida  : RespuestaIAG[M]                   -- puerto 8
exports   ejecutar(i entrada, i sintonizador, io informe)
                                                : RespuestaIAG[M]
                                -- sin sintonizador (diccionario vacío)
imports   Prompt (iag.prompt)
          Template (iag.template: esquema_respuesta)
          RespuestaIAG (iag.respuesta)
          Proveedor (iag.proveedor: completar)
comments  La configuración técnica (proveedor, base_url, api_key,
          modelo_generativo, timeout, salida_estructurada) la lee el
          adaptador que iag.proveedor resuelve, no el filtro. En
          informe deja, en fallidas, los prompts que no obtuvieron
          respuesta pese a los reintentos del adaptador.
```

- **Función.** *Exige:* un *recordset* `Prompt[M]`, posiblemente vacío, y un proveedor disponible según la configuración técnica; el sintonizador se ignora. *Garantiza:* si la entrada está vacía, devuelve el *recordset* vacío. En otro caso, para cada *prompt* y en su orden, pide al proveedor que complete su texto con la temperatura y la longitud de salida que el propio `Prompt` porta, entregándole además, como restricción de generación, el esquema de respuesta del *template* que el *prompt* referencia, y persiste una `RespuestaIAG` con el texto crudo tal como el servicio lo devolvió, sin interpretarlo ni validarlo. Si el servicio falla para un *prompt* pese a los reintentos del adaptador, la respuesta se persiste vacía y el motivo queda en el informe de la corrida con la referencia al *prompt*. La cardinalidad se preserva exactamente, con fallas o sin ellas: Δ son M registros, uno por *prompt*, creados en una única operación al final. Ningún registro se modifica ni se elimina, y el registro no guarda proveedor, modelo ni credenciales (Apéndice A.5). *Errores:* falla con un error de usuario, sin persistir nada, únicamente si el proveedor configurado no está disponible; la falla del servicio para un *prompt* no es un error del

filtro sino un dato del informe, y la compuerta de Decodificación es la que descarta y cuenta la respuesta vacía.

- **Secreto.** El filtro oculta poco, y es deliberado: el recorrido del *recordset* y la política ante una llamada fallida. Todo lo demás es secreto del adaptador que la configuración técnica selecciona —el protocolo del servicio, el formato de la petición, los reintentos ante fallas transitorias (tres intentos, con espera creciente, para errores de red, de tiempo de espera y del servidor; los errores del cliente no se reintentan), el modo de salida estructurada y su degradación— y el filtro lo alcanza por la interfaz común `iag.proveedor`, que resuelve el adaptador por nombre. El cambio previsto que absorbe esa partición es reemplazar el proveedor o el modelo sin tocar el filtro, y agregar un servicio nuevo escribiendo un adaptador que herede la interfaz, sin tocar nada más ([C]).
- **Superficie y rol.** Ninguna para el experto. La configuración técnica —adaptador, servicio, modelo, credenciales, tiempo de espera y modo de salida estructurada— se edita en los ajustes de administración de la plataforma y es potestad del implementador; el estudio la copia al iniciar la corrida, sin las credenciales. Los analistas leen y crean registros `RespuestaIAG` —lo segundo, por intermedio del filtro— sin modificarlos ni eliminarlos.
- **Responsabilidades**: [C] —aísla la dependencia del servicio generativo externo—.

Decisión de implementación. Dos decisiones del implementador condicionan la corrida y no son visibles para el experto. La primera es la política ante una llamada fallida: persistir una respuesta vacía en lugar de abortar la etapa. La alternativa —fallar y no persistir nada— perdería la etapa entera por una llamada, con M en las decenas o centenas; la adoptada preserva la cardinalidad y la procedencia, y delega el descarte a la compuerta de Decodificación, que ya existe para eso. La segunda es el *modo de salida estructurada*. Cuando el servicio admite restringir la generación a un esquema —capacidad que los proveedores exponen con mecanismos heterogéneos: gramáticas derivadas del esquema, modos de salida JSON, esquemas estrictos (Geng et al., 2025)—, el adaptador le entrega como restricción el esquema de respuesta del *template*, alcanzado por la referencia que cada `Prompt` porta: es el mismo acceso por referencia con que Decodificación valida aguas abajo, y no supone

configuración de dominio propia. Dos precisiones preservan la frontera del filtro. La restricción opera durante la generación y del lado del proveedor: Invocación no interpreta ni valida la respuesta. Y el modo eleva la tasa de conformidad pero no la garantiza: si el proveedor rechaza el modo configurado, el adaptador degrada al modo menos restrictivo y luego a texto libre, dejando constancia, y la conformidad de lo que circula sigue garantizada exclusivamente por la compuerta del filtro siguiente. El esquema llega así al modelo por dos vías complementarias —como instrucción dentro del texto del *prompt*, siempre; como restricción de generación, cuando el proveedor lo permite—: una decisión del implementador sobre la frontera generativa ([C]) acerca de un dato del estudio ([B]) que el framework transporta sin conocer. El peso de la decisión se midió sobre un mismo corpus de trece noticias con un modelo local de tres mil millones de parámetros: sin restricción, la compuerta persistió nueve de los trece análisis; con la restricción al esquema, los trece, y la etapa insumió 347 segundos en lugar de 570, porque la gramática elimina decisiones de muestreo. La medición corresponde a un proveedor y un modelo; en otros la capacidad existe con mecanismos y límites distintos (Geng et al., 2025).

Decodificación

Interfaz del módulo.

```
Module    Decodificacion (iag.filtro.decodificacion)
          inherits from iag.filtro                    -- filtro pasivo
inports   entrada : RespuestaIAG[M]                   -- puerto 9
outports  salida  : Análisis[M']                      -- puerto 10
exports   ejecutar(i entrada, i sintonizador, io informe)
                                                : Análisis[M']
                            -- sin sintonizador (diccionario vacío)
imports   RespuestaIAG (iag.respuesta), Análisis (iag.analisis)
          Template (iag.template: esquema_respuesta)
          Esquema (services/esquema.py: aislar_json, reparar_json,
                   canonicalizar)
          ConfiguracionTecnica (ir.config_parameter:
                                politica_no_conformidad)
comments  M' <= M: la compuerta no persiste lo que no conforma. El
          esquema se alcanza por la cadena respuesta -> prompt ->
          template, sin configuración de dominio. En informe deja la
          política aplicada, los descartes con su motivo y las
          reparaciones realizadas.
```

- **Función.** *Exige:* un *recordset* `RespuestaIAG[M]`, posiblemente vacío, y una política ante la no conformidad en la configuración técnica, *reparar* o *descartar*; el sintonizador se ignora. Que cada respuesta alcance un esquema lo garantiza el repositorio: las referencias de la respuesta al *prompt* y del *prompt* al *template* son obligatorias, y el *template* no se guarda sin esquema. *Garantiza:* para cada respuesta aísla el primer objeto JSON de su texto, quitando los delimitadores de código y la prosa circundante; si no lo logra y la política es reparar, intenta cerrar una estructura interrumpida al final —agrega únicamente los cierres faltantes; un valor interrumpido no se completa sino que se descarta como miembro incompleto, hasta tres veces—; luego canonicaliza el objeto contra el esquema del *template*: convierte cadenas numéricas en números y números en cadenas donde el esquema lo pide, reconoce afirmaciones y negaciones escritas en palabras como booleanos, hace coincidir los valores de un vocabulario sin distinguir mayúsculas, y comprueba claves requeridas, rangos y cantidades de elementos. Lo que conforma se persiste como `Análisis` con el contenido canónico; lo que no conforma se descarta y se cuenta con su motivo —los primeros cinco errores, y la reparación intentada si la hubo—; cada reparación que terminó en un análisis persistido queda registrada. Δ son los M' análisis creados en una única operación al final, con $M' \leq M$; una respuesta vacía —la que Invocación persistió ante una falla del servicio— se descarta y se cuenta como las demás. La propiedad que el filtro garantiza hacia abajo es que *todo* `Análisis` del repositorio conforma el esquema de su *template*. Ningún registro se modifica ni se elimina. *Errores:* el filtro no lanza ninguno: la no conformidad no es un error sino un descarte contado en el informe, y la etapa no se pierde por una respuesta.

- **Secreto.** Cómo se reconoce el fragmento estructurado dentro de la prosa, qué puede y qué no puede hacer la reparación, qué conversiones admite la canonicalización —que son la única interpretación que el filtro hace del texto— y el subconjunto de JSON Schema que valida: tipos, propiedades, claves requeridas, vocabularios, elementos, rangos y cantidades. Ese subconjunto es el mismo que el panel de *templates* verifica cuando el experto declara el esquema, de modo que ningún esquema declarado resulta indecodificable. Los cambios previstos que el secreto absorbe son otro formato de salida del proveedor —la decodificación de otro lenguaje de marcado no toca Invocación ni los filtros

siguientes— y otra política ante la no conformidad; extender el subconjunto de esquemas toca el validador y la verificación del panel a la vez, porque son la misma pieza ([B], [C]).

- **Superficie y rol.** Ninguna para el experto. La configuración técnica aporta la política ante la no conformidad, potestad del implementador. El experto ve el efecto en la ficha del estudio: cuántas respuestas se descartaron, por qué, y cuántos análisis provienen de una respuesta reparada. Los analistas leen y crean registros `Análisis` —lo segundo, por intermedio del filtro— sin modificarlos ni eliminarlos.

- **Responsabilidades**: [B] —valida contra el esquema del estudio sin conocerlo: lo alcanza por procedencia—; [C] —absorbe el formato de salida del proveedor y mantiene la etapa agnóstica—.

Decisión de implementación. De las tres políticas ante la no conformidad que el Capítulo 3 admite —reintentar la invocación, reparar, descartar— la implementación realiza dos. Reintentar la invocación desde este filtro queda excluido por el patrón de la Subsección 4.3.1: supondría que un filtro invoque a otro, y la reejecución es un fenómeno del proceso de uso —una nueva iteración del estudio— y no un tubo de retorno. La reparación es la única política que altera el texto recibido, y por eso se delimitó con precisión: se restringe a completar el cierre de una estructura interrumpida al final, sin agregar ni modificar contenido; el resultado reparado atraviesa la misma validación que cualquier otra respuesta; y cada reparación queda en la documentación de la corrida, de modo que el analista sepa qué análisis provienen de una respuesta reparada. La política modifica la proporción de respuestas que superan la compuerta, no el criterio con que se las juzga. Su motivación es empírica: en las corridas de desarrollo, con un modelo local, una parte de los descartes correspondía a respuestas interrumpidas antes de la llave de cierre con un motivo de terminación normal, no por límite de longitud; reparar el cierre recupera esos análisis sin inventar contenido, y la restricción de salida que Invocación aplica cuando el proveedor lo permite reduce la necesidad de reparar. La política por defecto es reparar. Por último, la división de trabajo con Invocación es deliberada: aquel filtro persiste vacía la respuesta que no obtuvo, y es éste el que la descarta y la cuenta, de modo que el informe de la corrida reúne en un solo lugar todo lo que no llegó a ser un análisis y por qué.

Verificación

Interfaz del módulo.

```
Module    Verificacion                 -- módulo lógico
comprises JuezGenerativo (iag.filtro.verificacion)  -- llm_judge
          RevisorHumano (iag.veredicto)             -- humano
inports   entrada : Análisis[M']                      -- puerto 11
outports  salida  : Veredicto[M']                     -- puerto 12
exports   JuezGenerativo.ejecutar(i entrada, i sintonizador,
                                   io informe) : Veredicto[M']
          RevisorHumano.emitir(i analisis, i estado, i confianza,
                               i sustento) : Veredicto
          sintonizar(i criterio_validez, i ejecutor)
imports   Análisis (iag.analisis), Noticia (wsf_noticias_resultados)
          Veredicto (iag.veredicto), Sustento (iag.veredicto.sustento)
          Proveedor (iag.proveedor: completar)
          Esquema (services/esquema.py: aislar_json, canonicalizar)
          ConfiguracionTecnica (ir.config_parameter:
                                max_chars_noticia)
comments  sintonizar() es la solapa Verificación del estudio. Con
          ejecutor humano, JuezGenerativo no crea registros y el
          estudio queda en verificación; emitir() es el formulario de
          veredicto (action_emitir) y el tubo 12->13 lo llena
          action_continuar_verificacion() de iag.estudio con el último
          veredicto de cada análisis. En informe: juicios no conformes
          y sin respuesta del proveedor.
```

- **Función.** *Exige:* un *recordset* `Análisis[M']`, posiblemente vacío; un sintonizador con el criterio de validez —texto libre, con un valor por defecto que exige anclaje literal de cada afirmación en la evidencia— y el ejecutor, *juez generativo* o *revisor humano*; con el juez, un proveedor disponible. *Garantiza, con el juez:* para cada análisis compone un *prompt* de verificación con el criterio, el contenido del análisis y la evidencia —las noticias que su *prompt* incorporó, alcanzadas por procedencia, con portal, fecha, título, enlace y cuerpo truncado— y las instrucciones para dictaminar; lo envía al proveedor con temperatura cero y, como restricción de generación, el esquema fijo del juicio: estado en el vocabulario del contrato, confianza en $[0, 1]$ y sustento como lista de afirmaciones, cada una con su relación con la evidencia —respalda, contradice o sin evidencia—, el fragmento literal y el enlace. Aísla y canonicaliza la respuesta contra ese esquema; si conforma, persiste el

`Veredicto`; si no conforma o el servicio no respondió pese a los reintentos, persiste igualmente un `Veredicto` con estado *observado*, confianza cero y un sustento que declara el motivo y conserva el inicio de la respuesta cruda. Ningún análisis queda sin veredicto: Δ son exactamente M' registros, creados en una única operación al final, y la cardinalidad se preserva. Deja en el informe cuántos juicios no conformaron el esquema y cuántos no obtuvieron respuesta. *Garantiza, con el revisor humano:* el filtro no crea registros y devuelve el *recordset* vacío; el *runner* deja el estudio en el estado *en verificación*; el revisor emite un `Veredicto` por análisis desde el formulario, que le propone como filas de sustento las afirmaciones que el propio análisis declara, con el fragmento citado y el enlace, y deja la relación en blanco para que la decida fila por fila; la continuación del estudio exige un veredicto por análisis, toma el último emitido para cada uno y llena el tubo. En ambos modos ningún análisis se suprime: la compuerta es blanda. Ningún registro se modifica ni se elimina; el sustento de un veredicto tampoco. *Errores:* con el juez, falla con un error de usuario, sin persistir nada, sólo si el proveedor no está disponible; la falla del servicio para un análisis se absorbe como veredicto observado. Con el revisor, la continuación falla si el estudio no está en verificación o si falta algún veredicto, y el formulario rechaza una confianza fuera de $[0, 1]$.

- **Secreto.** El *prompt* del juez —su redacción, el orden en que presenta la evidencia, las reglas que llevan de las relaciones por afirmación al estado del veredicto—, el esquema fijo del juicio, la política de degradación ante un juicio no conforme y la forma en que se sugiere el sustento al revisor humano. El contrato `Veredicto` deja el esquema del sustento a criterio del estudio (Apéndice A.7); esta realización fija una primera versión —anclaje literal, afirmación por afirmación— válida para el criterio de validez por defecto. Los cambios previstos que el secreto absorbe son los que el Capítulo 3 anticipó: otro ejecutor —otro modelo como juez, un conjunto de agentes que confronten sus juicios, otro procedimiento asistido— y otro esquema de sustento para otro criterio, sin alterar los puertos ni el resto del pipeline ([D]).

- **Superficie y rol.** El sintonizador vive en la solapa *Verificación* del estudio —criterio de validez y ejecutor—, para el grupo de analistas, y pasa a sólo lectura cuando el filtro se ejecutó. El revisor humano

opera dos superficies más: el formulario de veredicto, al que llega desde cada análisis pendiente y que encadena un análisis con el siguiente, y el botón del estudio que continúa la corrida tras la verificación. Los analistas leen y crean veredictos y su sustento —el juez por intermedio del filtro, el revisor por el formulario— sin modificarlos ni eliminarlos; los operadores los leen. La configuración técnica no aporta nada propio: el juez usa el proveedor y el modelo de Invocación y el largo máximo de noticia de Composición.

- **Responsabilidades**: [A] —criterio y ejecutor los fija el experto, y el revisor humano es él mismo—; [B] —el criterio es dato del estudio—; [D] —compuerta blanda: juzga, cuantifica y señala sin suprimir—.

Decisión de implementación. La ficha presenta a la Verificación como módulo lógico porque su realización se reparte entre dos módulos físicos que el sintonizador selecciona: el filtro propiamente dicho, que realiza al juez generativo, y el formulario de veredicto, que realiza al revisor humano. Con el revisor, el filtro no crea nada y la corrida se detiene en un estado de espera que es el único de la máquina de estados del estudio fuera de la secuencia lineal; no es un ciclo —no hay tubo de retorno— sino una pausa hasta que el tubo se llena por otra vía, y la continuación toma el último veredicto de cada análisis, lo que permite al revisor corregirse emitiendo de nuevo sin modificar ni eliminar nada. La segunda decisión es la degradación del juez: ante un juicio no conforme o un servicio que no responde, el filtro persiste un veredicto observado con confianza cero y el motivo, en lugar de fallar o de omitir el análisis. La compuerta blanda exige que todo análisis llegue a Normalización con su juicio; un veredicto degradado es distinguible de uno real —su sustento lo declara y el informe lo cuenta— y la política sobre los observados del filtro siguiente decide qué hacer con él. La tercera es que el juez de esta primera versión es el mismo servicio y el mismo modelo que produjeron el análisis, con temperatura cero y el esquema del juicio como restricción. Es una limitación conocida —un modelo que juzga su propia salida— que el diseño no oculta sino que deja en manos del sintonizador: cambiar el ejecutor es una decisión del experto, y la sustituibilidad del filtro permite realizar los ejecutores que el Capítulo 3 anticipó sin tocar el resto del pipeline.

Normalización

Interfaz del módulo.

```
Module    Normalizacion (iag.filtro.normalizacion)
          inherits from iag.filtro                    -- filtro pasivo
inports   entrada : Veredicto[M']                     -- puerto 13
outports  salida  : Resultado                         -- puerto 14
exports   ejecutar(i entrada, i sintonizador, io informe) : Resultado
          sintonizar(i politica_observados, i agrupar_por, i variable,
                     i condicion, i claves)   -- solapa Normalización
imports   Veredicto (iag.veredicto), Análisis (iag.analisis)
          Prompt (iag.prompt)
          Template (iag.template: esquema_respuesta)
          Noticia (wsf_noticias_resultados: medio,
                   departamento, fechas)
          Resultado (iag.resultado)
comments  Determinístico: no usa el proveedor ni la configuración
          técnica. No escribe en informe: los criterios aplicados y
          los conteos quedan materializados en Resultado.contenido.
          El sumidero (iag.resultado.fila, vista SQL de sólo lectura)
          es una proyección del Resultado y no parte del filtro.
```

- **Función.** *Exige:* un *recordset* `Veredicto[M']`, posiblemente vacío, y el sintonizador: la política sobre los observados —incluir todos, excluir los no sustentados o consolidar sólo los válidos—; la agrupación —ninguna, portal, adscripción geográfica, día, semana o la variable del experto, con su nombre, el dominio sobre el que se asigna y la tabla de asignación—; una condición de inclusión opcional, una clave del esquema y el valor que debe tomar; y las claves del esquema a consolidar, o ninguna, en cuyo caso se consolidan todas las comparables: vocabularios, cadenas, booleanos, números y listas de cadenas. *Garantiza:* persiste exactamente un `Resultado` que referencia los M' veredictos recibidos, todos, consolidados o no, y cuyo contenido materializa los criterios aplicados y el recorrido. Para cada veredicto determina el grupo de su análisis —por los metadatos de la única noticia de su *prompt*, alcanzados por procedencia, o según la tabla del experto, con un grupo *sin asignar* para lo que la tabla no cubre; un análisis agregado sobre varias noticias forma grupo propio— y si se consolida: queda fuera, con su motivo, el análisis cuyo veredicto la política excluye o que no cumple la condición. Sobre los consolidados cuenta, por clave, la distribución global y por grupo, y produce un cruce clave por grupo con la cantidad

y el porcentaje de cada valor en cada fila, la categoría dominante y su concentración, una marca de clave poco informativa por convención documentada —categoría dominante por encima del setenta por ciento; en las claves de sí o no, proporción por fuera del diez y el noventa por ciento—, la V de Cramér para las claves categóricas y el intervalo de Wilson al noventa y cinco por ciento para las proporciones de las claves booleanas; para las claves numéricas, mínimo, media y máximo, globales y por grupo; y para los veredictos, su cuenta por estado y la confianza media por grupo. Las claves se presentan en el orden en que el esquema las declara como requeridas, porque el almacenamiento JSON de la plataforma no conserva el orden de las propiedades. Δ es ese único registro; la cardinalidad pasa de M' a uno; no se invoca ningún modelo. Ningún registro se modifica ni se elimina. *Errores:* el filtro no lanza ninguno: una entrada vacía produce un resultado sin filas, una noticia que la tabla del experto no cubre va al grupo sin asignar y una clave ausente en un análisis no se cuenta.

- **Secreto.** Los procedimientos de conteo y comparación —los umbrales de la marca de colapso, los estadísticos elegidos, el tope de quince valores por cruce con el resto agrupado— y la forma del contenido consolidado. El cambio previsto que el secreto absorbe es otra consolidación —otros estadísticos, series temporales— sin tocar Verificación ni lo que viene antes; la forma del contenido, en cambio, es la frontera con la capa de presentación: la presentación en el formulario del resultado y la proyección del sumidero la leen, de modo que cambiarla exige cambiarlas a la vez. Lo que el secreto no incluye es ninguna decisión de dominio: qué se compara, con qué agrupación y bajo qué condición entra por el sintonizador y queda escrito en el resultado ([B], [E]).

- **Superficie y rol.** El sintonizador vive en la solapa *Normalización* del estudio —política sobre los observados, agrupación, variable del experto con su tabla y un botón que la puebla con los valores presentes en el corpus, condición de inclusión y claves a consolidar—, para el grupo de analistas, y pasa a sólo lectura cuando el filtro se ejecutó. El resultado se lee en su formulario, que presenta el contenido consolidado como tablas, y desde allí se abre el tablero del sumidero. Los analistas leen y crean resultados —lo segundo, por intermedio del filtro— sin modificarlos ni eliminarlos; los operadores los leen. No hay configuración técnica.

- **Responsabilidades**: [A] —la revisión del analista y la definición de la variable de comparación—; [B] —criterios y variable son datos del estudio—; [D] —aplica la política editorial sobre los observados sin borrar su rastro—; [E] —materializa en el resultado los criterios con que consolidó—.

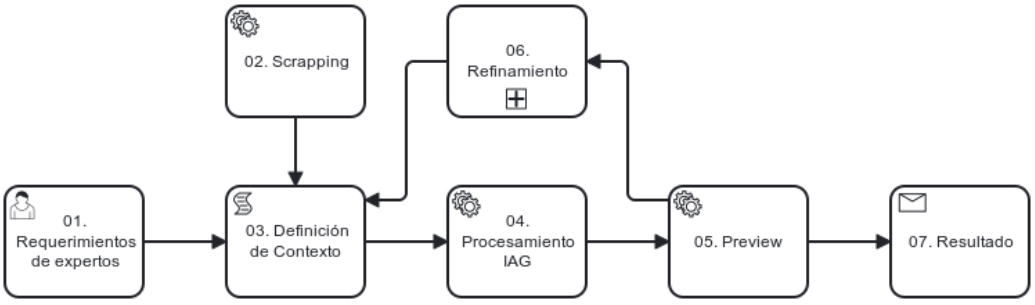
Decisión de implementación. Cuatro decisiones. La primera es que la consolidación es determinística y no interviene ningún modelo generativo: un informe narrativo producido en el sumidero llegaría sin juicio de fidelidad, en contradicción con [D]; lo que el filtro entrega es material listo para reportar, reproducible, y la lectura es del analista. La segunda es que el resultado referencia los M' veredictos aunque la política o la condición hayan excluido algunos de la consolidación: la exclusión se materializa fila por fila con su motivo, de modo que la compuerta blanda llega intacta al final del pipeline y el grafo de procedencia del resultado es completo. La tercera es que la variable del experto vive en este filtro y no en la configuración de los portales del *addon* institucional: la agrupación pertinente para una pregunta es un dato del estudio —otro estudio agrupa distinto— y modificar el *addon* institucional está excluido por [F]; la tabla de asignación se copia con el estudio en cada iteración y queda escrita en el resultado. La cuarta es que la marca de clave poco informativa es una convención documentada y no un sintonizador: no es una decisión de dominio sino una advertencia de presentación, y el analista la reporta sin interpretarla. El sumidero —la proyección aplanada del resultado, con una fila por análisis y clave, que alimenta las vistas de tabla dinámica y gráfico de la plataforma— es externo al filtro y se describe al cierre de la sección.

4.4. Síntesis del capítulo

Capítulo 5

Mapa de procesos

5.1. Proceso general

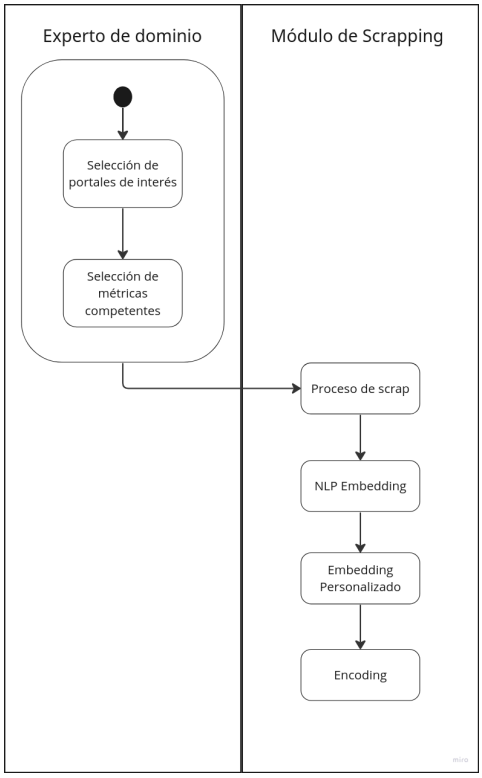


Objetivo: Diseñar e implementar un framework basado en el estilo arquitectónico de Tubos y Filtros que automatice el análisis de noticias de portales digitales mediante inteligencia artificial generativa, manteniendo trazabilidad del flujo de datos, control editorial por parte del experto del dominio y reproducibilidad del proceso analítico.
Alcance: Desarrollar un framework que permita recopilar requerimientos de expertos (01), recopilar datos del mundo real utilizando técnicas de *scraping* (02), definir el contexto del análisis mediante el uso de *prompt engineering* (03), obtener la salida de la IAG (04), realizar una visualización previa de los resultados (05), refinar la salida a través de la retroalimentación del análisis, ajustando y mejorando el contexto de manera iterativa; adaptándose progresivamente a las necesidades del experto a cargo (06), y presentar los resultados obtenidos (07).
Entrada: Datos empíricos de las noticias junto con información de contexto, requerimientos de análisis y tipo de salida deseada.
Salida: Resultados producidos por la IAG, que pueden ser en forma de narrativas, tablas, gráficos u otras representaciones visuales, dependiendo de los objetivos requeridos.
Responsabilidades: [A]–[F] —el ciclo completo realiza las seis responsabilidades transversales (Sección 1.3); cada proceso de soporte señala, junto a sus actores, las que le corresponden—.

De los procesos antes mencionados, a continuación se describen los procesos de soporte Nro. 02 -**Scraping**, 03 - **Definición de Contexto** y 04-**Procesamiento IAG**.

5.2. Recopilación de requerimientos

5.3. Scraping


Experto de dominio
Módulo de Scrapping
Selección de portales de interés
Selección de métricas competentes
Proceso de scrap
NLP Embedding
Embedding Personalizado
Encoding

Objetivo: Recolectar datos relevantes de resultados electorales, encuestas y percepción de la gente en portales digitales regionales. Además, se busca automatizar este proceso para hacerlo más eficiente y facilitar el análisis posterior de los datos extraídos.
Alcance: Selección de los portales digitales regionales más pertinentes para la obtención de datos. Por otro lado, se diseñarán formularios que permitan al experto especificar variables clave, como el rango de fechas y el tipo de elección (provincial o nacional).
Entrada: Portales digitales regionales relevantes y variables claves como el rango de fechas y el tipo de elección.
Salida: Datos procesados y transformados utilizando técnicas de NLP embedding, que permitirán su representación en forma numérica. Además, se aplicarán técnicas de encoding para almacenar los datos en un formato adecuado, teniendo en cuenta su posterior análisis y la implementación de algoritmos de inteligencia artificial.
Responsabilidades: [A] (01, 02, 05), [C] (04), [F] (03).

Inicio

01. Selección de portales de interés

Actor: Experto de dominio [A].

Actividad: Seleccionar los portales que serán scrapeados por la herramienta. Esta elección se basa en criterios de idoneidad, credibilidad, número de visitas de usuarios y región geográfica específica. El experto evaluará cuidadosamente cada portal para determinar su relevancia y confiabilidad en cuanto a los datos que se desean obtener. Esta selección se basa en el conocimiento y experiencia del experto en el dominio específico y busca garantizar que los portales elegidos aporten información valiosa y representativa para el análisis posterior del proceso de Scraping.

02. Selección de métricas competentes

Actor: Experto de dominio [A].

Actividad: Acotar la información que será recopilada por la aplicación de Scraping. Esto implica seleccionar las métricas y datos relevantes que se guardarán durante el proceso de extracción. Además de las palabras clave y los títulos importantes, el experto puede considerar la inclusión de otros elementos, como:

Categorías de información específicas relacionadas con los resultados electorales, encuestas o percepciones de la gente. Rango de fechas a recorrer para

obtener datos históricos o analizar tendencias a lo largo del tiempo. Variables demográficas o geográficas relevantes para segmentar y comprender mejor los datos recolectados. Otros criterios específicos basados en el objetivo y alcance de la investigación. El experto debe tener en cuenta que la selección de métricas competentes es fundamental para garantizar la calidad y relevancia de los datos extraídos durante el proceso de Scraping.

03. Proceso de scraping

Actor: Aplicación de Scraping [F].

Actividad: Recorrer los portales seleccionados y guardar la información relevante. Utiliza técnicas automatizadas para navegar por las páginas web, identificar los datos definidos previamente y extraerlos. La información se almacena en un formato adecuado para su posterior procesamiento y análisis.

04. NLP Embedding

Actor: Aplicación de Scraping [F].

Actividad: La aplicación utilizará técnicas de NLP (Natural Language Processing) Embedding para transformar los datos de texto recolectados en representaciones numéricas de alta dimensionalidad. Esto implica asignar un vector numérico a cada palabra o frase, capturando su significado y contexto semántico. El objetivo es organizar y estructurar de manera que las palabras similares se encuentren en un espacio vectorial cercano, permitiendo un análisis más eficiente y preciso en etapas posteriores del proceso. El NLP Embedding ayuda a la aplicación a comprender y procesar el texto de manera más sofisticada, facilitando la identificación de patrones, temas o tendencias en los datos extraídos.

05. Embedding Personalizado

Actor: Experto de dominio [A].

Actividad: Acotar o mejorar el embedding clásico aplicado a la data estructurada, de manera que pueda ser procesada de manera más efectiva por una inteligencia artificial generativa. El experto puede ajustar parámetros, aplicar técnicas avanzadas de embedding o incluso desarrollar nuevos enfoques para adaptar el embedding al contexto específico de la investigación. Puede involucrar la definición de reglas adicionales, ajustes finos en los pesos asignados a ciertos elementos o la inclusión de información adicional que sea relevante para el análisis. Este subproceso puede ser realizado de forma

manual por el experto o, en algunos casos, ser apoyado por herramientas de automatización desarrolladas específicamente para este propósito.

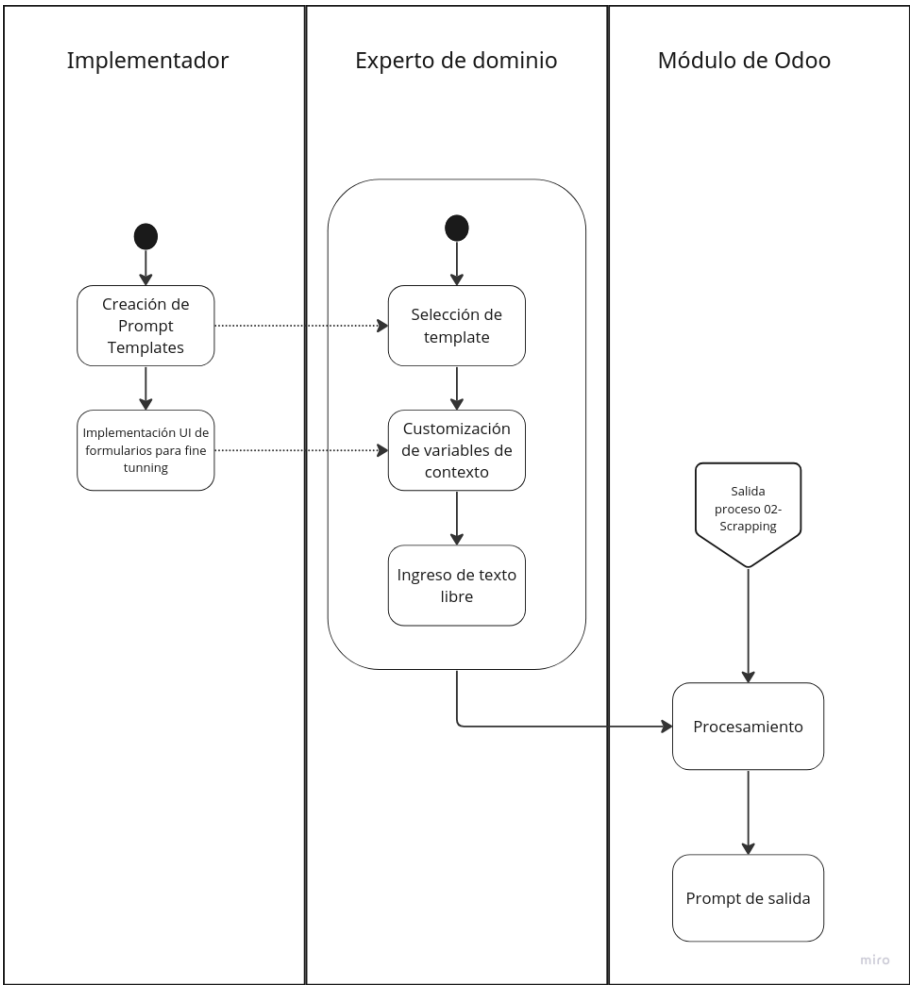
El objetivo de este subproceso es optimizar el embedding de manera personalizada y adaptada al dominio de estudio, lo que puede conducir a una mejora significativa en la calidad y precisión de los resultados generados por la inteligencia artificial generativa.

06. Encoding

Actor: Aplicación de Clustering.

Actividad: Convertir la información recopilada en un formato adecuado para la clusterización. El encoding consiste en representar los datos de manera numérica para poder aplicar algoritmos de clusterización y agruparlos en categorías o conjuntos similares. Esto implica asignar un valor numérico a cada dato o característica relevante que se utilizará en el proceso de agrupación. El proceso de encoding puede involucrar técnicas como la codificación one-hot, donde cada valor posible se convierte en una columna binaria independiente, o la codificación ordinal, donde se asignan valores numéricos a las categorías en función de un orden específico. El objetivo es proporcionar una representación numérica que capture las características y relaciones relevantes de los datos para facilitar su agrupación y análisis.

5.4. Definición de Contexto



Objetivo: Generar el prompt completo que se le va a dar como entrada a la IAG a partir de la integración entre los templates creados de antemano y la intervención del experto de dominio. Este último podrá ajustar variables de entrada y definir el formato de salida esperado para el análisis.
Alcance: Todas las interfaces gráficas necesarias para la toma de datos de los expertos
Entrada: Requerimientos blandos de expertos de dominio y datos empíricos de las noticias
Salida: Prompt completo para la IAG
Responsabilidades: [A] (03, 04, 05), [B] (01, 03, 04, 06).

Inicio

01. Creación de Prompt Templates

Actor: Implementador de la aplicación.

Actividad: Proporcionar una interfaz gráfica que habilite la selección de prompt templates para el experto de dominio. La elección de seleccionar un template en lugar de crear un prompt desde cero se realizó con el objetivo de evitar el "Síndrome de la Página en Blanco" (*Blank Page Syndrome*), el efecto en el que los usuarios, al encontrarse frente a una página en blanco, no saben por dónde comenzar (Zamfirescu-Pereira et al., 2023).

Los templates van a utilizar una lista de instrucciones básicas con características que queremos que todos cumplan basándonos en investigaciones previas sobre cómo crear un prompt adecuado (Zamfirescu-Pereira et al., 2023; Boonstra, 2024). Ej:

- Los prompts deben ser concisos, claros y fáciles de entender. Se debe evitar el lenguaje complejo y la información innecesaria.
- Usar verbos de acción como Actuar, Analizar, Clasificar, Describir, Extraer, Generar, Parsear, Resumir, etc.
- Uso de Instrucciones (qué hacer) en lugar de Restricciones (qué evitar).
- Proporcionar ejemplos de interacciones deseadas en los prompts (One-shot y Few-shot): El one-shot prompting proporciona un solo ejemplo, mientras que el few-shot prompting proporciona múltiples ejemplos. Múltiples ejemplos en few-shot prompting aumentan la probabilidad de que el modelo siga el patrón. Los ejemplos incluidos deben ser relevantes para la tarea, diversos, de alta calidad y bien escritos. Incluir casos extremos (edge cases).
- Escribir prompts que se asemejen (en cierta medida) a código fuente.

La personalización de *prompts* dentro del sistema propuesto no solo se basa en las combinaciones de ajustes que el experto pueda introducir, sino también en la comprensión de las distintas técnicas de *prompting* que permiten optimizar el comportamiento del modelo. En este sentido, los *templates* pueden variar tanto en los resultados esperados como en su estructura, dependiendo del tipo de información que procesen —por ejemplo, texto, imágenes, o una combinación de ambos— y del modelo utilizado (como ChatGPT para texto o DALL · E 2 para imágenes).

Una forma efectiva de categorizar y configurar estos *templates* es mediante la aplicación de estrategias de *prompting* diferenciadas. Distinguir entre estos tipos proporciona un marco conceptual claro para diseñar *prompts* con intención y eficacia (Boonstra, 2024):

- **System Prompting:** Establece el contexto general y el propósito de la tarea. Sirve para definir las capacidades fundamentales del modelo y cómo debe comportarse en términos estructurales. Por ejemplo, se puede usar para solicitar que las respuestas sigan un formato específico, como una estructura en JSON o una tabla.

- **Contextual Prompting:** Proporciona información específica o de fondo que es relevante para la tarea. Este tipo de *prompting* ayuda al modelo a interpretar correctamente la situación, como cuando se incluyen directamente en el *prompt* datos empíricos provenientes de noticias, investigaciones u otras fuentes confiables.

- **Role Prompting:** Asigna una identidad o rol al modelo, moldeando el estilo y tono de la salida. Aunque es menos central para tareas de análisis objetivo, puede ser útil cuando se requiere una narrativa con un enfoque determinado, como la voz de un periodista, un analista político o un divulgador científico.

Así, la construcción de *prompts* eficaces no solo depende del contenido a procesar o del modelo empleado, sino también del diseño intencional del contexto comunicativo. Integrar estos enfoques permite al experto desarrollar *templates* más robustos, flexibles y alineados con los objetivos analíticos del sistema, favoreciendo un mayor control sobre la salida generada por los modelos de lenguaje.

Agregaremos a la interfaz gráfica una forma intuitiva de seleccionar las distintas configuraciones mencionadas.

02. Implementación UI de formularios para fine tuning

Actor: Implementador de la aplicación.

Actividad: Proporcionar una interfaz gráfica que habilite el ajuste de las variables libres que no quedaron definidas en los templates anteriores.

03. Selección de template

Actor: Experto de dominio [A].

Actividad: Seleccionar un template inicial para el análisis.

04. Customización de variables de contexto
Actor: Experto de dominio [A].
Actividad: Configuración de las variables necesarias para el *template* seleccionado. En este paso, el experto podrá usar una batería de herramientas de interfaz gráfica para personalizar su contexto, por ejemplo: fechas desde-hasta de publicaciones, palabras clave, combinación de conceptos, etc.

Además de estos filtros contextuales iniciales, esta etapa también incluye la configuración de parámetros clave que afectan el comportamiento del modelo de lenguaje (Boonstra, 2024).

- **Temperature:** Controla el grado de aleatoriedad en la generación de texto. Un valor más bajo (cercano a 0) hace que el modelo sea más determinístico y predecible, lo que resulta útil para tareas fácticas o consistentes, como el análisis de noticias. Valores más altos incrementan la diversidad de la salida, lo cual puede ser útil en tareas creativas. Esta variable ya ha sido contemplada en la tabla del proceso de *scraping*, y forma parte de la definición operativa del contexto para la Inteligencia Artificial Generativa (IAG).

- **Top-K y Top-P:** Son técnicas que restringen la predicción del siguiente token a un subconjunto de tokens con mayor probabilidad. *Top-K* limita la elección a los K tokens más probables, mientras que *Top-P* (también conocido como *nucleus sampling*) considera los tokens cuya probabilidad acumulada alcanza un umbral P. Ambos parámetros permiten modular la aleatoriedad y control de la generación.

- **Output Length:** Define la cantidad máxima de tokens a generar por parte del modelo. Cuantos más tokens se generen, mayor será el cómputo requerido, lo que impacta en el tiempo de respuesta y los recursos utilizados. Esta longitud puede controlarse directamente en el sistema o bien incluirse como una instrucción dentro del *prompt*. La tesina ya hace referencia al "Token Limit" dentro de las tablas de ejemplo de *prompts*.

- **Uso de variables:** Para aumentar la reutilización y adaptabilidad de los *templates*, es recomendable incorporar variables dinámicas en los *prompts*. Estas permiten insertar datos contextuales (como fechas,

actores, ubicaciones o temas) sin necesidad de reescribir el *prompt* completo cada vez. Esta estrategia, también mencionada previamente en la tesina, optimiza el proceso de personalización y facilita la integración del sistema en flujos de trabajo automatizados.

05. Ingreso de texto libre

Actor: Experto de dominio [A].

Actividad: El experto podrá ingresar libremente aclaraciones o ejemplos que querrá incluir en el prompt para dejar en claro cual es su idea de análisis. Con el fin de guiar de manera efectiva el proceso de generación, resulta útil incluir ejemplos o escenarios específicos en los prompts. Estos ejemplos pueden ilustrar los tipos de interacciones o respuestas deseadas que la IAG debería generar. Al incorporarlos la IAG puede aprender y emular el comportamiento deseado.

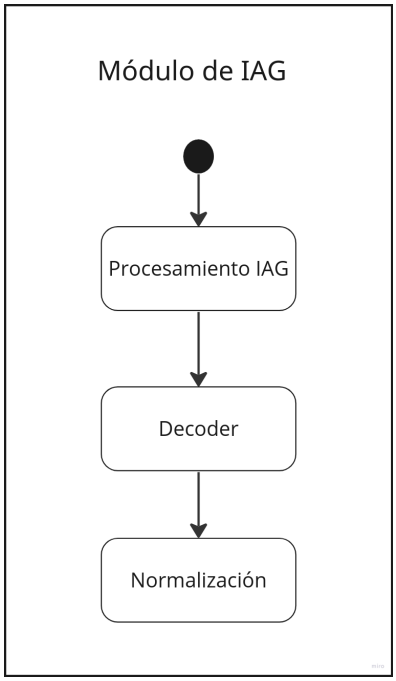
En un estudio previo todos los participantes mostraron una fuerte preferencia por la instrucción directa en lugar de proporcionar ejemplos en línea, y cuando se les animó a dar ejemplos, la mayoría de los participantes lo hicieron y señalaron su efectividad en la mejora de resultados (Zamfirescu-Pereira et al., 2023).

06. Procesamiento

Actor: Módulo de Odoo [B].

Actividad: Estructurizar el prompt que sera entregado a la IAG utilizando las entradas del experto y los datos empíricos de las noticias. Se espera la salida de una string formateada en algún lenguaje de marcado que sea interpretado por la IAG.

5.5. Procesamiento IAG



Responsabilidades: [C] (01, 02), [A], [D] y [E] (03).

01. Procesamiento IAG
Actor: Aplicación de Inteligencia Artificial Generativa [C].
Actividad: Utilizando los datos previamente descriptos, el prompt adecuadamente construido y una conexión estable con la API del modelo generativo seleccionado, se ejecuta la petición correspondiente. El objetivo principal de esta actividad es generar resultados creativos y originales a partir de la inteligencia artificial generativa.

02. Decoder
Actor: Aplicación de Decodificación [C].
Actividad: Preparar la información generada por la IAG para su posterior reutilización en otros contextos o plantillas.

03. Normalización
Actor: Investigador o analista de datos [A], [D], [E].

Actividad: Realizar un raconto final de todo el camino recorrido durante el proceso, desde los datos iniciales hasta la generación de la salida del modelo generativo. El objetivo de esta actividad es realizar una normalización de los datos y resultados obtenidos para poder realizar comparaciones, evaluaciones y análisis posteriores de manera coherente y sistemática.

La normalización implica varias tareas, como:

- Documentación detallada: documentar todo el proceso en un informe o documento, donde se especifican todas las etapas, herramientas utilizadas, parámetros, ajustes y cualquier otro detalle relevante. Esta documentación sirve como referencia para futuros análisis y también permite una mayor transparencia y reproducibilidad del proceso.

- Organización de los datos: organizar datos recopilados, los parámetros utilizados, el prompt generado y cualquier otra información relevante en una estructura coherente y accesible. Esto facilita la revisión y comparación de los datos en etapas posteriores.

- Comparación con variables: al definirse variables específicas, el investigador realiza una comparación de la salida generada por el modelo con estas variables. Esto permite evaluar qué tan bien se ajusta la salida a las expectativas y criterios establecidos, y también identificar posibles desviaciones o inconsistencias.

- Análisis de resultados: analizar en función de los criterios y objetivos deseados. Esto puede incluir la evaluación de la coherencia del texto generado, la calidad de las respuestas, la relevancia semántica o cualquier otro aspecto específico relacionado con el proyecto de investigación.

5.6. Visualización previa

5.7. Refinamiento iterativo

5.8. Presentación de resultados

5.9. Síntesis del capítulo

Capítulo 6

Ejemplo de aplicación

6.1. Selección del caso

6.2. Configuración inicial

6.3. Ejecución del scraping

6.4. Definición de contexto y template utilizado

6.5. Resultados del análisis generativo

6.6. Discusión

Capítulo 7

Conclusiones y Trabajo a Futuro

Apéndice A

Tipos de datos del pipeline

La especialización de tubos tipados adoptada en el Capítulo 3 exige que cada tubo declare el tipo del *recordset* que transporta (Subsección 3.2.4). El capítulo nombra esos tipos —`Noticia`, `Embedding`, `Prompt`, `RespuestaIAG`, `Análisis`, `Veredicto` y `Resultado`— y describe las transformaciones que los producen; este apéndice completa la caracterización especificando la estructura interna de cada uno. Las definiciones constituyen los contratos de datos del framework: fijan qué produce cada filtro y qué puede esperar el siguiente, con independencia de la plataforma sobre la que el framework se materialice. Se especifica además la estructura del *template* —la entidad del repositorio que ancla la definición del estudio—, que no circula por los tubos pero es referida por los tipos del flujo (Sección A.9).

A.1. Notación y criterios de diseño

Cada tipo se especifica como un registro de pares `clave: tipo`, en una notación de estilo JSON. Los tipos primitivos son `string`, `int`, `float` y `datetime`; `float[]` denota una lista de valores y `objeto` una estructura anidada cuyo esquema no fija el framework. El marcador `ref` denota una referencia a un registro del repositorio común —el comentario de cada clave indica el registro referenciado— y `ref[]`, una colección ordenada de referencias; esta representación es la contraparte, a nivel del dato, del transporte por referencia que caracteriza a los conectores del sistema (Subsección 3.2.5). El sufijo `?` marca una clave opcional. La cardinalidad con que cada tipo circula por los tubos —`Tipo[c]`— se detalló en la Subsección 3.2.3 y se recuerda al inicio de cada

sección.

Tres criterios, aplicados de manera uniforme, gobiernan qué claves integran cada contrato y cuáles se excluyen deliberadamente.

1. **Contrato mínimo** ([E]). Cada tipo contiene únicamente la carga útil que algún filtro posterior consume y la procedencia del dato —cuándo se produjo y a partir de qué registros—. Toda clave adicional volvería al contrato más exigente de lo que el pipeline requiere, dificultando la sustitución de los filtros que lo producen.

2. **La procedencia viaja; la identidad del productor, no** ([C], [E]). Registrar a partir de qué dato se produjo un registro es procedencia, y sostiene la auditabilidad; registrar con qué recurso técnico se lo produjo —el modelo de *embedding*, el proveedor generativo— describiría la identidad del filtro productor, que el estilo veda conocer a los restantes filtros (Subsección 3.1.2). Cambiar ese recurso equivale a sustituir el filtro, no a alterar los datos que emite.

3. **Los esquemas de dominio se delegan al estudio** ([B]). Donde el contenido depende del caso analítico —`Análisis` y `Resultado`—, el framework fija el sobre (procedencia y carga estructurada) y delega el esquema interno a la definición del estudio, declarada en el *template* (Sección A.9). Fijar claves de dominio en el contrato casaría al framework con un caso de estudio particular, contra el requisito de diversidad analítica (Subsección 3.1.3).

A.2. `Noticia`

Producido por Extracción; circula como `Noticia[N]` por el tubo 2→3, donde N es el tamaño del corpus.

```
Noticia {
  titulo:            string    // título de la publicación
  copete?:           string    // resumen editorial del portal
  texto:             string    // cuerpo completo, normalizado
  link:              string    // URL canónica; clave única
  medio:             ref       // portal de origen
  departamento:      string    // zona geográfica del portal
  fecha_hora?:       datetime  // fecha declarada de publicación
  fecha_registro:    datetime  // fecha de ingreso al corpus
  reglas_aplicadas:  string    // reglas que la seleccionaron
}
```

Las cuatro primeras claves constituyen el contenido extraído: el título, el copete —el resumen editorial que acompaña al título—, el cuerpo textual completo y el enlace canónico de la publicación, que oficia además de identificador único del registro en el repositorio. Las restantes registran la procedencia: el portal del que la noticia fue extraída, la adscripción geográfica de ese portal, la fecha de publicación que el portal declara —opcional, pues no todo portal la expone en forma interpretable—, la fecha cierta de incorporación al corpus y las reglas de extracción que motivaron su selección. Esta última clave completa la trazabilidad de la etapa: documenta por qué cada noticia integra el corpus, en términos de las reglas configuradas en el sintonizador de Extracción. El copete comparte la opcionalidad de la fecha de publicación: su disponibilidad depende de lo que el portal publique y de la vía de extracción empleada.

A.3. `Embedding`

Producido por Enriquecimiento semántico; circula como `Embedding[N]` por el tubo interno 4→5 del filtro compuesto Recuperación aumentada.

```
Embedding {
  noticia:         ref       // noticia que codifica
  vector:          float[]   // representación vectorial
  fecha_registro:  datetime  // fecha de cómputo del vector
}
```

El tipo reifica la transformación de Enriquecimiento semántico: cada registro asocia una noticia —por referencia— con su representación vectorial, de modo que el filtro emite las representaciones y no las noticias que las

originan (Subsección 3.2.3). El contrato ilustra el segundo criterio de la Sección A.1: no incluye un identificador del modelo que computó el vector, porque ese dato describiría la identidad del filtro productor y no la procedencia del dato. Dos vectores sólo son comparables dentro de un mismo espacio semántico; reemplazar el modelo de *embedding* equivale, por lo tanto, a sustituir el filtro completo y recomputar las representaciones del corpus, no a mezclar en el repositorio vectores de espacios heterogéneos.

A.4. `Prompt`

Producido por Composición de contexto; circula como `Prompt[M]` por el tubo 6→7, donde M es la cantidad de unidades de análisis del estudio.

```
Prompt {
  texto:            string    // prompt completo para el modelo
  parametros: {               // parámetros de generación
    temperatura:      float
    longitud_salida:  int
  }
  noticias:         ref[]     // noticias del contexto (M:N)
  template:         ref       // template de origen
  fecha_registro:   datetime  // fecha de composición
}
```

La clave `texto` materializa la composición completa —*template*, variables de contexto, aclaraciones de texto libre y material empírico recuperado—, por lo que el registro documenta en forma íntegra lo que se envía al modelo generativo; los ingredientes de la composición no se registran por separado, pues su combinación está ya materializada en el texto y el *template* de origen —con el objetivo del análisis y el esquema de respuesta que éste declara (Sección A.9)— queda anclado por referencia. La clave `parametros` merece una precisión: la temperatura y la longitud de salida las fija el experto en el sintonizador de Composición, pero su consumidor es Invocación generativa; dado que los sintonizadores no se invocan entre filtros (Subsección 3.2.6), la única vía compatible con el estilo para hacérselos llegar es el propio dato. No se trata de configuración del filtro filtrándose en el contrato, sino de información que un consumidor aguas abajo espera. La clave `noticias`, por último, registra la relación de muchos a muchos entre noticias y *prompts* que la recuperación semántica establece (Subsección 3.2.3).

A.5. `RespuestaIAG`

Producido por Invocación generativa; circula como `RespuestaIAG[M]` por el tubo 8→9.

```
RespuestaIAG {
  prompt:          ref       // prompt que la originó
  texto:           string    // respuesta cruda, sin decodificar
  fecha_registro:  datetime  // fecha de recepción
}
```

El registro conserva la respuesta cruda tal como la produjo el servicio generativo, sin interpretación alguna: su decodificación corresponde al filtro siguiente. Por el segundo criterio de diseño quedan fuera el proveedor, el modelo y sus credenciales —identidad del filtro de Invocación, cuya sustitución completa es precisamente lo que el aislamiento de la etapa habilita (Subsección 3.2.2)—, así como la telemetría operacional de la petición, que ningún filtro consume.

A.6. `Análisis`

Producido por Decodificación; circula como `Análisis[M]` por el tubo 10→11.

```
Análisis {
  respuesta:       ref       // respuesta cruda de origen
  contenido:       objeto    // estructura analítica; esquema
                             //   declarado por el template
  fecha_registro:  datetime  // fecha de decodificación
}
```

El tipo aplica el tercer criterio de diseño: el framework fija el sobre y delega el esquema. La clave `contenido` porta la estructura analítica ya validada y canonicalizada, cuyo esquema interno —qué claves, con qué tipos y vocabularios— lo declara el *template* del estudio (Sección A.9): la definición analítica que declara el objetivo del análisis y especifica qué se le pide al modelo especifica también, como su contraparte, qué estructura se espera de la respuesta. El `contenido` queda así plenamente caracterizado: es una instancia del `esquema_respuesta` del *template*, producida bajo el `objetivo` que éste declara. Esta delegación explica, además, cómo Decodificación valida sin exponer sintonizador: alcanza el esquema esperado siguiendo los punteros de procedencia que van de la respuesta a su *prompt* y de éste a su

template (Subsección 3.2.2). El tipo no incluye un estado de validación: la validación estructural es una compuerta y no un atributo, de modo que toda instancia de `Análisis` que circula por el pipeline es, por construcción, una estructura conforme. Tampoco incluye un juicio sobre la fidelidad de su contenido a la evidencia: esa valoración —de otra naturaleza que la conformidad estructural— la produce el filtro de Verificación aguas abajo y se reifica en un registro `Veredicto` separado (Sección A.7), no como una clave de este contrato.

A.7. `Veredicto`

Producido por Verificación; circula como `Veredicto[M]` por el tubo 12→13.

```
Veredicto {
  analisis:        ref       // análisis juzgado
  estado:          string    // válido | observado | no sustentado
  confianza:       float     // fidelidad del análisis a la evidencia
  sustento:        objeto     // evidencia que respalda o refuta cada
                             //   afirmación; esquema según el
    criterio
                             //   de validez del estudio
  fecha_registro:  datetime  // fecha de verificación
}
```

El tipo reifica el juicio de fidelidad que produce Verificación: cada registro asocia un análisis —por referencia— con la valoración de en qué medida sus afirmaciones están sostenidas por la evidencia empírica de la que dicen derivar (Subsección 3.2.2). La clave `estado` porta el dictamen canónico —válido, observado o no sustentado—; `confianza` lo cuantifica, reificando a nivel del dato la calibración de incertidumbre que la revisión sistemática identificó como brecha (Sección 2.4). Que ambas claves acompañen al veredicto, y que ninguno de los análisis se descarte, es lo que materializa la *compuerta blanda* ([D]): el filtro califica y señala, pero no suprime, y deja la decisión editorial última al analista. La clave `sustento` aplica el tercer criterio de diseño —delega al estudio el esquema de dominio—: su estructura interna —qué fragmentos se citan y cómo se los pondera— depende del criterio de validez configurado en el sintonizador de Verificación, no del framework. Por el segundo criterio, en cambio, el contrato omite deliberadamente la identidad del ejecutor de la verificación —el procedimiento automático, el revisor hu-

mano asistido o el conjunto de agentes que confrontan sus juicios—: registrar con qué recurso se produjo el veredicto describiría la identidad del filtro productor, cuya sustitución es precisamente lo que la agnosticidad del ejecutor habilita (Subsección 3.1.3), del mismo modo que `Embedding` omite el modelo que computó el vector y `RespuestaIAG`, el proveedor generativo.

A.8. `Resultado`

Producido por Normalización; se entrega por el puerto 14 al sumidero como registro único, tipado sin cardinalidad.

```
Resultado {
  veredictos:      ref[]     // veredictos consolidados
  contenido:       objeto    // producto consolidado; esquema
                             //   según criterios del estudio
  fecha_registro:  datetime  // fecha de consolidación
}
```

Como `Análisis`, el tipo fija el sobre y delega el esquema: la forma del producto consolidado —distribuciones temáticas, comparaciones entre portales, series temporales— depende del caso analítico y la definen los criterios de consolidación del estudio. La clave de procedencia `veredictos` referencia los veredictos que Normalización consolidó —y, a través de ellos, los análisis verificados que estos juzgan—, de modo que el juicio de fidelidad queda incorporado al grafo de procedencia del producto final (Sección A.11). Esos criterios, así como las variables de comparación del analista, no integran el contrato: son configuración del sintonizador de Normalización consumida por el propio filtro, y su efecto queda materializado en el contenido consolidado.

A.9. `Template`

A diferencia de los siete tipos anteriores, el *template* no circula por los tubos: es la entidad del repositorio común que ancla la definición del estudio ([B]), creada y editada por el experto del dominio ([A]) a través del panel de *templates* (Subsección 3.2.6) e incorporada al flujo por referencia —cada `Prompt` la ancla directamente y cada `Análisis` la alcanza siguiendo sus punteros de procedencia—.

```
Template {
  nombre:              string   // identificación de la plantilla
  objetivo:            string   // objetivo del análisis: alcance,
                                //   paradigma, referencias
  instrucciones:       string   // cuerpo del prompt, con variables
  variables:           objeto   // variables de contexto declaradas
  esquema_respuesta:   objeto   // esquema esperado del contenido
}
```

El tipo reifica los tres datos de entrada para la construcción de *prompts* anticipados en el Capítulo 1: los datos empíricos ingresan por la recuperación semántica en Composición de contexto; los objetivos requeridos para el análisis —alcance, paradigma de análisis, referencias a tener en cuenta— se declaran en la clave `objetivo`; y el tipo de salida o representación se fija en `esquema_respuesta`, el esquema contra el cual valida Decodificación. La clave `objetivo` merece énfasis: el pipeline no transporta únicamente datos empíricos transformados, sino también la intención analítica que les da sentido —bajo qué paradigma, con qué alcance y contra qué referencias se analiza—, que viaja por referencia con cada `Prompt` y cada `Análisis`, del mismo modo que todo contenido del sistema. Por tratarse de configuración editada por sintonizador, los registros `Template` quedan fuera del alcance de la monotonía del flujo (Sección A.10); la referencia ancla la identidad de la plantilla, y la composición íntegra efectivamente enviada al modelo queda materializada en el texto de cada `Prompt` (Sección A.4), lo que preserva la auditabilidad de lo ejecutado aun si la plantilla es editada con posterioridad.

A.10. Los filtros como transformaciones de estado

Definidos los tipos, las especificaciones funcionales del Capítulo 3 admiten una síntesis formal. Según la formulación introducida en la Subsección 3.2.2 —que extiende la notación funcional del catálogo de Cristiá and Pomponio (2025, p. 24) a la variante heterogénea—, cada filtro realiza una transformación $f : (T[c],\ Rep) \to (T'[c'],\ Rep')$, donde $Rep' = Rep \cup \Delta_f$, Δ_f es el conjunto de registros que el filtro persiste y el *recordset* emitido es $refs(\Delta_f)$. Las siete firmas del pipeline se encadenan:

Extracción:	(flujo del portal, Rep_0) $\rightarrow$ (`Noticia[N]`, Rep_1)
Enriquecimiento:	(`Noticia[N]`, Rep_1) $\rightarrow$ (`Embedding[N]`, Rep_2)
Composición:	(`Embedding[N]`, Rep_2) $\rightarrow$ (`Prompt[M]`, Rep_3)
Invocación:	(`Prompt[M]`, Rep_3) $\rightarrow$ (`RespuestaIAG[M]`, Rep_4)
Decodificación:	(`RespuestaIAG[M]`, Rep_4) $\rightarrow$ (`Análisis[M]`, Rep_5)
Verificación:	(`Análisis[M]`, Rep_5) $\rightarrow$ (`Veredicto[M]`, Rep_6)
Normalización:	(`Veredicto[M]`, Rep_6) $\rightarrow$ (`Resultado`, Rep_7)

con $Rep_i = Rep_{i-1} \cup \Delta_i$, donde cada Δ_i es el conjunto de registros del tipo correspondiente que la etapa persiste: los N registros `Noticia`, los N `Embedding`, los M `Prompt`, y así sucesivamente. El encadenamiento de los subíndices exhibe el enhebrado del repositorio a través del pipeline: cada filtro recibe el estado que dejó el anterior. La monotonía —$Rep_0 \subseteq Rep_1 \subseteq \cdots \subseteq Rep_7$— se afirma sobre los registros del flujo de datos: los sintonizadores editan configuración (*templates*, reglas de extracción, criterios de validez y de consolidación), pero esa configuración no pertenece al flujo ni es modificada por ningún filtro. El estado final Rep_7 contiene, por lo tanto, la historia completa del estudio: el corpus, sus representaciones semánticas, los *prompts* compuestos, las respuestas crudas, los análisis decodificados, los veredictos de verificación y el resultado consolidado.

Por el cierre composicional del estilo (Subsección 3.1.1), la composición de las siete firmas es a su vez un filtro en el sentido extendido: el sistema completo realiza la transformación (flujo del portal, Rep_0) $\rightarrow$ (`Resultado`, Rep_7), que convierte el flujo de los portales digitales en un producto analítico consolidado y deja persistida en el repositorio la historia íntegra de esa conversión. Sobre esa historia se construye el grafo de procedencia que se describe a continuación.

A.11. El grafo de procedencia

Leídas en conjunto, las claves de procedencia de los siete tipos definen sobre el repositorio un *grafo de procedencia* en el sentido del modelo PROV del W3C: un grafo dirigido, con raíz en la entidad cuya procedencia se quiere explicar, cuyas aristas apuntan a las entidades de las que ésta se derivó (Moreau and Missier, 2013). Los vértices son los registros persistidos; cada clave de procedencia es una arista de derivación —la relación *wasDerivedFrom* de PROV— que va del registro producido al registro consumido, y cada filtro

es la actividad que la genera. Como las aristas apuntan al pasado, el grafo recorre el pipeline en sentido inverso. Es la realización, a nivel del dato, de la responsabilidad de procedencia [E].

Conviene distinguir el esquema del grafo de sus instancias. El esquema —qué tipo referencia a qué tipo— es lineal:

`Resultado` → `Veredicto` → `Análisis` → `RespuestaIAG` → `Prompt` → `Noticia` (y *template*)

La instancia que un estudio deposita en el repositorio, en cambio, no es una cadena sino un grafo dirigido acíclico: las claves `Resultado.veredictos` y `Prompt.noticias` son referencias múltiples, de modo que un resultado ramifica en tantos veredictos como consolidó y cada *prompt* en tantas noticias como la recuperación semántica le incorporó; y como una misma noticia puede participar de varios *prompts*, distintos caminos convergen en ancestros compartidos. Las aristas restantes —de `Veredicto` a `Análisis`, de `Análisis` a `RespuestaIAG`, de `RespuestaIAG` a `Prompt`— son referencias únicas y obligatorias: cada registro conserva un *puntero a su predecesor*, en el mismo sentido en que los algoritmos de búsqueda en grafos anotan en cada vértice el vértice desde el que fue alcanzado y reconstruyen luego el camino hacia el origen siguiendo esos punteros sobre el subgrafo de predecesores (Cormen et al., 2009, §22.2). La rama `Embedding` → `Noticia` completa el cuadro para la capa de recuperación.

Sobre esta estructura, la pregunta que la responsabilidad [E] obliga a responder —qué registros intervinieron en la producción de un resultado o de un veredicto dado— es una consulta de *backward reachability*: el conjunto de los ancestros del registro de partida, esto es, la clausura transitiva de la relación de derivación restringida a los vértices alcanzables desde él. Cuevas-Vicenttín et al. (2012) señalan que las preguntas de procedencia sobre un grafo dirigido se traducen en dos tipos de consultas bien conocidos, *reachability* y *regular path queries*, resolubles por recorrido del grafo o por cómputo de la clausura transitiva. Desde el producto final del estudio puede así obtenerse, en una única consulta de ancestros, qué veredictos de verificación lo componen, sobre qué análisis se pronuncia cada uno, de qué respuestas crudas provienen esos análisis, con qué *prompts* —y bajo qué *template* y parámetros— se generaron, sobre qué noticias se apoyó cada composición y, para cada noticia, de qué portal fue extraída, cuándo, y qué reglas motivaron su ingreso al corpus. La consulta inversa —*forward reachability* desde una noticia: en qué *prompts*,

análisis, veredictos y resultados incidió— queda igualmente definida sobre el mismo grafo, aunque el framework no la requiere.

La documentación del recorrido del proceso que la especificación de Normalización promete (Subsección 3.2.2) no requiere, por lo tanto, un registro adicional: es una propiedad estructural del sistema de tipos, y constituye la materialización, a nivel del dato, de la auditabilidad lineal que motivó la elección del estilo (Subsección 3.1.3). La literatura sobre procedencia de datos —desde su caracterización fundacional en bases de datos (Buneman et al., 2001; Cheney et al., 2009) hasta los relevamientos más amplios del campo (Herschel et al., 2017)— llama *linaje* a esta misma información cuando se la usa para reconstruir o recomputar un dato a partir de sus orígenes; aquí se conserva el término *procedencia*, que es el del modelo PROV y el de la responsabilidad [E].

Bibliografía

Hamza Ameer. NexGen MediaCraft: Real-time AI news anchoring with LLM-based generation and web scraping. Zenodo, 2025. URL `https://doi.org/10.5281/zenodo.15856484`.

Apache Software Foundation. XComs — apache airflow documentation, 2026. URL `https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/xcoms.html`. Consultado en junio de 2026.

Nafeesa Begum J and Chandru D. An automated daily news reports generating application involving keyword-based news scraping, summarization, and sentimental analysis leveraging NLP models. *International Journal for Research in Applied Science and Engineering Technology*, 12(11): 1556–1565, 2024. doi: 10.22214/ijraset.2024.65443.

Amir Behbahanian. GlimzAI: A fully automated AI-powered news aggregation, predictive polling, and content distribution platform using LLM-based agentic workflows. ResearchGate publication 385661008, s.f. URL `https://www.researchgate.net/publication/385661008`. Fecha de publicación no verificable; ResearchGate restringe el acceso. La cita usa "s.f." por convención conservadora.

D. Berry. The computational turn: thinking about the digital humanities. *Culture Machine, Vol. 12*, 2011.

Franck Bertagnolio, Michaela Herr, and Kaj Dam Madsen. A roadmap for required technological advancements to further reduce onshore wind turbine noise impact on the environment. 2023.

Lee Boonstra. Prompt engineering. White paper, Google/Kaggle, 2024. URL `https://www.kaggle.com/whitepaper-prompt-engineering`. Guía técnica sobre ingeniería de prompts para modelos de lenguaje.

Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas C. Müller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake Vanderplas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: Experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013. URL `https://arxiv.org/abs/1309.0238`.

Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44503-X_20.

Mario Carlón. *Circulación del sentido y construcción de colectivos en una sociedad hipermediatizada*. Nueva Editorial Universitaria, San Luis, 2020.

Weisi Chen, Ahmad El Majzoub, Islam Al-Qudah, and Fethi A. Rabhi. A CEP-driven framework for real-time news impact prediction on financial markets. *Service Oriented Computing and Applications*, 2023. URL `https://doi.org/10.1007/s11761-023-00358-8`.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1 (4):379–474, 2009. doi: 10.1561/1900000006.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition, 2009.

N. Couldry and A. Hepp. The continuing lure of the mediated centre in times of deep mediatization: Media events and its enduring legacy. *Media, Culture & Society, 40(1)*, 2017.

Maximiliano Cristiá. Diseño de software. Apunte de cátedra, Ingeniería de Software 2, Facultad de Ciencias Exactas, Ingeniería y Agrimensura, Universidad Nacional de Rosario, 2026. URL `https://www.fceia.unr.edu.ar/ingsoft/diseno-a.pdf`.

Maximiliano Cristiá and Laura Pomponio. Catálogo incompleto de estilos arquitectónicos. Apunte de cátedra, Ingeniería de Software 2, Facultad

de Ciencias Exactas, Ingeniería y Agrimensura, Universidad Nacional de Rosario, 2025.

Víctor Cuevas-Vicenttín, Saumen Dey, Sven Köhler, Sean Riddle, and Bertram Ludäscher. Scientific workflows and provenance: Introduction and research opportunities. *Datenbank-Spektrum*, 12(3):193–203, 2012. doi: 10.1007/s13222-012-0100-z. URL `https://arxiv.org/abs/1311.4610`.

Robert M. Entman. Framing: Toward clarification of a fractured paradigm. *Journal of Communication*, 43(4):51–58, 1993. doi: 10.1111/j.1460-2466.1993.tb01304.x.

Yunfan Gao, Yun Xiong, Xinyu Gao, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024. URL `https://arxiv.org/abs/2312.10997`. Versión revisada, v3, abril 2024.

David Garlan and Mary Shaw. An introduction to software architecture. Technical Report CMU-CS-94-166, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, January 1994. URL `https://www.cs.cmu.edu/afs/cs/project/able/ftp/intro_softarch/intro_softarch.pdf`. También publicado como CMU Software Engineering Institute Technical Report CMU/SEI-94-TR-21, ESC-TR-94-21.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. arXiv:2501.10868 [cs.CL], 2025. URL `https://arxiv.org/abs/2501.10868`.

I. Gindin and M. Busso. Investigaciones en comunicación en tiempos de big data: sobre metodologías y temporalidades en el abordaje de redes sociales. *Revista Científica de Estrategias, Tendencias e Innovación en Comunicación, nº15. 25-43.*, 2018.

Samar Haider, Amir Tohidi, Jenny S. Wang, Timothy Dörr, David M. Rothschild, Chris Callison-Burch, and Duncan J. Watts. The media bias detector: A framework for annotating and analyzing the news at scale. *arXiv*

preprint arXiv:2509.25649, 2025. doi: 10.48550/arXiv.2509.25649. URL https://arxiv.org/abs/2509.25649.

Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. A survey on provenance: What for? What form? What from? *The VLDB Journal*, 26(6):881–906, 2017. doi: 10.1007/s00778-017-0486-1.

Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, Boston, MA, 2003. ISBN 978-0321200686.

A. Hotho, A. Nürnberger, and G. Paaß. A brief survey of text mining. *Ldv Forum, Vol. 20. 19–62*, 2005.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL https://arxiv.org/abs/2005.11401.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, et al. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023. URL https://arxiv.org/abs/2306.05499.

Niklas Luhmann. *The Reality of the Mass Media*. Stanford University Press, Stanford, CA, 2000.

Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://arxiv.org/abs/2303.17651.

Maxwell McCombs, Juan Pablo Llamas, Esteban Lopez-Escobar, and Federico Rey. Candidate images in Spanish elections: Second-level agenda-setting effects. *Journalism & Mass Communication Quarterly*, 74(4):703–717, 1997. doi: 10.1177/107769909707400404.

Maxwell E. McCombs and Donald L. Shaw. The agenda-setting function of mass media. *The Public Opinion Quarterly*, 36(2):176–187, 1972.

Jean-Guy Meunier. *Computational Semiotics*. Bloomsbury, London, 2021.

Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

Microsoft. Pipes and filters pattern — azure architecture center, 2025. URL `https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters`. Consultado en junio de 2026.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *Proceedings of ICLR 2013 Workshop*, 2013. URL `https://arxiv.org/abs/1301.3781`.

Jeremiah Milbauer, Ziqi Ding, Zhijin Wu, and Tongshuang Wu. NewsSense: Reference-free verification via cross-document comparison. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 422–430, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-demo.39. URL `https://aclanthology.org/2023.emnlp-demo.39/`.

Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium (W3C), April 2013. URL `https://www.w3.org/TR/2013/REC-prov-dm-20130430/`. Editores. Publicado el 30 de abril de 2013.

Amul Neupane, Kapalik Khanal, Nitesh Nepal, and Naseeb Dangi. Advanced news aggregation and content generation using LLMs and NLP algorithms. *European Journal of Applied Science, Engineering and Technology*, 3(2): 295–303, March 2025. ISSN 2786-9342. doi: 10.59324/ejaset.2025.3(2).24. URL `https://ieeexplore.ieee.org/abstract/document/11079577`.

Odoo S.A. ORM API. Odoo 17.0 Developer Documentation, 2024. URL `https://www.odoo.com/documentation/17.0/developer/reference/backend/orm.html`. Consultado en junio de 2026.

Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, Roger Chou, Julie Glanville, Jeremy M. Grimshaw, Asbjørn Hróbjartsson, Manoj M. Lalu, Tianjing Li, Elizabeth W. Loder, Evan Mayo-Wilson, Steve McDonald, Luke A. McGuinness, Lesley A. Stewart, James Thomas, Andrea C. Tricco, Vivian A.

Welch, Penny Whiting, and David Moher. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021. doi: 10.1136/bmj.n71. URL https://www.bmj.com/content/372/bmj.n71.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, et al. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*, 2023. URL https://arxiv.org/abs/2304.03442.

David L. Parnas, Paul C. Clements, and David M. Weiss. The modular structure of complex systems. *IEEE Transactions on Software Engineering*, SE-11(3):259–266, 1985. doi: 10.1109/TSE.1985.232209.

Bohdan M. Pavlyshenko. Using GPT models for qualitative and quantitative news analytics in the 2024 US presidential election process. *arXiv preprint arXiv:2410.15884*, 2024. URL https://arxiv.org/abs/2410.15884.

L. Prades, F. Romero, A. Estruch, A. García-Dominguez, and J. Serrano. Defining a methodology to design and implement business process models in bpmn according to the standard ansi/isa-95 in a manufacturing enterprise. *Procedia Engineering*, 63:115–122, 2013. ISSN 1877-7058. doi: https://doi.org/10.1016/j.proeng.2013.08.283. URL https://www.sciencedirect.com/science/article/pii/S1877705813014963. The Manufacturing Engineering Society International Conference, MESIC 2013.

Chen Qian, Wei Liu, Hongzhang Liu, et al. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.acl-long.810. URL https://arxiv.org/abs/2307.07924.

Natalia Raimondo Anselmino, Irene Gindin, Alejandro Sartorio, Franco Cellone, Verónica Aráoz, and Francisco J. Alomar. Herramientas y técnicas computarizadas al servicio de la construcción de corpus y datasets. desafíos metodológicos del análisis de discursos mediatizados en plataformas con base en internet. Capítulo de libro, Simposio de Ciencias Sociales Computacionales 2024. Manuscrito facilitado por los autores; datos editoriales pendientes de verificación, 2024.

Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *arXiv preprint arXiv:2310.10501*, 2023. URL `https://arxiv.org/abs/2310.10501`.

Pablo Rey-Mazón. *COLOR OF CORRUPTION. Visual evidence of agenda-setting in a complex mass media ecosystem*. PhD thesis, Universitat Oberta de Catalunya, 2023.

Daniel Ritter. Experiences with business process model and notation for modeling integration patterns. In Jordi Cabot and Julia Rubin, editors, *Modelling Foundations and Applications (ECMFA 2014)*, volume 8569 of *Lecture Notes in Computer Science*, pages 254–266, Cham, 2014. Springer. doi: 10.1007/978-3-319-09195-2_17. Versión técnica extendida disponible como arXiv:1403.4053.

Jim Samuel, Tanya Khanna, Julia Esguerra, Srinivasaraghavan Sundar, Alexander Pelaez, and Soumitra S. Bhuyan. The rise of artificial intelligence phobia! unveiling news-driven spread of AI fear sentiment using ML, NLP, and LLMs. *IEEE Access*, 13:125944–125969, 2025. doi: 10.1109/ACCESS.2025.3588179.

Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, Upper Saddle River, NJ, 1996. ISBN 978-0131829572.

Noah Shinn, Federico Cassano, Edward Berman, et al. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL `https://arxiv.org/abs/2303.11366`.

Somasundharam et al. Personalized news aggregation with AI: Leveraging LLMs, user data, and portfolio information for automated content creation. DiVA Portal, 2025. URL `https://www.diva-portal.org/smash/get/diva2:1978319/FULLTEXT01.pdf`. Autores secundarios pendientes de completar al acceder al texto completo.

Ah-Hwee Tan. Text mining: the state of the art and the challenges. In *Proceedings of the PAKDD 1999 Workshop on Knowledge Discovery from Advanced Databases*, pages 71–76, Beijing, China, 1999.

The Kubeflow Authors. Pipelines: Artifacts — kubeflow documentation, 2026. URL https://www.kubeflow.org/docs/components/pipelines/user-guides/data-handling/artifacts/. Consultado en junio de 2026.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems, 30*, 2017.

Lei Wang, Chen Ma, Xueyang Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. doi: 10.1007/s11704-024-40231-1. URL https://arxiv.org/abs/2308.11432.

Gian Wiher, Clara Meister, and Ryan Cotterell. On decoding strategies for neural text generators. *Transactions of the Association for Computational Linguistics*, 10:997–1012, 2022. doi: 10.1162/tacl_a_00502.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. arXiv:2307.09702 [cs.CL], 2023. URL https://arxiv.org/abs/2307.09702.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM 2024 (Conference on Language Modeling)*, 2024. URL https://arxiv.org/abs/2308.08155.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023. URL https://arxiv.org/abs/2309.07864.

Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*, 2023. URL https://arxiv.org/abs/2210.03629.

J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. Why johnny can't prompt: How non-ai experts try (and fail) to design llm prompts. In *Proceedings of the 2023 CHI Conference on Human*

Factors in Computing Systems, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394215. doi: 10.1145/3544548.3581388. URL https://doi.org/10.1145/3544548.3581388.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685.